



שפות תכנות

מדריך למידה

ד. לורנץ

20905

מהדורה זמנית
לא להפצה ולא למכירה
מק"ט 20905-5037



Author : lorenz Revision : 2.16 מהדורה 

 The Open University of Israel ©

20905 >



האוניברסיטה הפתוחה

צוות הקורס:

כותב: פרופ' דוד לורנץ, האוניברסיטה הפתוחה

עורכת: רקפת שכנר-לביא, האוניברסיטה הפתוחה

גרסת כותב

הדפסה דיגיטלית – ינואר 2017

© תשע"ז - 2017. כל הזכויות שמורות לאוניברסיטה הפתוחה. בית ההוצאה לאור של האוניברסיטה הפתוחה, הקריה ע"ש דורותי דה רוטשילד, דרך האוניברסיטה 1, ת"ד 808, רעננה 73534.

The Open University of Israel, The Dorothy de Rothschild Campus, 1 University Road, P.O.Box 808, Raanana 43537. Printed in Israel.

אין לשכפל, להעתיק, לצלם, להקליט, לתרגם, לאחסן במאגר מידע, לשדר או לקלוט בכל דרך או בכל אמצעי אלקטרוני, אופטי, מכני או אחר כל חלק שהוא מהחומר שבספר זה. שימוש מסחרי בחומר הכלול בספר זה אסור בהחלט, אלא ברשות מפורשת ובכתב ממדור זכויות יוצרים של האוניברסיטה הפתוחה.

תוכן עניינים

11	רשימת איורים
13	רשימת טבלאות
17	פתח דבר
21	I עקרונות שפות תכנות
23	1 חומר הלימוד
23	1.1 תיאור קטלוגי
24	1.2 דרישות קדם
24	1.3 ספר הלימוד
25	1.4 מטלות
25	1.5 חבילת התוכנה
25	1.6 כיצד לקרוא את חומר הלימוד
26	1.7 ניתן בהם סימנים
27	1.8 הכרת תודה
29	2 מבוא

29	...	2.1	מדוע כדאי ללמוד שפות תכנות ¹
31	...	2.2	כיצד נלמד לעצב שפת תכנות
33		3	קבוצות נתונים אינדוקטיביות
33	...	3.1	מבנים המאופיינים רקורסיבית
36	...	3.2	אינדוקציה מבנית
37	...	3.2.1	אשור ריקנות
37	...	3.2.2	חישוב אורך של רשימה
38	...	3.2.3	משתנה חופשי
41	...	3.3	סיכום
41	...	3.3.1	רשימת מונחים (בסדר מלוני)
42	...	3.3.2	שגרות שמוזכרות בפרק (בסדר מלוני)
44	...	3.3.3	מקורות לקריאה נוספת
45		4	הפשטת נתונים
45	...	4.1	אפיון מבנה נתונים באופן אלגברי
46	...	4.2	קבוצת המספרים הטבעיים
47	...	4.2.1	אפיון אלגברי
48	...	4.2.2	פעולות אריתמטיות
51	...	4.2.3	ייצוג אונרי
52	...	4.2.4	ייצוג בינארי
54	...	4.2.5	ייצוג בבסיס 3
55	...	4.2.6	ייצוג בבסיס כלשהו
55	...	4.2.7	ייצוג בקידוד Church
59	...	4.3	סביבת משתנים מופשטת

¹פרק זה אינו מופיע בספר הלימוד

59	4.3.1	אפיון אלגברי
61	4.3.2	ייצוג באמצעות מבנה נתונים מופשט
64	4.3.3	ייצוג באמצעות פונקציה
66	4.4	תחביר מופשט לטיפוס נתונים רקורסיבי
69	4.4.1	ביטויים בתחשיב למדא
69	4.5	תחשיב למדא כטיפוס נתונים מופשט
70	4.5.1	בנאים
71	4.5.2	מנבאים
72	4.5.3	מחלצים
74	4.6	משתנה חופשי?
75	4.7	כלים להגדרת טיפוס נתונים רקורסיבי
75	4.7.1	מנגנון להגדרת טיפוס נתונים
76	4.7.2	מנגנון לבחינת מרכיביו של טיפוס נתונים
77	4.8	המרה בין ייצוג מוחשי לייצוג מופשט
79	4.9	סיכום
79	4.9.1	רשימת מונחים (בסדר מלוני)
80	4.9.2	שגרות שמוזכרות בפרק (בסדר מלוני)
80	4.10	תרגילים נוספים

5 ביטויים 91

91	5.1	אסטרטגיה לאפיון ומימוש של שפות
92	5.2	"וִיֵהִי": שפת תכנות פשוטה (LET)
93	5.2.1	תחביר השפה "וִיֵהִי"
94	5.2.2	אפיון קבוצת הערכים
97	5.2.3	אפיון ההתנהגות של ביטויים

98	דוגמה של שערוך ביטוי	5.2.4
100	אפיון התנהגות של תכניות	5.2.5
101	אפיון התנהגות של יתר הביטויים	5.2.6
103	הביטוי if	5.2.7
103	הביטוי let	5.2.8
104	מימוש שפת "ויהי"	5.2.9
111	בדיקת המימוש	5.3
113	"וישגור": שפה עם שגרות (PROC)	5.4
113	תחביר שפת "וישגור"	5.4.1
115	דוגמה לתכניות בשפת "וישגור"	5.4.2
116	אפיון קבוצת הערכים	5.4.3
117	אפיון התנהגות של שגרות	5.4.4
118	דוגמה לשערוך שגרה	5.4.5
121	ייצוג שגרות	5.4.6
125	"וייסוג": שפה עם שגרות רקורסיביות	5.5
125	תחביר שפת "וייסוג"	5.5.1
127	אפיון של ביטויי letrec	5.5.2
128	דוגמה לשערוך שגרה רקורסיבית	5.5.3
131	תחימה, כריכה, והיקף	5.6
132	חילוץ שמות של משתנים	5.6.1
133	כיתוב מילולי (לקסיקלי)	5.6.2
134	תרגילים נוספים	5.7
141		6 המצב
141	תוצא חישובי	6.1

142	 "ויפנה": שפה עם הפניות מפורשות	6.2
144	 אפיון מבוסס העברת־החסן	6.3
145	 אפיון אלגברי של שלושה סוגי ביטויים	6.3.1
145	 6.3.1.1 פעולה עם תוצא	
145	 6.3.1.2 פעולה ללא־תוצא	
146	 6.3.1.3 פעולות עם תוצא סדרתי	
146	 אפיון פעולות על הפניות מפורשות	6.3.2
146	 הגדרת פעולות על הפניות המפורשות	6.3.3
148	 מימוש	6.3.4
152	 "וירמוז": שפה עם הפניות מרומזות (IMPLICIT-REFS)	6.4
160	 "ויבגוד": שפה עם חילופי זוגות (MUTABLE-PAIRS)	6.5
160	 ערכים	6.5.1
161	 העברת פרמטרים	6.6
162	 תרגילים נוספים	6.7
167	טיפוסים	7
169	 הגדרת בטיחות טיפוסים	7.1
171	 קשירת טיפוס לביטוי	7.2
173	 תכניות מוטפסות־היטב בשפת "וישגור"	7.3
174	 נאותות טיפוסים	7.4
174	 7.4.1 אלגוריתם עבור שפת "ויהי"	
175	 7.4.2 אלגוריתם עבור שפת "וישגור"	
176	 "ויבדוק": שפה עם טיפוסים מוצהרים	7.5
177	 7.5.1 תחביר השפה "ויבדוק"	
178	 "ויקיש": שפה שמקישה טיפוסים	7.6

179	אלגוריתם להקשת טיפוסים עבור תכניות בשפת "וייסוג"	7.6.1
179	אלגוריתם לחילול אילוצי טיפוס עבור שפת "וייסוג" . .	7.6.2
181	דוגמה	7.6.3
181	פתרון מערכת האילוצים	7.6.4
197	תרגילים נוספים	7.7

8 פרדיגמות תכנות 201

201	תכנות אימפרטיבי	8.1
202	תכנות פונקציונלי	8.2
203	תכנות לוגי	8.3
204	תכנות מונחה עצמים	8.4
205	סיכום	8.5

סוף דבר 209

II מדריך לשפת Scheme 211

9 מדוע שפת Scheme? 213

213	דרישות משפת תכנות	9.1
215	נימוקים כבדי משקל	9.2
216	הכרת תודה	9.3

10 פגישה עם סביבת הפיתוח Racket 217

217	תפעול בסיסי של Racket	10.1
218	הרצת תכנית פשוטה	10.2

221 **11 פגישה עם שפת Scheme**

221 11.1 צורות מיוחדות וצורת הפעלה

222 11.2 תחביר של ביטויים

224 11.3 סמנטיקה

225 11.4 בדיקת תחביר

227 **12 המפרש של Scheme**

227 12.1 המפרש

230 12.2 רשימות ורקורסיה

235 12.2.1 ביטוי Cond

239 **13 קריאה נוספת**

239 13.1 כותרים של האו"פ

239 13.2 ספרות מומלצת ומקורות מקוונים

240 13.3 מקורות נוספים ללימוד Scheme

241 **נספחים**

243 פתרונות לתרגילים נבחרים

263 מפתח מושגים

269 מפתח קוד

277 תיקון טעויות בספר הלימוד

רשימת איורים

19	 בלי מילים	1
26	 תלויות בין פרקים בספר הלימוד	1.1
31	 תהליך הפירוש של תכנית	2.1
68	 מבנה מופשט	4.1
68	 עץ גזירה	4.2
218	 RacketRacket סביבת הפיתוח	10.1

רשימת טבלאות

93	תחביר השפה "וִיָּהִי"	5.1
113	תחביר שפת "וִישְׁגוֹר"	5.2
126	תחביר שפת "וִייסוֹג"	5.3
147	תחביר שפת "וִיפְנָה"	6.1
177	תחביר שפת "וִיבְדוֹק"	7.1
181	משתנה טיפוס לכל משתנה קשור	7.2
182	משתנה טיפוס לכל תת־ביטוי	7.3
183	מערכת אילוצים	7.4
184	מערכת אילוצים (המשך)	7.5
185	מערכת אילוצים ממוינת	7.6
186	מערכת אילוצים ממויינת (המשך)	7.7
187	מערכת אילוצים מצומצמת	7.8
188	מערכת אילוצים מצומצמת (המשך)	7.9
190	צמצום נוסף של מערכת האילוצים	7.10
191	צמצום נוסף של מערכת האילוצים (המשך)	7.11
192	מערכת אילוצים מעודכנת	7.12
193	מערכת אילוצים מעודכנת (המשך)	7.13

194	מערכת אילוצים לפני האחדה	7.14
194	מערכת האילוצים לאחר האחדה	7.15
195	מערכת האילוצים לאחר פישוט	7.16
195	מערכת האילוצים הסופית	7.17
196	מערכת האילוצים הסופית (המשך)	7.18

פתח דבר

בקורס שפות תכנות נלמדים מושגי יסוד במימושן של שפות תכנות מודרניות. ישנן שתי אסכולות מקובלות באקדמיה להוראת קורס בשפות תכנות: גישה השוואתית וגישת המפרשים.

גישה השוואתית – בגישה זו המורה סוקר כמה שפות תכנות מפרדיגמות שונות, והתלמיד נדרש לכתוב תכניות פשוטות בכל אחת מאותן שפות. למשל, מקובל ללמוד ביסודיות שפה מהפרדיגמה הפונקציונלית. יתרון הגישה הוא בכך שהתלמיד מתנסה בתכנות בפרדיגמות השונות ולומד את נושאי הקורס מנקודת ראותו של משתמש בשפה. חסרון הגישה הוא בכך שמידת חשיפתו של התלמיד לנקודת ראותו של מעצב השפה היא מועטה.

גישת המפרשים – בגישה זו נושאי הקורס נלמדים על-ידי כתיבה של תוכניות פרוש (interpreters, להלן מפרשים) המיישמים שפות תכנות פשוטות בעלות תכונות שונות. יתרון הגישה הוא בכך שהמפרשים משקפים את המבנים הסמנטיים של שפת התכנות הממומשת. התלמיד יכול לשנות את תוכנית הפרוש (להלן המפרש) ולבדוק כיצד שינויים בהחלטות עיצוב השפה משפיעים על התנהגותה. בגישה זו התלמיד נחשף תוך התנסות למושגי היסוד התיאורטיים של שפות תכנות. התלמיד גם נחשף ישירות לפרדיגמה הפונקציונלית, הואיל ומקובל לממש את המפרשים בשפת תכנות מפרדיגמה זו. חסרון הגישה הוא בכך

Revision : 2.16.00

שהשפות המיושמות מתאימות רק לצרכי לימוד.

גישה דידקטית

באוניברסיטה הפתוחה הקורס 20905 - שפות תכנות מאמץ את גישת המפְרָשִׁים, משני טעמים. ראשית, הגישה מתאימה יותר ללימוד עצמי ולסוג ההוראה באוניברסיטה הפתוחה. שנית, זו הגישה המועדפת לקורס מתקדם בשפות תכנות.

רגע של עברית

מלאכת הכתיבה הטכנית בעברית כרוכה בקושי לתרגם מונחים מקצועיים כהלכה. לא לכל מושג יש מינוח עברי מוצלח או מקובל. למשל, שני מונחים שנזדקק להם כבר בפרקים הראשונים הם "top-down" לעומת "bottom-up". בתרגום מילולי נקבל "מהעלית כלפי מטה" לעומת "מהתחתית כלפי מעלה". אולם הרישא מקורה בפתגם "from top to bottom" שבתרגום חופשי משמעו "מן המסד עד הטיפחות". לפיכך, נעדיף כאן את התרגום היצירתי, "מן המסד מעלה" לעומת "מהטיפחות מטה", על פני המינוח התקני. גם לאקדמיה ללשון עברית לא תמיד הפתרונים. מלאכת הכתיבה קשה; מלאכת הקריאה קשה שבעתיים. סבלנות הקורא והצעות לשיפור יתקבלו בברכה.²

תרגילים

בכל פרק של המדריך תמצאו מספר תרגילים, חלקם עם פתרונות מלאים. להטמעת החומר מומלץ לפתור תרגילים אלה. מגוון תרגילים נוספים מובאים בספר הלימוד.

²כתובת דוא"ל להערות, תיקונים והצעות לשיפור: lorenz+20905@cs.openu.ac.il



איור 1: בלי מילים

להלן תרגיל דוגמה.

★**תרגיל 0.1** לפי האקדמיה ללשון עברית, כיצד יש לתרגם bottom-up ו-top-down? סמנו את התשובה הנכונה ביותר:

(א) מהתחתית-מעלה; מהעלית-מטה

(ב) מלמטה-מעלה; מלמעלה-מטה

(ג) מן המסד-מעלה; מהטפחות-מטה

(ד) מכף-הרגל-מעלה; מהראש-מטה

(ה) מבירא-מעלה; מאיגרא-מטה

(ו) מהעכוז-מעלה; מהחזה-מטה

✍️ **פתרון:** הפתרון מובא בעמוד 243 יחד עם פתרונות לתרגילים נבחרים אחרים.

□

מקורות לקריאה נוספת

- אלי גיא. (2007). שימוש לשוני על פי תקן או יצירתיות. מאגר המאמרים הישראלי.

Revision : 2.16.00

בהצלחה!

לא נותר אלא לאחל לכם הצלחה והנאה מהקורס. בשם צוות הקורס אני תקווה שתפיקו מקורס זה את המרב.

פרופ' דוד לורנץ

רעננה, 2017

יחידה I

עקרונות שפות תכנות

פרק 1

חומר הלימוד

מדריך למידה זה מלווה את הקורס 20905 – שפות תכנות, המקנה ארבע נקודות זכות בלימודי הנדסת תוכנה ומדעי המחשב באוניברסיטה הפתוחה. מטרת מדריך הלמידה היא להקל על הסטודנט את הלימוד העצמי של ספר הלימוד ולסייע בהבנתו. המדריך מפנה את הלומד לפרקים ולסעיפים הרלוונטיים בספר, מתמצת את עיקרי הדברים ומציג שאלות מנחות. המדריך כולל גם תרגילים נבחרים ומספק פתרונות לחלקם.

1.1 תיאור קטלוגי

הבנה תיאורטית ומעשית של מושגי יסוד במימושן של שפות תכנות מודרניות: ערכים, מבני נתונים מופשטים, זיכרון, מבני בקרה, כריכת משתנים, מערכות טיפוסים, מודולים ועצמים. מבוא לסמנטיקה פורמלית של שפות תכנות באמצעות מימוש מפרשים בשפה פונקציונלית.

Revision : 2.16.01

1.2 דרישות קדם

הקורס אינו מצריך ידע קודם בתכנות פונקציונלי. אולם, נדרשת יכולת למידה מהירה של סביבת תכנות חדשה. רצויה גם הכרה של לפחות שתי שפות תכנות. ידע קודם דרוש: מבני נתונים ומבוא לאלגוריתמים (20407), ותכנות מונחה עצמים (20906) [או הנדסת תוכנה: הפשטה ועיצוב מונחה עצמים (20442)]. ידע קודם מומלץ: אוטומטים ושפות פורמליות (20440), וקומפילציה (20364). קהל היעד: תלמידי תואר ראשון במדעי המחשב במסלול להנדסת תוכנה, ותלמידי תואר ראשון במדעי המחשב בכלל.

1.3 ספר הלימוד

ספר הלימוד הוא:

Friedman, D. and Wand, M. (2008). Essentials of Programming Languages, 3rd Edition. The MIT Press.

ספר זה נוקט את גישת המפרשים. הוא אחד מספרי הלימוד הקלאסיים בתחום, ומשמש הן בקורסי הסמכה והן בקורסי מוסמכים באוניברסיטאות רבות בעולם. הספר מציג סדרת מפרשים החושפים בהדרגה את מרכיבי המימוש הפנימיים של שפות תכנות. במהלך התצוגה מתבררת השפעתן של החלטות עיצוב על כוחה של השפה, על מידת הקושי של המימוש ועל יעילותו. הספר כתוב באופן בהיר, כולל דוגמאות הרצה רבות ומתאים ללימוד עצמי.

דוגמאות הקוד בספר הן רובן ככולן בשפת Scheme, שנלמדת במסגרת הקורס. יש לקרוא תחילה את המדריך לשפת Scheme ורק לאחר מכן את המדריך הזה.

1.4 מטלות

הקורס כולל מטלות תכנות רבות. הקורס מציג את המושגים באמצעות מִפְרָשִׁים בשפת Scheme הניתנים להרצה, שאותם צריך התלמיד להבין היטב על מנת לשנותם. ביצוע שינויים במִפְרָשִׁים וכתובת תכניות עבור המִפְרָשִׁים הם מרכיב מרכזי בלימוד. תהליך הלמידה מתרחש באמצעות קריאת הקוד, שינויו והרצתו. מימוש מִפְרָשׁ בשפת Scheme פשוט יותר ומהיר בהרבה ממימוש, למשל, בשפת C או בשפת Java. דבר זה מאפשר למקד את תשומת לב לסמנטיקה של שפת התכנות הממומשת במקום לפרטים הטכניים של התחביר. מימוש מִפְרָשִׁים בשפת Scheme הוא כיום משימה שגרתית במסגרת מחקר בשפות תכנות.

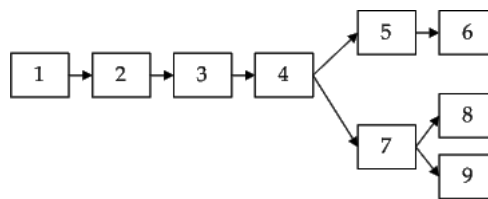
1.5 חבילת התוכנה

התוכנה המשמשת בקורס היא סביבת העבודה Racket (לשעבר Dr. Scheme, המיועדת לתכנות ב־Scheme). הכרות עם שפת התכנות Scheme וסביבת הפיתוח שלה היא הכרחית לביצוע המטלות. הדוגמאות במהדורה השלישית של הספר עודכנו והותאמו במיוחד לעבודה בסביבה זו. על מנת להקל את לימוד השפה ואת השימוש בסביבת העבודה, אתר הקורס כולל חומרי עזר נלווים ללימוד, כגון תיאור התחביר, הספריות, מדריך למשתמש, התיעוד הנלווה לתוכנה, דוגמאות תכנות, בכלל זה מימוש המִפְרָשִׁים עבור השפות המוצגות בספר.

1.6 כיצד לקרוא את חומר הלימוד

מטרת מדריך למידה זה היא לעזור לתלמיד להטמיע את חומר הלימוד בקורס. המדריך דן בנושאים המוצגים בספר תוך מתן דגשים שונים והפניה לסעיפים המתאימים. במדריך

Revision : 2.16.01



איור 1.1: תלויות בין פרקים בספר הלימוד

תמצאו הן הנחיות לגבי חומר הקריאה והן הסברים, שאלות לתרגול וכן דוגמאות נוספות על אלה שבספר. אך המדריך אינו מהווה תחליף לקריאת הספר. מומלץ וחיוני אפוא לשלב את קריאת שני המקורות.

מומלץ להתחיל את תהליך הלימוד בקריאת מדריך הלימוד (להלן "המדריך"). בדרך כלל יפנה אתכם המדריך לקריאת סעיפים בספר הלימוד (בהמשך נכנה אותו בקיצור "הספר"), לאחר מכן ידון המדריך בחומר הלימוד המוגש בספר, תוך תמצות החומר, והוספת דוגמאות. במקרים אחדים הוספו נושאים שאינם נכללים בספר, במטרה להרחיב ולפרט בהם יותר.

איור 1.1 מראה את תרשימים התלויות בין הפרקים בספר הלימוד. חומר הלימוד כולל את פרקים 1, 2, 3, 4, 7, ו-9. פרקים 5, 6, ו-8 הם חומר מתקדם, שניתן ללמוד לאחר הקורס במסגרת סדנה בשפות תכנות או במסגרת סמינר קבוצתי.

◆ מספרי הפרקים במדריך לא תואמים בהכרח למספרי הפרקים בספר, ומספור הסעיפים לעולם אינו תואם, שלא במידה. ◆

1.7 ניתן בהם סימנים

שימו לב, המדריך מכיל הפניות הן לחומרים שמופיעים במדריך והן לחומרים שמופיעים בספר. ההפניה לקוד היא בדרך כלל למדריך, והפניה לסעיף, עמוד, או תרגיל מסויים

היא בדרך כלל לספר, אלא אם כן מצוין אחרת. בהפניות למדריך מושמטת לעתים המילה "מדריך". לדוגמה, בהפניה לקוד 9.9 במדריך יירשם "ראה קוד 9.9" (וכאן הכוונה למדריך הלמידה). בהפניות לספר, או שמצוין "הספר" במפורש או שנעשה שימוש בסימון מוסכם. לדוגמה, בהפניה לסעיף 8.8 בעמוד 77 בספר יירשם "ראה סעיף 8.8 בספר" או בקיצור "ראה §8.8" או בקיצור נמרץ "עמוד 77" (וכאן הכוונה לספר הלימוד).

במדריך נעשה שימוש בסימונים אלה:

-  - הפניה לספר הלימוד - יש לקרוא בספר את הפרקים המתאימים.
- § - הפניה לסעיף מסוים - יש לעיין בסעיפים המצוינים.
- * - תרגילים - להטמעה של החומר.
-  - פתרונות לחלק מהתרגילים, המובאים בגוף המדריך ובסופו.
-  - רמזים והנחיות.
- ◆ - הערות.
-  - הארות.
-  - חומר העשרה (לא למבחן).
-  - חומר האשרה (יופיע במבחן ; -).

1.8 הכרת תודה

חומר הלימוד מבוסס בחלקו על הרצאות של קורס בשפות תכנות שלימדתי (בטכניון בגישה ההשוואתית ובארה"ב בגישת המפגשים על-פי אותו ספר הלימוד), שהתבססו, בין היתר, על הרצאות של עמיתיי: יוסי גיל, עמיר תומר, וויל קלינגר, ומיטשל וונד. עיקר החומר במדריך הוא תרגום, פעמים מתומצת, פעמים מורחב, של ספר הלימוד.

פרק 2

מבוא

2.1 מדוע כדאי ללמוד שפות תכנות¹

במה מועיל לימוד שפות תכנות למהנדס התוכנה? נימנה שלוש סיבות: ראשית, בעבודה השוטפת מתעורר לעתים הצורך לעצב ולממש שפת תכנות, או שפה שדומה לשפת תכנות. למשל, כתיבת תכנית לעיבוד נתונים דומה מבחינות מסוימות לכתיבת מפרש לשפת תכנות פשוטה.

שנית, לימוד שפות תכנות מקנה הבנה מעמיקה של ההבדלים בין שפות התכנות השונות, ועל כן מסייע בבחירת שפת התכנות המתאימה.

שלישית, מחקרים מראים שלימוד עקרונות של שפות תכנות משפר את מיומנות התכנות. תופעה דומה מוכרת בתחום לימוד השפות הטבעיות. לימוד עקרונות הדקדוק והלשון העברית משפר את יכולת ההבעה בכתב ובעל-פה. יתרה מזו, לימוד המבנה של שפות זרות משפר את השליטה בשפת האם.

★**תרגיל 2.1** אילו שפות טבעיות אתם מבינים?

★**תרגיל 2.2** באילו מהן אתם מסוגלים לנהל שיחה ובאילו מהן לקרוא

¹פרק זה אינו מופיע בספר הלימוד

ולכתוב?

★תרגיל 2.3 אילו שפות תכנות אתם מכירים?

★תרגיל 2.4 אילו מהן אתם מסוגלים להבין ובאילו מהן לתכנת?

★תרגיל 2.5 במה נבדלת שפת תכנות משפה טבעית?

★תרגיל 2.6 במה נבדלת שפת תכנות משפת תיאור, כגון XML?

על אף שאינכם אמורים בשלב זה להכיר את המושגים, נסו לבחון שפת תכנות שאתם מכירים תוך התייחסות לשאלות האלה:

- מה הם הערכים (values) בשפה? אילו פעולות מותר לבצע על ערכים אלה?
- מה הם הטיפוסים (types) בשפה?
- מה הם חוקי התחימה (scoping) וכיצד משתנים נכרכים (binding)? איך משתמשים במשתנים? אילו משתנים הם בתחום ההכרזה ומתי?
- מה הם התוצאים (אפקטים effects) בשפה? האם מותרים תוצאי־לוואי (side effects)? האם ה"עולם" מושפע מהתוצאים בתוכנית? האם תכניות יכולות להשפיע על תכניות אחרות? האם הרצת תכנית יכולה שלא להסתיים?
- מה הם המאפיינים הסטטיים של השפה? מה ניתן לחזות על התנהגות התכנית ללא ידיעת הערכים של זמן־ריצה?

במהלך הקורס נלמד להקשות קושיות כגון אלה ועל חלקן אף לענות!



איור 2.1: תהליך הפירוש של תכנית

2.2 כיצד נלמד לעצב שפת תכנות

בכדי לממש שפת תכנות נדרש לתאר שני דברים בסיסיים: את התחביר (מבנה) ואת הסמנטיקה (משמעות, התנהגות) של השפה. לשם כך, נידרש להגדיר טיפוסים נתונים: עץ תחביר מופשט, סביבות, שגרות. אנו נידרש גם למושגי היסוד הללו: דקדוק (פרק 3), תחביר מופשט, מזהים, ערכים, סביבות (פרק 4), ביטויים, משתנים, תחימה, כריכה והיקף (פרק 5), החסן לערכים (פרק 6) והחסן לטיפוסים (פרק 7), תרגום, ופירוש.

הקורס מתמקד במפרש. נתבונן בתהליך ההרצה של תכנית מחשב (program) המתואר באיור 2.1. תחילה הקוד נקרא, מנותח ומיוצג במבנה תחבירי מפשט (ab-stract syntax). אחרי כן המבנה התחבירי מעובד על-ידי מפרש (interpreter), לפעמים תוך כדי ביצוע פעולות קלט ופלט. לבסוף מתקבלת תשובה (answer). תיאור התהליך האמור ניתן לעידון, למשל על-ידי פיצול המנתח הקדמי לסורק לקסיקלי (scanner) ולנתח תחבירי (parser) או על-ידי הכללת מהדר (compiler) בתהליך. נושאים אלה נלמדים בהרחבה בקורס קומפילציה.

🔗 בקורס 20905 - שפות תכנות, נניח בדרך כלל שהתכנית מוצגת כבר במבנה התחבירי המופשט ונתעמק בעיקר בהבנת פעולת המפרש.



פרק 3

קבוצות נתונים אינדוקטיביות

פרק זה עוסק בקשר שבין הגדרות אינדוקטיביות ודדוקטיביות לבין תכניות רקורסיביות.

פרק זה מלווה את פרק 1 בספר הלימוד. שימו לב: חיוני לקרוא את חומר הלימוד בספר. מדריך הלמידה אינו תרגום מלא של הפרק אלא חומר עזר בלבד.

3.1 מבנים המאופיינים רקורסיבית

קראו את סעיף §1.1 בספר הלימוד.

ניתן לאפיין קבוצות או מבנים רקורסיביים בכמה דרכים:

- הגדרה מן המסד מעלה (bottom-up)
- הגדרה מהטפחות מטה (top-down)
- הגדרה באמצעות כללי היסק (rule of inference)

Revision : 2.16.03

• הגדרה באמצעות דקדוק (grammar)

דוגמה פשוטה מעולם המתמטיקה היא קבוצת המספרים הטבעיים:

$$N = N_0 = 0, 1, 2, \dots$$

בספר מובאת הגדרה של N בארבעת אופני ההגדרה האמורים. יש לשים לב שקבוצת המספרים הטבעיים מוגדרת בספר כקבוצת השלמים האי־שליליים, לרבות האפס.¹ דוגמה מעולם מבני הנתונים היא המבנה "רשימה־של־שלמים" (list-of-integers), שמוגדר כאן באמצעות דקדוק:

$$\begin{aligned} LoN &::= () \\ &::= (Int.LoN) \end{aligned}$$

ובכתיב מקוצר:

$$LoN ::= () \mid (Int.LoN)$$

או בקיצור נמרץ:

$$LoN ::= (\{Int\}^*)$$

¹במתמטיקה יש המקפידים שלא לכלול את המספר אפס בקבוצת המספרים הטבעיים, אין אפס.

♦ יש לשים לב שציון הרשימה הריקה נעשה על-ידי "(). הביטוי "(h.t)" שקול לביטוי "(cons h t)", וציון "מארוז" (צמד, זיאה) אשר ראשו (המכונה car) הוא h, וזנבו (המכונה cdr) הוא t. בפרט, הביטוי "(h.())" שקול לביטוי "(h)", וציון רשימה ובה האיבר היחיד h. ♦

ניגש עתה להגדרות הבסיסיות (הגדרה 3.1) של $S\text{-List}$, $S\text{-Exp}$, ו- $\lambda\text{-Exp}$, אשר יישמשו אותנו בהמשך. בחנו את עצמכם אם ההגדרות הללו נהירות לכם ופתרו את התרגילים 3.1 ועד 3.5 במדריך.

$$\begin{aligned}
 S\text{-List} &::= (\{S\text{-Exp}\}^*) \\
 S\text{-Exp} &::= \text{Symbol} \mid S\text{-List} \\
 \lambda\text{-Exp} &::= \text{Identifier} \\
 &::= (\text{lambda } (\text{Identifier}) \lambda\text{-Exp}) \\
 &::= (\lambda\text{-Exp } \lambda\text{-Exp})
 \end{aligned}
 \tag{3.1}$$

★תרגיל 3.1 תרגילים 1.1, 1.2, 1.3, בעמוד 5 בספר.

★תרגיל 3.2 רשמו הגדרה של המבנה הקרוי "רשימה" (List) בשפת Scheme בארבעת האופנים:

1. מהמסד-מעלה (bottom-up),
2. מהטפחות-מטה (top-down),
3. באמצעות כללי היסק (rule of inference),
4. ובאמצעות דקדוק (grammar).

רמז: (רמז לתרגיל 3.2): $List ::= () \mid (\text{SchemeValue}.List)$

★תרגיל 3.3 בהגדרות של $S\text{-Exp}$ ו- $\lambda\text{-Exp}$, מהו `lambda`, מהו `Symbol`, ומהו `Identifier`?



רמז: (תרגיל 3.3) ראו הגדרה 1.1.8 בעמוד 9 בספר.

★תרגיל 3.4 האם $(\lambda (\lambda (\lambda) \lambda)) \in \lambda\text{-Exp}$?

★תרגיל 3.5 האם $?x \in \lambda\text{-Exp}$

3.2 אינדוקציה מבנית

קראו את סעיפים §1.1.3, §1.2 ו-§1.3 בספר הלימוד.

אופן ההגדרה הרקורסיבית שימושי הן בהוכחת משפטים באינדוקציה על מבני הנתונים והן בכתיבת שגרות רקורסיביות המטפלות בהם. באינדוקציה מבנית מוכיחים תחילה את נכונות הטענה במבנים הפשוטים, שאין בהם תת־מבנים. לאחר מכן מוכיחים שאם הטענה נכונה בתת־מבנים, אזי היא נכונה גם במבנה המורכב. למשל, ניתן להוכיח באינדוקציה מבנית כי אם $e \in \lambda\text{-Exp}$, אזי יש ב־ e מספר שווה של סוגריים פותחים וסוגרים (נשאיר את ההוכחה כתרגיל לקורא). בדומה לכך בכתיבת קוד המממש שגרה רקורסיבית, יש לדעת במדויק אילו ערכים יכולים להתקבל כארגומנטים לשגרה ואילו ערכים מותר לשגרה להחזיר.

המבנה האינדוקטיבי מקל את הכתיבה של שגרות רקורסיביות: ניתן לעצב את השגרה בצלם ההגדרה, באופן שמבנה השגרה משקף את מבנה הגדרה.

★תרגיל 3.6 הוכיחו כי אם $e \in \lambda\text{-Exp}$, אזי יש ב־ e מספר שווה של סוגריים פותחים וסוגרים.

3.2.1 אִשּׁוּשׁ רִיקָנוּת

דוגמה פשוטה לשגרה המעוצבת בצלם ההגדרה היא השגרה

$$\text{null?} : \text{List} \rightarrow \text{Bool}$$

שמחזירה $\#t$ אם הרשימה היא ריקה, ו- $\#f$ אחרת.

✦ שימו לב כי לא זו הלבד שהטו סימך-שאלה (?) יכול להיות כאחד התווים בשם מזהה של שורה, אלא זה אף הנוהג המקובל כאשר השורה מממשת מנבא (פרז'יקט) המחזיר $\#t$ או $\#f$. ✦

3.2.2 חישוב אורך של רשימה

דוגמה נוספת לשגרה המעוצבת בצלם ההגדרה היא השגרה

$$\text{list-length} : \text{List} \rightarrow N$$

לחישוב אורך של רשימה. אם הרשימה שאורכה נדרש היא $()$, דהיינו הרשימה הריקה, אזי האורך המוחזר הוא 0. ולא, הרשימה היא בהכרח מהצורה $(h.t)$, והאורך המוחזר הוא המספר הטבעי העוקב של $(\text{list-length } t)$, דהיינו אורך הרשימה t ועוד אחד (ראה §1.2.1 בספר). את המימוש ניתן למצוא בקוד 3.1:

```
;; list-length : List -> Int
;; usage: (list-length l) = the length of l
;; Page: 14
(define list-length
  (lambda (lst)
    (if (null? lst)
        0
        (+ 1 (list-length (cdr lst))))))
```

Listing 3.1: list-length.scm

✦ כדאי לשים לב לסגנון הכתיבה המקובל.
 השורה הראשונה היא הערה שמתעצת את החתימה (signature) של
 השורה: התאמת רשימה למספר שלם. השורה השנייה מתעצת את
 המשתמש של השורה: המספר השלם המוחזר הוא אורך הרשימה.
 ✦ ההצחה (indentation) המקובלת לשורות קוד היא של שני תווי רווח.

★תרגיל 3.7 מה אורך הרשימה (a (b c) d) ?

★תרגיל 3.8 בדקו את תשובתכם על-ידי הרצת:

```
(list-length '(a (b c) d))
```

שימו לב לשימוש שנעשה כאן בגרש הבודד (quote). מה קורה אם

משמיטים אותו?

3.2.3 משתנה חופשי

דוגמה מורכבת יותר היא השגרה

$\text{occurs-free?} : \text{Sym} \times \lambda\text{-Exp} \rightarrow \text{Bool}$

אשר לוקחת משתנה k , המיוצג כ- Sym , וביטוי e המיוצג כ- $\lambda-Exp$, מחזירה $\#t$ אם k הוא משתנה המופיע חופשי ב- e , ואחרת $\#f$.

 הגדרה: משתנה k מופיע חופשי ב- e אם יש מופע של k ב- e שלא כרוך בביטוי λ כלשהו (ראו §1.2.4 בספר). 

בחנו את עצמכם אם ההגדרה הזו נהירה לכם באמצעות התרגילים האלה:

★תרגיל 3.9 אמת או שקר? $(occurs-free? \ 'x \ 'x)$

★תרגיל 3.10 אמת או שקר? $(occurs-free? \ 'x \ 'y)$

★תרגיל 3.11 אמת או שקר? $(occurs-free? \ 'x \ '(\lambda (x) (x \ y)))$

★תרגיל 3.12 אמת או שקר? $(occurs-free? \ 'x \ '(\lambda (y) (x \ y)))$

★תרגיל 3.13 אמת או שקר?

$(occurs-free? \ 'x \ '(\lambda (x) (x) (x \ y)))$

★תרגיל 3.14 אמת או שקר?

$(occurs-free? \ 'x \ '(\lambda (y) (\lambda (z) (x \ (y \ z))))$

★תרגיל 3.15 ממש את $occurs-free?$ על-פי ההגדרה (עמוד 38 במדריך).

 פתרון: (תרגיל 3.15)

הבדיקה האם משתנה מופיע חופשי נשמעת מסובכת למימוש. אולם המימוש דווקא פשוט, כל עוד מקפידים שמבנה השגרה משקף את מבנה ההגדרה. כזכור (3.1) ההגדרה האינדוקטיבית של $\lambda-Exp$ היא:

$\lambda-Exp ::= Identifier$

$::= (\lambda (Identifier) \lambda-Exp)$

$::= (\lambda-Exp \ \lambda-Exp)$

Revision : 2.16.03

נדון בכל מקרה לחוד:

• כאשר $e = id \in Identifier$ אזי:

k מופיע חופשי ב־ e אם ורק אם $e = k$.

• כאשר $e = (\text{lambda } (y) e')$ אזי:

k מופיע חופשי ב־ e אם ורק אם y שונה מ־ k , ו־ k מופיע חופשי ב־ e' .

• כאשר $e = (e_1 e_2)$ אזי:

k מופיע חופשי ב־ e אם ורק אם k מופיע חופשי ב־ e_1 או ב־ e_2 (בשניהם).

עמ' 19

עתה המימוש הוא מידי ומובא בקוד 3.2:

```
;; occurs-free? : Sym * Lcexp -> Bool
;; usage:
;; returns #t if the symbol var occurs free in exp,
;; otherwise returns #f.
;; Page: 19
(define occurs-free?
  (lambda (var exp)
    (cond
      ((symbol? exp) (eqv? var exp))
      ((eqv? (car exp) 'lambda)
       (and
        (not (eqv? var (car (cadr exp))))
        (occurs-free? var (caddr exp))))
      (else
       (or
        (occurs-free? var (car exp))
        (occurs-free? var (cadr exp)))))))
```

Listing 3.2: occurs-free.scm

□

♦ שימו לב לעימול שצורה כאן ב- `cadr`, `cddr`, `caddr`. אלה הם קיצורים שילויים:

```
(cadr exp) = (car (cdr exp))
(cddr exp) = (cdr (cdr exp))
(caddr exp) = (car (cdr (cdr exp)))
```

ככלל, הסדר בו "a" ו-"a" מופיעים משמאל לימין בין "c" לבין "r" קובע בהתאמה את הסדר בו "car" ו-"cdr" מופיעים.

★ **תרגיל 3.16** תרגיל 1.5 בעמוד 12.

★ **תרגיל 3.17** פתרו שלושה מתוך התרגילים המסומנים ב-* ב- § 1.4

★ **תרגיל 3.18** פתרו שניים מתוך התרגילים המסומנים ב-** ב- § 1.4

★ **תרגיל 3.19** פתרו אחד מתוך התרגילים המסומנים ב-*** בעמוד 30 בספר.

3.3 סיכום

יחידה זו דנה בקבוצות נתונים אינדוקטיביות. עם סיום לימוד יחידה זו יש להכיר את המונחים האלה.

3.3.1 רשימת מונחים (בסדר מלוגי)

- אקסיומה (axiom)
- דקדוק חסר־הקשר (context-free grammar) עמוד 10 בספר.
- הנחה (hypothesis, antecedent)

Revision : 2.16.03

- הַסָּקָה (conclusion, consequent)
- חוֹזָה (contract)
- כָּלֵל הַיִּסָּק (rule of inference)
- מִפְּרֵט אֶכְלֹוֹל אִינְדוּקְטִיבִּי (inductive specification) עמוד 1 בספר.
- מִתְחָם (scope)
- עֵץ־גִּוִּירָה (derivation tree) עמוד 3 בספר.
- עֵץ־אֶפְרוּט (deduction tree)
- עֵץ־הַיִּסָּק (inference tree)
- צב"ן (צורת BNF) ((Backus-Naur Form)

3.3.2 שגרות שמוזכרות בפרק (בסדר מלוגי)

- count-occurences תרגיל 1.20 בספר.
- down תרגיל 1.17 בספר.
- duple תרגיל 1.15 בספר.
- every? תרגיל 1.24 בספר.
- exists? תרגיל 1.25 בספר.
- filter-in תרגיל 1.22 בספר.
- flatten תרגיל 1.27 בספר.

- `in-S?` עמוד 2 בספר.
- `interior-node` תרגיל 1.31 בספר.
- `invert` תרגיל 1.16 בספר.
- `leaf` תרגיל 1.31 בספר.
- `leaf?` תרגיל 1.31 בספר.
- `list-index` תרגיל 1.23 בספר.
- `list-length` עמוד 13 בספר.
- `list-set` תרגיל 1.19 בספר.
- `list-sum` עמוד 24 בספר.
- `lson` תרגיל 1.31 בספר.
- `merge` תרגיל 1.28 בספר.
- `nth-element` עמוד 14 בספר.
- `number-elements` עמוד 23 בספר.
- `number-elements-from` עמוד 23 בספר.
- `occurs-free?` עמוד 18 בספר.
- `partial-vector-sum` עמוד 25 בספר.
- `product` תרגיל 1.21 בספר.
- `remove-first` עמוד 16 בספר.

Revision : 2.16.03

- sort תרגיל 1.29 בספר.
- subst עמוד 20 בספר.
- subst-in-s-exp עמוד 21 בספר.
- swapper תרגיל 1.18 בספר.
- up תרגיל 1.26 בספר.
- vector-sum עמוד 25 בספר.
- rson תרגיל 1.31 בספר.
- contents-of תרגיל 1.31 בספר.
- double-tree תרגיל 1.32 בספר.
- mark-leaves-with-red-depth תרגיל 1.33 בספר.
- path תרגיל 1.34 בספר.

3.3.3 מקורות לקריאה נוספת

Abelson, H., & Sussman, G. J. (1996). Structure and Interpretation of Computer Programs (2nd ed.). MIT Press. (Abelson & Sussman, 1996)

פרק 4

הפשטת נתונים

פרק זה עוסק בסוגיה מרכזית: הבחנה בין מהות מבנה הנתונים לבין ייצוגו.



פרק זה נלווה לפרק 2 בספר הלימוד.

4.1 אפיון מבנה נתונים באופן אלגברי

קראו את סעיף §2.1 בספר הלימוד.

הפשטת נתונים (data abstraction) מאפשרת לבודד בין מפרט (specification) השימוש לבין פרטי המימוש (implementation). המפרט של טיפוס הנתונים מתאר את מהות הנתונים, את הפעולות המוגדרות עליהם ואת תכונותיהם. לעומת זאת, המימוש קובע את אופן ייצוג הנתונים ומספק את הקוד הפועל על הייצוג הנבחר.

Revision : 2.16.04

טיפוס נתונים מופשט (abstract data type) הוא טיפוס נתונים שבו ניטל הצמוד בין המפרט ("מה") ובין הייצוג ("איך").

החציצה מתבצעת באמצעות ממשק (interface), מושג שמוכר לכם מתכנות. הממשק מתאר את ה"מה": מה מייצגים הנתונים מטיפוס זה, מה הן הפעולות שניתן לבצע עליהם, ולמה לצפות בפעולתם. המימוש מתאר את ה"איך": איך מיוצגים הנתונים, ובהתאם לייצוג, איך ממומש הקוד של הפעולות. תכנות כנגד ממשק מאפשר לברור בין ייצוגים ולהחליף מימושים בדיעבד ובאופן שלא פוגע בקודי־לקוח. קודי־לקוח (client) המקפיד להסתמך רק על הממשק של טיפוס הנתונים המופשט הוא בלתי־תלוי בייצוג. גם אם לעתים אין מנוס מלהתייחס באפיון אל הייצוג, אפשר לעשות זאת ללא יצירת תלות בייצוג.

4.2 קבוצת המספרים הטבעיים

דוגמה אחת שווה אלף מילים. נשוב להגדרה של קבוצת המספרים השלמים האי־שליליים N :

$$N = N_0 = \{0, 1, 2, \dots\}$$

נגדיר מבנה נתונים מופשט עבור N , התומך בארבע פעולות בסיסיות:

- **zero** – בנאי (constructor) המחזיר את המספר "אפס".
- **predecessor** – בנאי שבהינתן מספר טבעי גדול מאפס מחזיר את הסודר הקודם.

- **successor** – בנאי שבהינתן מספר טבעי מחזיר את הסודר העוקב.
- **is-zero?** – מנבא (פְּרִדִּיקט), שבהינתן מספר טבעי, מְאַשֵׁשׁ (confirms) או מְרַעֵעַ (disconfirms) את היות המספר "אפס".

4.2.1 אפיון אלגברי

חתימה. נגדיר את החתימה (signature) של כל אחת מהפעולות הללו כך:

$$\text{zero} : () \rightarrow N$$

$$\text{successor} : N \rightarrow N$$

$$\text{predecessor} : N \rightarrow N$$

$$\text{is-zero} : N \rightarrow N$$

החתימה מגדירה את התחום ואת הטווח של כל פעולה (שגרה). אולם, היא אינה מגדירה איזה מספר שלם מוחזר על-ידי **zero**. היא אינה מגדירה איזו התאמה מ- N ל- N ממומשת על-ידי **successor** ואיזו על-ידי **predecessor**. היא גם אינה מגדירה אילו מספרים מותאמים ל- $\#t$ על-ידי **is-zero? is-zero?** ואילו ל- $\#f$. לכן לא די בחתימות ויש להגדיר משמעות לפעולות אלה.

משמעות. נגדיר את המשמעות של הפעולות האלה, ללא תלות במימוש, בכך שנבדיל בין המושג המתמטי "אפס", שאותו נסמן על-ידי הסיפורה "0", לבין הייצוג של "אפס", שאותו נסמן על-ידי צומדיים יתדיים באופן הבא: "[0]".

באופן כללי, נשתמש בסימון $[\xi]$ על מנת לתאר את הייצוג של הנתון ξ .

היתרון בסימון זה הוא בכך שאפשר להגדיר בעזרתו במדויק מה מחוללות הפעולות הללו מבלי להתחייב מהו המימוש. אפשר לחשוב על הצומדיים היתדיים כעל התאמה בין ξ לבין ייצוג כלשהו של ξ באופן שנחליט. בעזרת סימון זה נאפיין את משמעות הפעולות כך:

$$\begin{aligned}(\text{zero}) &= [0] \\(\text{successor } [n]) &= [n + 1] \quad (n \geq 0) \\(\text{predecessor } [n + 1]) &= n \quad (n \geq 0) \\(\text{is-zero } [n]) &= \begin{cases} \#t & (n = 0) \\ \#f & (n > 0) \end{cases}\end{aligned}$$

במילים: יהיה אשר יהיה הייצוג, הבנאי **zero** מחזיר את הייצוג של אפס. עבור $n \geq 0$, כאשר מעבירים אל הבנאי **successor** את הייצוג של n כארגומנט, הוא מחזיר את הייצוג של $n + 1$, וכאשר מעבירים אל הבנאי **predecessor** את הייצוג של $n + 1$ כארגומנט, הוא מחזיר את הייצוג של n . אחרון חביב, כאשר מעבירים אל הפרדיקט **is-zero?** את הייצוג של n כארגומנט, הוא מחזיר **#t** אם $n = 0$ ו-**#f** אחרת.

★תרגיל 4.1 מה מחזיר **is-zero?** כאשר $n < 0$?

4.2.2 פעולות אריתמטיות

עתה אפשר לנסח פעולות אריתמטיות נוספות ולממשן, והן תעבודנה ללא תלות בבחירת הייצוג למספרים טבעיים. למשל, אפשר כבר לנסח שגרת־לקוח המחברת שני מספרים טבעיים (ראה קוד 4.1):


```
;; plus : N * N -> N
;; usage: (plus x y) = the sum of x and y
(define (plus x y)
  (if (is-zero? x)
      y
      (successor (plus (predecessor x) y))))
```

Listing 4.1: plus.scm

חשוב לשים לב לכך שגם ללא תלות בייצוג $[n]$ מובטח כי:

$$(\text{plus } [x] [y]) = [x + y]$$



כאשר כאן הסימון "+" הוא פעולת החיבור המתמטית.

משמעות הדבר היא שמוחזר הסכום הנכון, יהיה הייצוג אשר יהיה. מיד נסקור כמה מימושים אפשריים, אך תחילה נוכיח טענה זו.

★תרגיל 4.2 הוכיחו באינדוקציה כי: $(\text{plus } [x] [y]) = [x + y]$

פתרון: צריך להוכיח כי:

$$(\text{plus } [x] [y]) = [x + y]$$

ההוכחה באינדוקציה על x .

ראשית, נבדוק נכונות עבור $x = 0$. כאשר $(\text{is-zero? } [0]) = \#t$, אכן:

$$(\text{plus } [0] [y]) = [y] = [0 + y] \quad \checkmark$$

Revision : 2.16.04

עתה, נניח כי עבור $0 \leq x = n - 1$ אמנם מתקיים:

$$(\text{plus } \lceil (n - 1) \rceil \lceil y \rceil) = \lceil (n - 1) + y \rceil$$

ונוכיח כי גם עבור $0 < x = n$ מתקיים:

$$(\text{plus } \lceil n \rceil \lceil y \rceil) = \lceil n + y \rceil$$

הוכחה:

נתון כי $n \neq 0$, וכאשר $(\text{is-zero? } \lceil n \rceil) = \#f$ נקבל כי:

$$\begin{aligned} (\text{plus } \lceil n \rceil \lceil y \rceil) &= (\text{successor } (\text{plus } (\text{predecessor } \lceil n \rceil) \lceil y \rceil)) = \\ &= (\text{successor } (\text{plus } \lceil n - 1 \rceil \lceil y \rceil)) = \\ &= (\text{successor } \lceil (n - 1) + y \rceil) = \\ &= \lceil (n - 1) + y + 1 \rceil = \\ &= \lceil n + y \rceil \end{aligned}$$

□

□ מ.ש.ל. ■

★תרגיל 4.3 ממשו את פעולת הכפל האריתמטית `.times`.



רמז: (לתרגיל 4.3) מותר להיעזר ב־`.plus`.

★תרגיל 4.4 הוכיחו באינדוקציה כי: $(\text{times } \lceil x \rceil \lceil y \rceil) = \lceil x \times y \rceil$

★תרגיל 4.5 ממשו את פעולת העצרת `.fact`.



רמז: (לתרגיל 4.5) מותר להיעזר ב־`.times`.

★תרגיל 4.6 הוכיחו באינדוקציה כי: $(\text{fact } \lceil n \rceil) = \lceil n! \rceil$

🔗 וכעת נסקור ייצוגים שונים: ייצוג אונרי (סעיף 4.2.3 להלן), ייצוג בינארי (סעיף 4.2.4 בעמוד 52 במדריך), ייצוג בבסיס כלשהו (סעיפים 4.2.5 ו-4.2.6), וייצוג בקידוד Church (סעיף 4.2.7 בעמוד 55 במדריך). 🔗

4.2.3 ייצוג אונרי

יהא $[n]$ רשימה המוגדרת באופן הבא:

$$\begin{aligned} [0] &= () \\ [n + 1] &= (\#t . [n]) \end{aligned}$$

כלומר, המספר n מיוצג על-ידי רשימה באורך n המכילה n פעמים את $\#t$. למשל, 0 מיוצג על-ידי הרשימה הריקה $()$; 1 מיוצג על-ידי $(\#t)$; 2 מיוצג על-ידי $(\#t \#t)$, וכן הלאה. המימוש מובא בקוד 4.2:

עמ' 33

```
;; plus : N * N -> N
;; Unary Representation
;; page 33 10.
(define zero (lambda () '()))
(define is-zero? (lambda (n) (null? n)))
(define successor (lambda (n) (cons #t n)))
(define predecessor (lambda (n) (cdr n)))
```

Listing 4.2: plus-unary-rep-1.scm

תרגילים. התרגילים הללו מתייחסים לקוד 4.2.

*תרגיל 4.7 מהו הייצוג של 0?

★תרגיל 4.8 מהו הייצוג של 1?

★תרגיל 4.9 מהו הייצוג של 2?

★תרגיל 4.10 מהו הייצוג של 3?

★תרגיל 4.11 כמה ייצוגים יש ל-0?

★תרגיל 4.12 מהו `(predecessor (zero))`? האם זו שגיאה?

★תרגיל 4.13 השוו את המימוש שבספר (קוד 4.2) למימוש המובא להלן (קוד 4.3). מה הבדל ביניהם, אם בכלל יש הבדלים כאלה?

```
(define zero '())
(define is-zero? null?)
(define successor (lambda (n) (cons #t n)))
(define predecessor cdr)
```

Listing 4.3: plus-unary-rep-2.scm

4.2.4 ייצוג בינארי

יהא $[n]$ הייצוג של המספר השלם n ב־Scheme. המימוש מובא בקוד 4.4:

```

;; plus : N * N -> N
;; Scheme number representation
;; page 33
(define zero (lambda () 0))
(define is-zero? (lambda (n) (zero? n)))
(define successor (lambda (n) (+ n 1)))
(define predecessor (lambda (n) (- n 1)))

```

Listing 4.4: plus-binary-rep.scm

תרגילים. התרגילים הללו מתייחסים לקוד 4.4.

★תרגיל 4.14 מהו הייצוג של 0?

★תרגיל 4.15 מהו הייצוג של 1?

★תרגיל 4.16 מהו הייצוג של 2?

★תרגיל 4.17 מהו הייצוג של 3?

★תרגיל 4.18 כמה ייצוגים יש ל-0?

★תרגיל 4.19 מהו (predecessor (zero)) האם זו שגיאה?

★תרגיל 4.20 האמנם הייצוג בינארי? כיצד ניתן לבדוק זאת?

★תרגיל 4.21 מה ההבדל בין המימוש של is-zero? המובא בקוד 4.4

למימוש הזה: (define is-zero zero?) אם יש בכלל הבדל?

Revision : 2.16.04

4.2.5 ייצוג בבסיס 3

יהא $[n]$ רשימה של מספרים המייצגים את n בבסיס 3:

$$[n] = \begin{cases} () & n = 0 \\ (r \cdot [q]) & n = 3q + r, 0 \leq r < 3 \end{cases}$$

למשל, הייצוג של 258 הוא:

$$[258] = [0 \times 3^0 + 2 \times 3^1 + 1 \times 3^2 + 0 \times 3^3 + 0 \times 3^4 + 1 \times 3^5] = (0 \ 2 \ 1 \ 0 \ 0 \ 1)$$

המימוש נותר לקריאה ולעבודה עצמית.

תרגילים. התרגילים הללו מתייחסים לייצוג בבסיס 3:

★**תרגיל 4.22** בהגדרה $(r \cdot [q])$, מדוע אין סוגרי ייצוג סביב r ומדוע חיוניים סוגרי הייצוג סביב q ?

★**תרגיל 4.23** מהו הייצוג של 0?

★**תרגיל 4.24** מהו הייצוג של 1?

★**תרגיל 4.25** מהו הייצוג של 2?

★**תרגיל 4.26** מהו הייצוג של 3?

★**תרגיל 4.27** כמה ייצוגים יש ל-0?

★**תרגיל 4.28** מהו (predecessor (zero)? האם זו שגיאה?

★**תרגיל 4.29** הכלל את הייצוג לבסיס כלשהו B.

4.2.6 ייצוג בבסיס כלשהו

ייצוג בבסיס B הוא הכללה מיידית של הייצוג בבסיס 3. למשל, עבור $B = 16$ נקבל שהייצוג של 258 הוא:

$$\lceil 258 \rceil = \lceil 2 \times 16^0 + 0 \times 16^1 + 1 \times 16^2 \rceil = (2 \ 0 \ 1)$$

המימוש נותר לקריאה ולעבודה עצמית.

👉 קראו את סעיף §2.1.3 בספר הלימוד. 👈

★תרגיל 4.30 פתרו תרגיל 2.1 בעמוד 34 בספר.

4.2.7 Church ייצוג בקידוד Church

✂️ ✨ סעיף זה הוא להעשרה בלבד ואינו כולל בחומר האימוץ. ✨ ✂️

יהא $\lceil n \rceil$ שגרה המוגדרת כך:

$$\lceil n \rceil = \begin{cases} (\text{lambda } (f) (\text{lambda } (x) x)) & n = 0 \\ (\text{lambda } (f) (\text{lambda } (x) (f ((\lceil n - 1 \rceil f))))) & n > 0 \end{cases}$$

כלומר, המספר הטבעי n מיוצג בקידוד צ'רץ (Church encoding) על-ידי ההרכבה n -פעמים של פונקציה נתונה f :

$$f^n = \overbrace{f \circ f \circ \dots \circ f}$$

Revision : 2.16.04

כלומר,

$$\begin{aligned}
[0] &= \lambda f. \lambda x. x &= (\text{lambda } (f) (\text{lambda } (x) x)) \\
[1] &= \lambda f. \lambda x. f(x) &= (\text{lambda } (f) (\text{lambda } (x) (f x))) \\
[2] &= \lambda f. \lambda x. f^2(x) &= (\text{lambda } (f) (\text{lambda } (x) (f (f x)))) \\
[3] &= \lambda f. \lambda x. f^3(x) &= (\text{lambda } (f) (\text{lambda } (x) (f (f (f x))))) \\
[n] &= \lambda f. \lambda x. f^n(x) &= (\text{lambda } (f) (\text{lambda } (x) (f^n x)))
\end{aligned}$$

לספקנים ולסקרנים מומלץ להריץ את המימוש המובא בקוד 4.5:

```

;; (zero) = λf. (λu. u)
(define zero (lambda () (lambda (f) (lambda (x) x))))

;; succesor = λn. λx. f ((n f) x)
(define (successor n)
  (lambda (f) (lambda (x) (f ((n f) x)))))

;; predecessor = λ n f x. n (λ g h. h (g f)) (λ u. x) (λ u. u)
(define (predecessor n)
  (lambda (f)
    (lambda (x)
      (((n (lambda (g)
        (lambda (h)
          (h (g f)))))
        (lambda (u) x))
       (lambda (u) u)))))

;; is-zero? = λn. n (λx. #f) #t
(define (is-zero? n) ((n (lambda (z) #f)) #t) )

```

Listing 4.5: church.scm

תרגילים. התרגילים הללו מתייחסים לייצוג בקידוד Church:

★תרגיל 4.31 מהו הייצוג של 0?

★תרגיל 4.32 מהו הייצוג של 1?

★תרגיל 4.33 מהו הייצוג של 2?

★תרגיל 4.34 מהו הייצוג של 3?

★תרגיל 4.35 כמה ייצוגים יש ל-0?

★תרגיל 4.36 בדקו שההגדרה של `is-zero?` פועלת כמצופה על 0, על 1 ועל 2.

★תרגיל 4.37 בדקו שההגדרה של פעולת הסודר העוקב פועלת כמצופה על 0, 1 ועל 2.

★תרגיל 4.38 בדקו שמימוש של `plus` פועל כמצופה על הסכום $0 + 1$ ועל הסכום $1 + 1$.

★תרגיל 4.39 מהו `(predecessor (zero))`? האם זו שגיאה?

★תרגיל 4.40 כמה ייצוגים יש ל-0?

✎ פתרון: ("יצוג בקידוד Church)

את הייצוג של 0, 1, 2, ו-3 כבר ראינו. נבדוק שהמימוש של `is-zero?` פועל כמצופה (קוד 4.6):

Revision : 2.16.04

```

(define one (successor (zero)))
(define two (successor one))

(is-zero? (zero)) ;returns #t
(is-zero? one) ;returns #f
(is-zero? two) ;returns #f

;; testing
(define (f n) (+ n 2)) ;f(n)=n+2
(= 2 (((zero) f) 2)) ;returns #t
(= 4 ((one f) 2)) ;returns #t
(= 6 ((two f) 2)) ;returns #t

(define (g n) (* n n)) ;g(n)=n*n
(= 2 (((zero) g) 2)) ;returns #t
(= 4 ((one g) 2)) ;returns #t
(= 16 ((two g) 2)) ;returns #t

```

Listing 4.6: church-test-1.scm

מענה חלקי על יתר השאלות אפשר למצוא בקוד הזה (קוד 4.7):

```

;; expressions that return #t
(is-zero? (predecessor one))
(is-zero? (predecessor (predecessor two)))
(is-zero? (predecessor (plus (zero) one)))
(is-zero? (predecessor (predecessor (plus one one))))
;; expressions that return #f
(is-zero? (predecessor (plus one one)))

```

Listing 4.7: church-test-2.scm

□

4.3 סביבת משתנים מופשטת

סביבת משתנים (environment) היא התאמה חד-ערכית מקבוצה סופית של משתנים אל ערכים. נייצג משתנה על-ידי תג (symbol) ונייצג ערכים באמצעות Scheme. הסביבה מתאימה ערך לכל תג בתחום. אפשר לחשוב על סביבה כעל אוסף של זוגות סדורים מתוך המכפלה הקרטזית $Sym \times Val$:

$$env = \{(var_1, val_1), \dots, (var_n, val_n)\} \subseteq Sym \times Val$$

כאשר

$$\{var_i : i = 1, \dots, n\} \subseteq Sym$$

היא קבוצת משתנים זרים ו-

$$\{val_i : i = 1, \dots, n\} \subseteq Val$$

היא קבוצת ערכים כלשהם.

🔗 כאשר המשתנה var_i כרוך (bound) בסביבה env לערך val_i , נאמר ש- var_i נכרך וש- val_i הוא הערך שכרוך עם var_i . הערך val_i גם נקרא לפעמים הפריקה (binding) של var_i בסביבה env . 🔗

4.3.1 אפיון אלגברי

נאפיין מבנה נתונים מופשט לייצוג סביבה עם הפעולות האלה:

$$\text{empty-env} : () \rightarrow Env$$

$$\text{extend-env} : Sym \times Val \times Env \rightarrow Env$$

$$\text{apply-env} : Env \times Sym \rightarrow Val$$

Revision : 2.16.04

נגדיר את המשמעות של הפעולות האלה, ללא תלות במימוש, כך:

$$\begin{aligned}(\text{empty-env}) &= [\emptyset] \\ (\text{apply-env } [f] \ s) &= f(s) \\ (\text{extend-env } s \ v \ [f]) &= [g] \\ \text{where } g(s') &= \begin{cases} v & (s = s') \\ f(s') & \text{otherwise} \end{cases}\end{aligned}$$

במילים:

- **empty-env** - בנאי שמחזיר את הייצוג של הסביבה הריקה (קבוצה ריקה).
- **extend-env** - בנאי שמחזיר את הייצוג של הסביבה שמרחיבה את הסביבה f על-ידי כריכת המשתנה k עם הערך v .
- **apply-env** - מְשַׁקֵּף (observer) שמחזיר את הערך שכרוך למשתנה k בסביבה f .

יש שני סוגי פעולות במנשק של טיפוס נתונים: בנאים - מייצרים ערכים יש מאין; משקיפים - מחלצים מידע מערכים דקיימא.

◆ שימו לב שהפעולת **apply-env** על סביבה ריקה אינה מוגדרת ואל
 כן להעביר רשימה ריקה כארגומנט לזה שיטא.

★תרגיל 4.41 הוכיחו באינדוקציה על גודל הסביבה כי זוהי שגיאה לחשב את $(\text{apply-env } env \ x)$ אם המשתנה x אינו כרוך עם שום ערך בסביבה env .

4.3.2 ייצוג באמצעות מבנה נתונים מופשט

יהא:

$$\begin{aligned}
 [\emptyset] &= (\text{empty-env}) \\
 [f \cup \{\langle k, v \rangle\}] &= (\text{extend-env } k \ v \ [f])
 \end{aligned}$$

בייצוג זה, סביבה היא מבנה רקורסיבי המוגדר על-ידי הדקדוק:

$$\begin{aligned}
 \text{Env-exp} &::= (\text{empty-env}) \\
 &::= (\text{extended-env } Identifier \ Scheme\text{-}value \ \text{Env-exp})
 \end{aligned}$$

כלומר, סביבה ריקה מאופיינת על-ידי רשימה שאיברה היחיד הוא התג `empty-env`. לעומתה, סביבה מורחבת מאופיינת על ידי רשימה בת ארבעה איברים: התג `extend-env`; המשתנה; הערך; והסביבה שמורחבת. דוגמה: על מנת להגדיר סביבה, מתחילים עם הסביבה הריקה ומרחיבים אותה בהדרגה (קוד 4.8):

```

(define e
  (extend-env 'd 6
    (extend-env 'y 8
      (extend-env 'x 7
        (extend-env 'y 14
          (empty-env))))))

```

Listing 4.8: env-example.scm

נקבל את הסביבה:

$$e = \{(d, 6), (y, 8), (x, 7)\}$$

Revision : 2.16.04

כלומר, את ההתאמה:

$$\{d, x, y\} \xrightarrow{e} \{6, 7, 8\}$$

שבה:

$$e(d) = 6$$

$$e(x) = 7$$

$$e(y) = 8$$

◆ שימו לב כי הכריכה המוקדמת של y -ל-14 נרמסה על-ידי הכריכה
המאוחרת של y -ל-8. ◆

```

;; data definition:
;; Env ::= (empty-env) | (extend-env Var Schemeval Env)

;; empty-env : () -> Env
(define empty-env
  (lambda () (list 'empty-env)))

;; extend-env : Var * Schemeval * Env -> Env
(define extend-env
  (lambda (var val env)
    (list 'extend-env var val env)))

;; apply-env : Env * Var -> Schemeval
(define apply-env
  (lambda (env search-var)
    (cond
      ((equal? (car env) 'empty-env)
       (report-no-binding-found search-var))
      ((equal? (car env) 'extend-env)
       (let ((saved-var (cadr env))
             (saved-val (caddr env))
             (saved-env (cadddr env)))
         (if (eqv? search-var saved-var)
             saved-val
             (apply-env saved-env search-var))))
      (else
       (report-invalid-env env)))))

```

Listing 4.9: env-def.scm

השגרה `apply-env` בוחנת את מבנה הנתונים `env` המייצג סביבה, מבררת את טיבו ועושה את "הדבר הנכון". אם מסתבר שזו הסביבה הריקה, מונפקת הודעת שגיאה. אם מסתבר שזו סביבה שנוצרה על-ידי `extend-env`, אז השגרה בודקת האם התג שמחפשים אותו זהה לתג שכרוך בסביבה. אם התג אכן זהה, אז הערך הכרוך מוחזר.

Revision : 2.16.04

אחרת, מחפשים את התג בסביבה `.save-env`.

שגרות העוזר מוגדרות כך (קוד 4.10):

```
(define report-no-binding-found
  (lambda (search-var)
    (eopl:error 'apply-env "No binding for ~s" search-var)))

(define report-invalid-env
  (lambda (env)
    (eopl:error 'apply-env "Bad environment: ~s" env)))
```

Listing 4.10: env-report.scm

תרגיל 4.42 פתרו שלושה מתוך התרגילים המסומנים ב-[] בסעיף §2.2.2 בעמוד 39 בספר.

4.3.3 ייצוג באמצעות פונקציה

כאשר לטיפוס נתונים מופשט יש משקיף אחד (ואין אף `mutator`), ניתן לייצג מופעים שלו כפונקציות. יהא

$$[f] = f : Sym \rightarrow Val$$

בייצוג זה, הסביבה עצמה היא פונקציית ההתאמה, שבהינתן משתנה מחזירה את הערך שכרוך אליו. ייצוג פונקציונלי כזה אפשרי כאשר יש משקיף (`observer`) יחיד: מבנה הנתונים ממלא את תפקיד המשקיף. (ניתן להכליל זאת גם למבנה נתונים מופשט עם יותר ממשקיף יחיד.) המימוש מובא בקוד 4.11:


```

;; data definition:
;; Env = Var -> Schemeval

;; empty-env : () -> Env
(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))

;; extend-env : Var * Schemeval * Env -> Env
(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
          saved-val
          (apply-env saved-env search-var)))))

;; apply-env : Env * Var -> Schemeval
(define apply-env
  (lambda (env search-var)
    (env search-var)))

```

Listing 4.11: env-func.scm

הסבר: אם לסביבה הריקה, שנוצרה על-ידי `empty-env`, מועבר כארגומנט תג כלשהו, היא מנפיקה הודעת שגיאה שהתג לא נמצא בתחום ההגדרה. השגרה `extend-env` מחזירה שגרה שמייצגת את הסביבה המורחבת. כאשר מעבירים לשגרה זו תג כארגומנט, היא בודקת אם תג זה כרוך בסביבה. אם התג כרוך בסביבה, אז הערך הכרוך מוחזר. אחרת, מחפשים את התג בסביבה המצומצמת יותר. שגרות העזר מוגדרות כמו מקודם.

בייצוג פונקציונלי, כל נתון מיוצג על-ידי פעולתו תחת `.apply-env`.

★תרגיל 4.43 השוו את המימוש המובא בספר (קוד 4.11) למימוש המובא להלן (קוד 4.12). מה ההבדל ביניהם, אם יש בכלל הבדל?

```
;; data definition:
;; Env = Var -> Schemeval

;; empty-env : () -> Env
(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))

;; extend-env : Var * Schemeval * Env -> Env
(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
          saved-val
          (saved-env search-var)))))
```

Listing 4.12: env-alt.scm

4.4 תחביר מופשט לטיפוס נתונים רקורסיבי

קראו את סעיף §2.5 בספר הלימוד.

נחזור פעם נוספת להגדרה של $\lambda\text{-Exp}$ באמצעות דקדוק:

$$\begin{aligned}\lambda\text{-Exp} &::= \text{Identifier} \\ &::= (\text{lambda } (\text{Identifier}) \lambda\text{-Exp}) \\ &::= (\lambda\text{-Exp } \lambda\text{-Exp})\end{aligned}$$

דקדוק זה אינו מופשט דיו. הוא כולל מידע תחבירי מוחשי (קונקרטי), כגון מילת המפתח `lambda` ומיקום הסוגריים. דוגמה למילה השייכת לשפה שמוגדרת על-ידי דקדוק זה היא הביטוי:

$$(\text{lambda } (x) (f (f x)))$$

אולם ניתן היה להגדיר דקדוק אחר עבור $\lambda\text{-Exp}$, עם תחביר מופשט דומה אך עם תחביר מוחשי שונה, למשל התחביר:

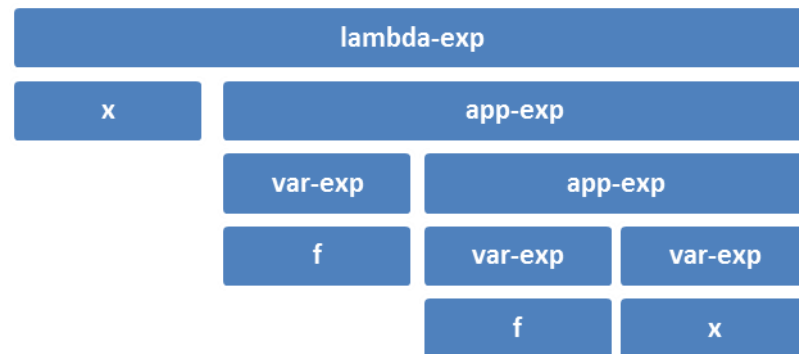
$$\begin{aligned}\lambda\text{-Exp} &::= \text{Identifier} \\ &::= \text{proc Identifier} \Rightarrow \lambda\text{-Exp} \\ &::= \lambda\text{-Exp } (\lambda\text{-Exp})\end{aligned}$$

במקרה זה, אותו ביטוי ממש היה נרשם כך:

$$\text{proc } x \Rightarrow f(f(x))$$

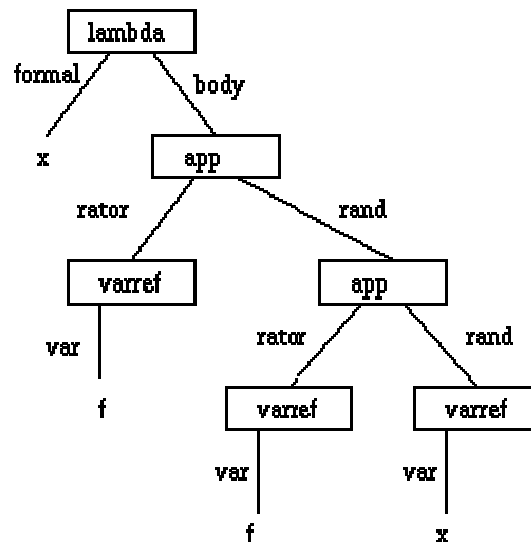
התחביר המוחשי חיוני לצורך זיהוי מילים בשפה. התחביר המופשט מספיק לצורך ייצוגם. למשל, את הביטוי המופשט עבור הדוגמאות לעיל ניתן להציג באופן גרפי כך:

Revision : 2.16.04



איור 4.1: מבנה מופשט

או באמצעות עץ גזירה כך:



איור 4.2: עץ גזירה

4.4.1 ביטויים בתחשיב למדא

אנו נייצג את התחביר המופשט של $\lambda\text{-Exp}$ באמצעות מבנה הנתונים המופשט הזה:

$$\begin{aligned}\lambda\text{-Exp} &::= (\text{var-exp Identifier}) \\ &::= (\text{lambda-exp Identifier } \lambda\text{-Exp}) \\ &::= (\text{app-exp } \lambda\text{-Exp } \lambda\text{-Exp})\end{aligned}$$

★תרגיל 4.44 פתרו תרגילים 2.15, 2.16, 2.17 בספר.

✍️ פתרון: ראו פתרון בהמשך (סעיף 4.5, עמוד 69 במדריך). □

★תרגיל 4.45 ממשו שגרות הממירות בין הייצוג המוחשי לבין הייצוג המופשט. החתימות של השגרות הן:

$$\begin{aligned}\text{parse-expression} &: \text{SchemeVal} \rightarrow \lambda\text{-Exp} \\ \text{unparse-expression} &: \lambda\text{-Exp} \rightarrow \text{SchemeVal}\end{aligned}$$

✍️ פתרון: ראו פתרון בהמשך (קוד 13.2 וקוד 13.3, עמוד 244 במדריך). □

4.5 תחשיב למדא כטיפוס נתונים מופשט

(פתרון תרגיל 4.44).

יש להגדיר מנשק ראוי ל- $\lambda\text{-Exp}$, הכולל בנאים, מנבאים ומחלצים.

Revision : 2.16.04

4.5.1 בנאים

נגדיר בנאים (קוד 4.13):

 $\text{var-exp} : \text{Var} \rightarrow \lambda\text{-Exp}$ $\text{lambda-exp} : \text{Var} \times \lambda\text{-Exp} \rightarrow \lambda\text{-Exp}$ $\text{app-exp} : \lambda\text{-Exp} \times \lambda\text{-Exp} \rightarrow \lambda\text{-Exp}$

```
;; var-exp : Var -> Lc-exp
(define var-exp
  (lambda (var)
    '(var-exp ,var)))

;; lambda-exp : Var * Lc-exp -> Lc-exp
(define lambda-exp
  (lambda (var lc-exp)
    '(lambda-exp ,var ,lc-exp)))

;; app-exp : Lc-exp * Lc-exp -> Lc-exp
(define app-exp
  (lambda (lc-exp1 lc-exp2)
    '(app-exp ,lc-exp1 ,lc-exp2)))
```

Listing 4.13: lambda-cons.scm

✦ שימו לב לשימוש שנעשה כאן בזהר (הקרו') (back-quote (quasiquote

ובפסיק (הקרו') (unquote).

4.5.2 מנבאים

נגדיר משקיפים מסוג מנבאים (קוד 4.14):

 $\text{var-exp?} : \lambda\text{-Exp} \rightarrow \text{Bool}$ $\text{lambda-exp?} : \lambda\text{-Exp} \rightarrow \text{Bool}$ $\text{app-exp?} : \lambda\text{-Exp} \rightarrow \text{Bool}$

```
;; var-exp? : Lc-exp -> Bool
(define var-exp?
  (lambda (x)
    (and (pair? x) (eq? (car x) 'var-exp))))

;; lambda-exp? : Lc-exp -> Bool
(define lambda-exp?
  (lambda (x)
    (and (pair? x) (eq? (car x) 'lambda-exp))))

;; app-exp? : Lc-exp -> Bool
(define app-exp?
  (lambda (x)
    (and (pair? x) (eq? (car x) 'app-exp))))
```

Listing 4.14: lambda-pred.scm

✦ עי'נו לר' לשימוש *eq?* כ- *pair?* ✦

4.5.3 מחלצים

נגדיר משקיפים (observers) מסוג מחלצים (extractors), (קוד 4.15):

$$\text{var-exp} \rightarrow \text{var} : \lambda\text{-Exp} \rightarrow \text{Var}$$

$$\text{lambda-exp} \rightarrow \text{bound-var} : \lambda\text{-Exp} \rightarrow \text{Var}$$

$$\text{lambda-exp} \rightarrow \text{body} : \lambda\text{-Exp} \rightarrow \lambda\text{-Exp}$$

$$\text{app-exp} \rightarrow \text{rator} : \lambda\text{-Exp} \rightarrow \lambda\text{-Exp}$$

$$\text{app-exp} \rightarrow \text{rand} : \lambda\text{-Exp} \rightarrow \lambda\text{-Exp}$$


```

;; var-exp->var : Lc-exp -> Var
(define var-exp->var
  (lambda (x)
    (cadr x)))

;; lambda-exp->bound-var : Lc-exp -> Var
(define lambda-exp->bound-var
  (lambda (x)
    (cadr x)))

;; lambda-exp->body : Lc-exp -> Lc-exp
(define lambda-exp->body
  (lambda (x)
    (caddr x)))

;; app-exp->rator : Lc-exp -> Lc-exp
(define app-exp->rator
  (lambda (x)
    (cadr x)))

;; app-exp->rand : Lc-exp -> Lc-exp
(define app-exp->rand
  (lambda (x)
    (caddr x)))

```

Listing 4.15: lambda-extr.scm

◆ שימו לב ש(*rator*-אופ) ו-(*rand*-אופ) מחזירים מתק הביטוי את האופרטור
 ואת האופרנד, ומכאן שם. ◆

4.6 משתנה חופשי?

התבוננו שוב במימוש (בקוד 3.2, עמוד 40) של השגרה

`occurs-free? : Sym × λ-Exp → Bool`

לבדיקה האם משתנה הוא חופשי. עכשיו ניתן לשכתב את המימוש (קוד 3.2) כך שלא יהיה עוד תלוי ייצוג (קוד 4.16). בחנו את עצמכם אם המימוש המשוכתב נהיר לכם באמצעות התרגילים האלה:

★תרגיל 4.46 אמת או שקר?

```
(occurs-free? 'a (lambda-exp 'a (app-exp (var-exp 'b) (var-exp 'a))))
```

★תרגיל 4.47 אמת או שקר?

```
(occurs-free? 'b (lambda-exp 'a (app-exp (var-exp 'b) (var-exp 'a))))
```

```
;; occurs-free? : Sym * Lcexp -> Bool
(define occurs-free?
  (lambda (search-var exp)
    (cond
      ((var-exp? exp) (eqv? search-var (var-exp->var exp)))
      ((lambda-exp? exp)
       (and
        (not (eqv? search-var (lambda-exp->bound-var exp)))
        (occurs-free? search-var (lambda-exp->body exp))))
      (else
       (or
        (occurs-free? search-var (app-exp->rator exp))
        (occurs-free? search-var (app-exp->rand exp)))))))
```

Listing 4.16: lambda-occurs-free.scm

4.7 כלים להגדרת טיפוס נתונים רקורסיבי

👉 קראו את סעיף §2.4 בספר הלימוד. 👈

יש דרך קלה יותר להגדיר את אוסף השגרות הדרוש למימוש טיפוס נתונים מופשט.

4.7.1 מנגנון להגדרת טיפוס נתונים

ההצהרה

```
(define-datatype type-name type-predicate-name {  
    (variant-name {(field-name predicate)}*)+})
```

מגדירה:

1. טיפוס נתונים בשם "type-name";

2. מנבא בשם "type-predicate-name" לצורך אישוש שערך נתון הוא מטיפוס
"type-name";

3. ולכל סוג, בנאי בשם "variant-name" שמספר הארגומנטים שלו כמספר ה-"
"field-name" שמופיעים ברשימה; הבנאי מאשש באמצעות ה-"predicate"
המתאים שכל ארגומנט הוא מהטיפוס המצופה.

לדוגמה (קוד 4.17):

Revision : 2.16.04

```
(define-datatype lc-exp lc-exp?
  (var-exp
    (var symbol?))
  (lambda-exp
    (bound-var symbol?)
    (body lc-exp?))
  (app-exp
    (rator lc-exp?)
    (rand lc-exp?)))
```

Listing 4.17: define-datatype.scm

קוד זה שקול להגדרת אוסף השגרות האלה:

$lc\text{-}exp? : \lambda\text{-}Exp \rightarrow Bool$

$var\text{-}exp : Sym \rightarrow \lambda\text{-}Exp$

$lambda\text{-}exp : Sym \times \lambda\text{-}Exp \rightarrow \lambda\text{-}Exp$

$app\text{-}exp : \lambda\text{-}Exp \times \lambda\text{-}Exp \rightarrow \lambda\text{-}Exp$

4.7.2 מנגנון לבחינת מרכיביו של טיפוס נתונים

הביטוי

```
(cases type-name expression
  {(variable-name ({field-name}* consequent))}
  (else default))
```

מזהה את סוג הנתון ומפרק אותו למרכיביו. לדוגמה, את המימוש (קוד 4.16) של `occurs-free?` ניתן לרשום באופן מתומצת כך (קוד 4.18):

```
;; occurs-free? : Sym * Lcexp -> Bool
(define occurs-free?
  (lambda (search-var exp)
    (cases lc-exp exp
      (var-exp (var)
        (eqv? var search-var))
      (lambda-exp (bound-var body)
        (and
          (not (eqv? search-var bound-var))
          (occurs-free? search-var body)))
      (app-exp (rator rand)
        (or
          (occurs-free? search-var rator)
          (occurs-free? search-var rand))))))
```

Listing 4.18: occurs-free-cases.scm

4.8 המרה בין ייצוג מוחשי לייצוג מופשט

פתרון תרגיל 4.45. נגדיר:

Revision : 2.16.04

```

;; parse-expression: SchemeVal -> LcExp
(define parse-expression
  (lambda (datum)
    (cond
      ((symbol? datum) (var-exp datum))
      ((pair? datum)
       (if (eqv? (car datum) 'lambda)
           (lambda-exp (caadr datum)
                        (parse-expression (caddr datum)))
           (app-exp
            (parse-expression (car datum))
            (parse-expression (cadr datum))))))
      (else (eopl:error 'parse-expression
                        "Invalid concrete syntax ~s" datum)))))

```

Listing 4.19: parse-expression.scm

ונגדיר:

```

;; unparse-expression: LcExp -> SchemeVal
(define unparse-expression
  (lambda (exp)
    (cases lc-exp exp
      (var-exp (id) id)
      (lambda-exp (id body)
        (list 'lambda (list id)
              (unparse-expression body)))
      (app-exp (rator rand)
        (list (unparse-expression rator)
              (unparse-expression rand))))))
    (eopl:error 'apply-env "No binding for ~s" search-var)))

```

Listing 4.20: unparse-expression.scm

*תרגיל 4.48 הרחיבו את ההגדרה של $\lambda\text{-Exp}$ כך שיכלול גם מספרים, ופתרו שוב את תרגיל 4.45.

4.9 סיכום

פרק זה דן בהפשטת נתונים. ניתן להגדיר באופן שיטתי מנשק לטיפוס נתונים רקורסיבי בכך שמגדירים בנאי לכל סוג נתונים, מנבא לכל סוג נתונים ומחליץ לכל נתון שמועבר לבנאי.

4.9.1 רשימת מונחים (בסדר מלוני)

- בלתי־תלוי בייצוג (representation independent) עמוד 23 בספר.
- הפשטת נתונים (data abstraction) עמוד 13 בספר.
- טיפוס נתונים מופשט (abstract data type) עמוד 13 בספר.
- כריכה (binding) עמוד 63 בספר.
- מימוש (implementation) עמוד 13 בספר.
- מנשק (interface) עמוד 13 בספר.
- מפרט (specification) עמוד 23 בספר.
- משקיף (observer) עמוד 33 בספר.
- סביבה (environment) עמוד 53 בספר.
- קוד־לקוח (client code) עמוד 13 בספר.
- רשימה אסוציאטיבית (a-list, association-list) עמוד 93 בספר.

עמ' 23

Revision : 2.16.04

• שפה ממוקדת תחום (domain specific language) עמוד 94 בספר.

• תג (symbol) עמוד 53 בספר.

• תחביר מופשט (abstract syntax) עמוד 15 בספר.

• תחביר מוחשי (concrete syntax) עמוד 15 בספר.

עמ' 15

4.9.2 שגרות שמוזכרות בפרק (בסדר מלוגני)

• empty-env? תרגיל 2.8 בספר.

• extend-env תרגיל 2.10 בספר.

• has-binding? תרגיל 2.9 בספר.

• is-zero? עמוד 33 בספר.

• plus עמוד 33 בספר.

• predecessor עמוד 33 בספר.

• successor עמוד 33 בספר.

• zero עמוד בספר.

4.10 תרגילים נוספים



תרגילים שהופיעו בהחיות

★תרגיל 4.49 (שאלה מבחינה) פולינום היא פונקציה מהצורה:

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_kx^k$$

דרגת הפולינום היא החזקה הגבוהה ביותר בעלת מקדם שונה מאפס. יהא *poly* טיפוס נתונים מופשט (ADT) המייצג פולינום ותומך בפעולות הבאות:

- zero – מחזירה פולינום ריק (ללא איברים), כלומר את $p(x) = 0$.
- make-poly – מקבלת כפרמטרים מקדם (coefficient) וחזקה (expo-) (nent ומחזירה פולינום מתאים.
למשל, (make-poly 3 4) מחזירה את הפולינום $p(x) = 3x^4$.
- add-poly – מקבלת זוג פולינומים ומחזירה פולינום המייצג את סכומם.
למשל, ((add-poly (make-poly 3 4) (make-poly 2 1))) מחזירה את הפולינום $p(x) = 2x^1 + 3x^4$.
- degree – מקבלת פולינום ומחזירה את דרגתו.
למשל, (degree (add-poly (make-poly 3 4) (make-poly 2 1))) מחזירה את הערך 4.
- coeff – מקבלת פולינום וחזקה ומחזירה את המקדם של האיבר בעל החזקה הנתונה. אם אין איבר כזה מוחזר 0.
למשל, (coeff (make-poly 3 4) 4) מחזירה 3.
אבל (coeff (make-poly 3 4) 9) מחזירה 0.

Revision : 2.16.04

• `is-zero?` – מקבלת פולינום ובודקת האם הוא הפולינום הריק. הפעולה מחזירה ערך בוליאני מתאים.

למשל, `(is-zero? (zero))` מחזירה `true`.

ואילו, `(is-zero? (make-poly 3 4))` מחזירה `false`.

1. כתבו אפיון לחתימות של הפעולות. סווגו את הפעולות השונות לסוגים (בנאים, מחלצים, מנבאים).

2. כתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה (המשמעות) שלהם.

3. ממשו את `poly` ואת המנשק לעבודה אתו (לפי הפעולות שהוגדרו למעלה). אתם רשאים לבחור לייצג את `poly` באחת משלוש האפשרויות הבאות: ייצוג על-ידי רשימות, ייצוג פרוצדורלי, או ייצוג באמצעות `.define-datatype`.

★**תרגיל 4.50** (שאלה מבחינה) בשאלה זו עליכם לממש טיפוס נתונים מופשט המייצג חוצץ-מעגלי (`circular buffer`) ופעולות שונות על טיפוס נתונים זה. חוצץ-מעגלי הוא טיפוס נתונים מופשט (ADT) המייצג חוצץ בגודל נתון שאליו אפשר להוסיף איברים, כאשר החוצץ מתמלא, הוספת איברים נוספים תגרום למחיקת האיברים הוותיקים יותר בחוצץ והכנסת האיברים החדשים במקומם. חוצץ-מעגלי תומך בפעולות הבאות:

• `create` – מקבלת מספר המייצג את גודלו של החוצץ, יוצרת ומחזירה חוצץ מעגלי (ריק) בגודל זה.

• `insert` – מקבלת כפרמטרים חוצץ-מעגלי ונתון חדש, ומוסיפה את הנתון לחוצץ. הפעולה מחזירה את החוצץ החדש. למשל, אם נתון

חוצץ b , בגודל 3 המכיל את האיברים 10, 20, 30 אזי הכנסת הערך

40 לחוצץ תחזיר את החוצץ הבא: $[\text{insert } 40 \text{ } b] = [40 \text{ } 20 \text{ } 30]$

• `empty?` – מקבלת כפרמטר חוצץ-מעגלי ובודקת האם הוא ריק?
פעולה זו תחזיר ערך בוליאני מתאים.

• `last` – מקבלת כפרמטר חוצץ-מעגלי ומחזירה את האיבר האחרון
שהוכנס לחוצץ. אם החוצץ ריק, תודפס הודעת שגיאה.

• `print` – מקבלת חוצץ-מעגלי ומדפיסה את האיברים המאוחסנים בו
מהאיבר הוותיק יותר ועד לאיבר האחרון. למשל, בהדפסת חוצץ
בגודל 3 שאליו הוכנסו (משמאל לימין) האיברים 10, 20, 30, 40 יודפס
20 לאחריו 30 ולבסוף 40.

• `count` – מקבלת חוצץ-מעגלי ומחזירה כמה איברים הוכנסו אליו
עד כה (כולל אלה שנמחקו עקב המעגליות). למשל, אם נתון חוצץ-
מעגלי בגודל 3, שאליו הוכנסו חמשת האיברים: 10, 20, 30, 40, 50.
הכנסת 40 ו-50 גרמה ל-10 ו-20 להימחק, כך שמצב החוצץ הוא
כרגע $[40 \text{ } 50 \text{ } 30]$. עבור חוצץ כזה `count` תחזיר 5 מאחר שעד כה
הוכנסו אליו 5 נתונים.

1. כתבו אפיון לחתימות של הפעולות. סווגו את הפעולות השונות
לסוגים (בנאים, מחלצים, מנבאים).

2. כתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה
(המשמעות) שלהם. הקפידו על הגדרה מלאה לכל פעולה.

3. ממשו את טיפוס הנתונים חוצץ-מעגלי ואת הפעולות עליו. כלומר
עליכם לממש את: `create`, `insert`, `empty?`, `last`, `print`, `count` אתם

רשאים לייצג את החוצץ כרצונכם.

★**תרגיל 4.51** (שאלה מבחינה) בשאלה זו עליכם לממש טיפוס נתונים מופשט המייצג קבוצה (set) ופעולות שונות על טיפוס נתונים זה. set הוא טיפוס נתונים מופשט (ADT) המייצג קבוצה ותומך בפעולות הבאות:

- empty-set – מחזירה קבוצה ריקה.
- insert – מקבלת כפרמטרים קבוצה ונתון כלשהו ומכניסה נתון זה לקבוצה. פעולה זו מחזירה את הקבוצה החדשה.
- intersect – מקבלת שתי קבוצות ומחזירה את קבוצת החיתוך ביניהן.
- union – מקבלת שתי קבוצות ומחזירה את קבוצת האיחוד ביניהן. בקבוצת האיחוד לא יהיו כפילויות של נתונים.
- is-empty? – מקבלת קבוצה ומחזירה האם קבוצה זו ריקה. פעולה זו מחזירה ערך בוליאני מתאים.
- contains? – מקבלת שתי קבוצות ובודקת האם הקבוצה השנייה מוכלת בתוך הקבוצה הראשונה. פעולה זו מחזירה ערך בוליאני מתאים.

1. כתבו אפיון לחתימות של הפעולות. סווגו את הפעולות השונות לסוגים: בנאים, מחלצים, מנבאים.

2. כתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה (המשמעות) שלהן. הקפידו על הגדרה מלאה לכל פעולה.

3. ממשו באמצעות שימוש ברשימות את הפעולות שהוגדרו באפיון של טיפוס הנתונים קבוצה. כלומר, עליכם לממש את הפרוצדורות הבאות: `contains?`, `is-empty?`, `union`, `intersect`, `insert`, `empty-set`. ניתן לכתוב בנוסף פרוצדורות עזר במקרה הצורך. שימו לב: אין להשתמש בפעולות על קבוצות המוגדרות בשפת Scheme.

***תרגיל 4.52** (שאלה מבחינה) בשאלה זו עליכם לממש טיפוס נתונים מופשט המייצג עץ בינרי ממין ופעולות שונות על טיפוס נתונים זה. `BinaryTree` הוא טיפוס נתונים מופשט (ADT) המייצג עץ בינרי ממין ותומך בפעולות הבאות:

- `create` – מחזירה עץ בינרי ריק.
 - `insert` – מקבלת כפרמטרים עץ בינרי ונתון מספרי כלשהי ומכניסה נתון זה לעץ למקומו המתאים. פעולה זו מחזירה את העץ הבינרי החדש.
 - `is-leaf?` – מקבלת עץ בינרי ובודקת האם הוא מייצג עלה. פעולה זו מחזירה ערך בוליאני מתאים.
 - `right` – מקבלת עץ בינרי ומחזירה את תת העץ הימני שלו.
 - `left` – מקבלת עץ בינרי ומחזירה את תת העץ השמאלי שלו.
 - `root` – מקבלת עץ בינרי ומחזירה את הערך הנמצא בשורש העץ.
 - `count-leaf` – מקבלת עץ בינרי ומחזירה כמה עליים יש בעץ.
1. כתבו אפיון לחתימות של הפעולות. סווגו את הפעולות השונות לסוגים: בנאים, מחלצים, מנבאים.

2. כתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה (המשמעות) שלהן. הקפידו על הגדרה מלאה לכל פעולה.

3. ממשו באמצעות שימוש ברשימות את הפעולות שהוגדרו באפיון של טיפוס הנתונים. כלומר, עליכם לממש את הפרוצדורות הבאות: `create`, `insert`, `is-leaf?`, `right`, `left`, `count-leaf`, `root`. ניתן לכתוב בנוסף פרוצדורות עזר במקרה הצורך.

★**תרגיל 4.53** (שאלה מבחינה) בשאלה זו עליכם לממש טיפוס נתונים מופשט המייצג מטריצה (`matrix`) ופעולות שונות על טיפוס נתונים זה. `matrix` הוא טיפוס נתונים מופשט (ADT) המייצג מטריצה ותומך בפעולות הבאות:

- `zeros` – מקבלת 2 ארגומנטים המייצגים מספר שורות ומספר עמודות ומחזירה מטריצה עם מספר שורות ועמודות אלה שאיבריה הם אפסים.

- `mul-scalar` – מקבלת כפרמטרים מספר שלם ומטריצה ומכפילה את איברי המטריצה במספר זה. פעולה זו מחזירה את המטריצה החדשה.

- `insert` – מקבלת מטריצה, מספר שורה, מספר עמודה ונתון מספרי. ומאחסנת את הנתון המספרי בתוך המטריצה במיקום הנתון על-ידי השורה והעמודה שהתקבלו כפרמטרים. פעולה זו תחזיר את המטריצה החדשה

- `add-mat` – מקבלת שתי מטריצות ומחזירה מטריצה המייצגת את סכומם.

- rows – מקבלת מטריצה ומחזירה מספר המייצג את מספר השורות שיש למטריצה.
 - cols – מקבלת מטריצה ומחזירה מספר המייצג את מספר העמודות שיש למטריצה.
 - is-I? – מקבלת מטריצה ומחזירה האם מטריצה זו מייצגת מטריצת־יחידה. פעולה זו מחזירה ערך בוליאני מתאים.
1. כתבו אפיון לחתימות של הפעולות. סווגו את הפעולות השונות לסוגים: בנאים, מחלצים, מנבאים.
 2. כתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה (המשמעות) שלהם. הקפידו על הגדרה מלאה לכל פעולה.
 3. ממשו באמצעות שימוש ברשימות את הפעולות שהוגדרו באפיון של טיפוס הנתונים מטריצה. כלומר, עליכם לממש את הפרוצדורות הבאות: `zeros`, `mul-scalar`, `insert`, `add-mat`, `rows`, `cols`, `is-I?`. ניתן לכתוב בנוסף פרוצדורות עזר במקרה הצורך.
- ***תרגיל 4.54** (שאלה מבחינה) רצף (sequence) הוא טיפוס נתונים מופשט (ADT) המייצג אוסף סדור של ערכים בעלי טיפוס זהה. למשל, רצף של מספרים כלשהם. רצף תומך בביצוע הפעולות הבאות:
- empty – מחזירה רצף ריק.
 - insert – מקבלת ערך ורצף ומוסיפה את הערך לתחילתו של הרצף.
 - isEmpty – בודקת האם רצף נתון הוא ריק?

- `odd` - מקבלת רצף ומחזירה רצף חדש הכולל רק את האיברים במקומות האי-זוגיים. למשל,

`odd([])=[]` -

`odd([1,11,2,22,3,33])=[11,22,33]` -

`odd([a,b,c])=[b]` -

- `even` - מקבלת רצף ומחזירה רצף חדש הכולל רק את האיברים במקומות הזוגיים. למשל,

`even([])=[]` -

`even([1,11,2,22,3,33])=[1,2,3]` -

`even([a,b,c])=[a,c]` -

- `alternate` - מקבלת שתי רצפים ומחזירה רצף חדש המורכב משילוב שתי הרצפים לסירוגין. שימו לב, אם לרצפים אורכים שונים, השילוב שלהם יעשה לפי כמות הערכים ברצף הקצר יותר, והתעלמות מהערכים העודפים ברצף הארוך, כפי שמודגם להלן:

`alternate([1,2,3],[11,22,33])=[1,11,2,22,3,33]` -

`alternate([a,b,c],[aa])=[a,aa]` -

`alternate([a,b,c],[])=[]` -

1. כיתבו אפיון לחתימות של הפעולות. וסווגו את הפעולות השונות לסוגים: בנאים, מחלצים, מנבאים.

2. כיתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה (המשמעות) שלהם. הקפידו על הגדרה מלאה לכל פעולה.
3. ממשו את האפיון. אתם רשאים לממש כרצונכם (מימוש עם מבנה נתונים או מימוש פרוצדורלי או עם `define-datatype`).

Revision : 2.16.04

פרק 5

ביטויים

 בפרק זה נממש מספר שפות תכנות פשוטות המבוססות על ביטויים. 

שפות התכנות הראשונות שנממש הם שפות קטנות שמבוססות על ביטויים. בפרק זה נגדיר כמה שפות כאלה. ניעזר בדקדוק כדי לתאר את התחביר המוחשי ואת התחביר המופשט של כל אחת מהשפות. את המשמעות של כל שפה נתאר באמצעות מימוש מפרש הפועל על עץ התחביר המופשט ועל מבנה נתונים שנקרא סביבה. המפרש מקבל כארגומנט ראשון את עץ התחביר המופשט, וכארגומנט שני מופע של הסביבה. באמצעות הסביבה ננהל רישום של ערכי המשתנים המופיעים חופשיים בביטוי. פרק זה מלווה את פרק 3 בספר הלימוד.

5.1 אסטרטגיה לאפיון ומימוש של שפות

 קראו את סעיף §3.1 בספר הלימוד. 

Revision : 2.16.05

באפיון נעזר במשוואה מהצורה:

$$(\text{value-of } \text{exp } \rho) = \text{val}$$

שמשמעותה: הערך של הביטוי exp בסביבה ρ הוא val .

שפת המקור (source language) היא השפה שאותה אנו מגדירים ומממשים. שפת המימוש (בקורס זה היא Scheme) היא השפה שבה אנו נממש את המפרש. המנתח הלקסיקלי והמנתח התחבירי שבחזית המפרש (front end), או המהדר, מתרגמים את התכנית משפת המקור למבנה פנימי של עץ תחביר מופשט. המהדר מתרגם את עץ התחביר המופשט לשפת המטרה (target language), למשל לקוד של מכונה וירטואלית (bytecode) או לשפת מכונה של מעבד מסוים, שאפשר להריץ. לעומת זאת, המפרש "מריץ" ישירות את עץ התחביר המופשט.

מחולל נתח תחבירי הוא תכנית שמקבלת כקלט אפיון מילולי (לקסיקלי), כלומר דקדוק של שפה, ולכל חוק גזירה בדקדוק - תיאור של הענף בעץ התחביר המופשט שיש לבנות. הפלט של המחולל הוא נתח מילולי (לקסיקלי) ונתח תחבירי. המנתח המילולי מפצל, בהתאם לאפיון המילולי, את הטקסט הגולמי של הקוד לזרם מופעים מילוליים משמעותיים הנקראים **אסימונים** (tokens). המנתח התחבירי מתרגם את זרם האסימונים למבנה עץ תחבירי מופשט, בהתאם לדקדוק.

אנו ניעזר במחולל שנקרא SLLgen. לעת עתה, הדבר החשוב לדעת הוא שהשגרה scan&parse מקבלת כקלט מחרוזת המכילה ייצוג טקסטואלי של התכנית, ומחזירה כפלט את ייצוג התכנית כעץ תחבירי מופשט.

5.2 "וִיֵּהִי": שפת תכנות פשוטה (LET)

קראו את סעיף §3.2 בספר הלימוד. 📖

5.2.1 תחביר השפה "ויהי"

התחביר של שפת "ויהי" מסוכם בטבלה 5.1. התחביר המוחשי מופיע בעמודה השמאלית, ובעמודה הימנית מתואר הייצוג המופשט.

תחביר מופשט	תחביר מוחשי
a-program (exp1)	Program ::= Expression
const-exp (num)	Expression ::= Number
diff-exp (exp1 exp2)	::= -(Expression , Expression)
zero?-exp (exp1)	::= zero? (Expression)
if-exp (exp1 exp2 exp3)	::= if Expression then Expression else Expression
var-exp (var)	::= Identifier
let-exp (var exp1 body)	::= let Identifier = Expression in Expression

טבלה 5.1: תחביר השפה "ויהי"

למשל, הביטוי

```
(scan&parse "let x = 4 in -(x,-(1,x))")
```

משוערך למבנה המופשט המתקבל על-ידי

```
(a-program
  (let-exp 'x
    (const-exp 4)
    (diff-exp (var-exp 'x)
      (diff-exp (const-exp 1)
        (var-exp 'x))))))
```

Revision : 2.16.05

5.2.2 אפיון קבוצת הערכים

מאפיין חשוב של שפת תכנות הוא קבוצת הערכים שהשפה פועלת עמם. בכל שפת תכנות אפשר להבחין בשתי קבוצות ערכים לפחות. קבוצת הערכים שניתנים לביטוי (expressed values) הם הערכים שאפשר לקבל משערוך ביטויים. קבוצת הערכים הניתנים לציון (denoted values) הם הערכים שניתן לכרוך אותם עם משתנים. בשפת "ויהי", קבוצות הערכים שניתנים לביטוי ($ExpVal$) ולציון ($DenVal$) הן באופן יוצא מן הכלל זהות ביניהן. נסמן זאת כך:

$$ExpVal = Int + Bool$$

$$DenVal = Int + Bool$$

הערכים שניתנים לביטוי ולציון יהיו מטיפוס נתונים מופשט עם האפיון האלגברי:

$$\text{num-val} : Int \rightarrow ExpVal$$

$$\text{bool-val} : Bool \rightarrow ExpVal$$

$$\text{expval} \rightarrow \text{num} : ExpVal \rightarrow Int$$

$$\text{expval} \rightarrow \text{bool} : ExpVal \rightarrow Bool$$

כאשר

$$(\text{expval} \rightarrow \text{num} (\text{num-val } n)) = n$$

$$(\text{expval} \rightarrow \text{bool} (\text{bool-val } b)) = b$$

לעתים נשתמש בסימון $[n]$ במקום לרשום `(num-val n)` ובסימון $[val]$ במקום לרשום `(expval->num val)`.

◆ $[n] = n$ אלא $[val] \neq val$ ◆

★תרגיל 5.1 פתרו תרגיל 3.1 בעמוד 63 בספר.

★תרגיל 5.2 פתרו תרגיל 3.2 בעמוד 63 בספר.

נעזר במנגנון `define-datatype` להגדרת מבנה הנתונים:

```
(define-datatype expval expval?
  (num-val
    (value number?))
  (bool-val
    (boolean boolean?)))
```

Listing 5.1: expval-datatype.scm

נגדיר מחלצים:

```
(define expval->num
  (lambda (v)
    (cases expval v
      (num-val (num) num)
      (else (expval-extractor-error 'num v)))))
```

Listing 5.2: expval-num.scm

```
(define expval->bool
  (lambda (v)
    (cases expval v
      (bool-val (bool) bool)
      (else (expval-extractor-error 'bool v)))))
```

Listing 5.3: expval-datatype.scm

```
(define expval-extractor-error
  (lambda (variant value)
    (eopl:error 'expval-extractors "Looking for a ~s, found ~s"
      variant value)))
```

Listing 5.4: expval-error.scm

סביבות

נשתמש בסימונים הבאים:

- ρ - יציין סביבה כלשהי
- $[]$ - יציין את הסביבה הריקה
- $[var = val]\rho$ - יציין `(extend-env var val  $\rho$ )`, כלומר הרחבת ρ תוך כדי כריכת הערך val למשתנה var .
- $[var = val][]$ - יציין `(extend-env var val [])`, כלומר הרחבת הסביבה הריקה תוך כדי כריכת הערך val למשתנה var .

★תרגיל 5.3 מה משמעות $[var_1 = val_1, var_2 = val_2]\rho$?



רמז: ראו סעיף §3.2.3 בספר.

★תרגיל 5.4 מה משמעות $[var_1 = val_1, var_2 = val_2]?$

5.2.3 אפיון ההתנהגות של ביטויים

יש שישה סוגים של ביטויים בשפת "ויהי". המנשק של ביטויים בשפת "ויהי" כולל בנאי אחד לכל חוק גזירה של Exp המופיע בדקדוק, ומחלץ אחד. בנאים:

$const\text{-}exp : Int \rightarrow Exp$

$zero?\text{-}exp : Exp \rightarrow Exp$

$if\text{-}exp : Exp \times Exp \times Exp \rightarrow Exp$

$diff\text{-}exp : Exp \times Exp \rightarrow Exp$

$var\text{-}exp : Symbol \rightarrow Exp$

$let\text{-}exp : Symbol \times Exp \times Exp \rightarrow Exp$

ומחלץ:

$value\text{-}of : Exp \times Env \rightarrow ExpVal$

משמעות. עתה צריך לאפיין את ההתנהגות. כאמור, האפיון יכול להבטחות מהצורה

$(value\text{-}of\ e\ \rho) = v$

שמשמעותה: הערך של הביטוי e בסביבה ρ צריך להיות v .

Revision : 2.16.05

נתחיל באפיון של שלושה מסוגי הביטויים:

$$(\text{value-of } (\text{const-exp } n) \rho) = (\text{num-val } n)$$

$$(\text{value-of } (\text{var-exp } var) \rho) = (\text{apply-env } \rho var)$$

$$\begin{aligned} (\text{value-of } (\text{diff-exp } exp_1 exp_2) \rho) = \\ = (- (\text{expval} \rightarrow \text{num } (\text{value-of } exp_1 \rho)) (\text{expval} \rightarrow \text{num } (\text{value-of } exp_2 \rho))) \end{aligned}$$

5.2.4 דוגמה של שערוך ביטוי

יהא $\rho = [x = 10, v = 5, i = 1]$. נרשום $\langle\langle e \rangle\rangle$ על מנת לציין את עץ התחביר המופשט של הביטוי e . נראה את שערוך הביטוי עקב בצד אגודל.

$$(\text{value-of-expression } \langle\langle -(x,3), -(v,i) \rangle\rangle \rho) =$$

$$\begin{aligned} = & (- \\ & (\text{value-of-expression } \langle\langle -(x,3) \rangle\rangle \rho) \\ & (\text{value-of-expression } \langle\langle -(v,i) \rangle\rangle \rho)) = \end{aligned}$$

```
= (-
  (-
    (value-of-expression <<x>> ρ)
    (value-of-expression <<3>> ρ))
  (value-of-expression <<-(v,i)>> ρ)) =
```

```
= (-
  (-
    10
    (value-of-expression <<3>> ρ))
  (value-of-expression <<-(v,i)>> ρ)) =
```

```
= (-
  (-
    10
    3)
  (value-of-expression <<-(v,i)>> ρ)) =
```

```
= (-
  7
  (value-of-expression <<-(v,i)>> ρ)) =
```

```
= (-
  7
  ( -
    (value-of-expression <<v>> rho)
    (value-of-expression <<i>> rho)))) =
```

Revision : 2.16.05

```
= (-
  7
  ( -
    5
    (value-of-expression <<i>> rho))) =
```

```
= (-
  7
  ( -
    5
    1)) =
= (-
  7
  4) =
= 3
```

5.2.5 אפיון התנהגות של תכניות

בשפת "ויהי", התכנית כולה היא בסך הכל ביטוי אחד. על מנת לשערך ביטוי זה, צריך לפרט מה הם ערכי המשתנים החופשיים בתכנית. כלומר, ערך התכנית הוא שערך הביטוי הראשי בסביבה מתאימה. על כן, עבור שפת "ויהי", אפיון ההתנהגות של תכניות מסתכם באפיון הסביבה התחילית. ברוב שפות התכנות, הסביבה התחילית כוללת ספריות סטנדרטיות שמוגדרות מראש. עבור שפת "ויהי", נספק הגדרה מראש של שלושה מזהים. נגדיר אפוא,

$$(\text{value-of-program } exp) = (\text{value-of } exp \rho_0)$$

ונקבע שרירותית:

$$\rho_0 = [i = [1], v = [5], x = [10]]$$

5.2.6 אפיון התנהגות של יתר הביטויים

לשפת "ויהי" יש בנאי zero ומשקיף if של ערכים בוליאנים. הערך של zero? הוא #t אם ורק אם ערך הארגומנט שלו הוא אפס. נאפיין זאת באמצעות שני כללי היסק:

$$\frac{\begin{array}{l} (\text{value-of } exp_1 \rho) = val_1 \\ (\text{expval} \rightarrow \text{num } val_1) = 0 \end{array}}{(\text{value-of } (\text{zero?-exp } exp_1) \rho) = (\text{bool-val } \#t)}$$

$$\frac{\begin{array}{l} (\text{value-of } exp_1 \rho) = val_1 \\ (\text{expval} \rightarrow \text{num } val_1) = n \quad (n \neq 0) \end{array}}{(\text{value-of } (\text{zero?-exp } exp_1) \rho) = (\text{bool-val } \#f)}$$

כאן, bool-val הוא בנאי המשמש להסבת ערך בוליאני לערך בר-ביטוי. expval → num הוא משקיף שבודק אם הערך המובע הוא מספר שלם, ואם כן - מחזיר את המספר. הביטוי if הוא משקיף של ערכים בוליאנים. הערך של ביטוי if הוא הארגומנט השני או השלישי, תלוי בערך האמת של הארגומט הראשון:

$$\frac{\begin{array}{l} (\text{value-of } exp_1 \rho) = val_1 \\ (\text{expval} \rightarrow \text{bool } val_1) = \#t \end{array}}{(\text{value-of } (\text{if-exp } exp_1 exp_2 exp_3) \rho) = \text{value-of } exp_2 \rho}$$

Revision : 2.16.05

$$\frac{\begin{array}{l} (\text{value-of } \text{exp}_1 \rho) = \text{val}_1 \\ (\text{expval} \rightarrow \text{bool } \text{val}_1) = \#f \end{array}}{(\text{value-of } (\text{if-exp } \text{exp}_1 \text{exp}_2 \text{exp}_3) \rho) = (\text{value-of } \text{exp}_3 \rho)}$$

אפשר לרשום זאת בקיצור כך:

$$\begin{aligned} & (\text{value-of } (\text{if-exp } e_1 e_2 e_3) \rho) = \\ & = (\text{if } (\text{expval} \rightarrow \text{bool } (\text{value-of } e_1 \rho)) \\ & \quad (\text{value-of } e_2 \rho) \\ & \quad (\text{value-of } e_3 \rho)) \end{aligned}$$

ואחרון חביב, אפיון של פקודת "ויהי", שעל שמה נקראת גם השפה, משערך את ה"גוף" בסביבה שהורחבה על ידי כריכת המשתנה לערך של הארגומנט השני:

$$\frac{(\text{value-of } \text{exp}_1 \rho) = \text{val}_1}{(\text{value-of } (\text{let-exp } \text{var } \text{exp}_1 \text{body}) \rho) = (\text{value-of } \text{body } [\text{var} = \text{val}_1] \rho)}$$

אפשר לרשום זאת בקיצור כך:

$$\begin{aligned} & (\text{value-of } (\text{let-exp } x e_1 e_2) \rho) = \\ & = (\text{value-of } e_2 [x = (\text{value-of } e_1 \rho)] \rho) \end{aligned}$$

5.2.7 הביטוי if

יהא $\rho = [x = 33, y = 22]$.

```
(value-of
  <<if zero?(-(x,11)) then -(y,2) else -(y,4)>>
  ρ)=
= 18
```

5.2.8 הביטוי let

נראה עוד דוגמה פשוטה:

```
let x = 5
in -(x,3)
```

המשתנה x כרוך בגוף המופיע אחרי `in`. גוף הביטוי יכול להיות ביטוי כלשהו. בפרט הוא יכול להיות ביטוי `let` מקונן:

```
let z = 5
in let x = 3
  in let y = -(x,1)
    in let x = 4
      in -(z, -(x,y))
```

*תרגיל 5.5 מה הערך של x בכל סביבה? מה הערך של y ? מה הערך של z ? מה הערך של הביטוי כולו?

הערך שנכרך עם המשתנה מובע גם על-ידי ביטוי כלשהו, ויכול להיות בעצמו

ביטוי `let`:

```
let x = 7
in let y = 2
    in let y = let x = -(x,1)
        in -(x,y)
    in -(-(x,8),y)
```

★תרגיל 5.6 מה הערך של x בכל סביבה? מה הערך של y ? מה הערך של הביטוי כולו?

5.2.9 מימוש שפת "ויהי"

האפיון המילולי (לקסיקלי) של שפת ויהי מוגדר על-ידי SLLgen כך (קוד 5.5):

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% (arbno (not #\newline))) skip)
    (identifier
      (letter (arbno (or letter digit "_" "-" "?")))
      symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

Listing 5.5: let-the-lexical-spec.scm

התחביר של שפת "ויהי" מוגדר באמצעות דקדוק (קוד 5.6):


```

(define the-grammar
  '((program (expression) a-program)

    (expression (number) const-exp)
    (expression
      ("-" "(" expression "," expression ")")
      diff-exp)

    (expression
      ("zero?" "(" expression ")")
      zero?-exp)

    (expression
      ("if" expression "then" expression "else" expression)
      if-exp)

    (expression (identifier) var-exp)

    (expression
      ("let" identifier "=" expression "in" expression)
      let-exp)

  ))

```

Listing 5.6: let-the-grammar.scm

מתוך האפיון האמור, SLLgen מחולל את ההגדרות הבאות (קוד 5.7):

Revision : 2.16.05

```

(define-datatype program program?
  (a-program (a-program13 expression?)))

(define-datatype expression expression?
  (const-exp (const-exp14 number?))
  (diff-exp
    (diff-exp15 expression?)
    (diff-exp16 expression?))
  (zero?-exp (zero?-exp17 expression?))
  (if-exp
    (if-exp18 expression?)
    (if-exp19 expression?)
    (if-exp20 expression?))
  (var-exp (var-exp21 symbol?))
  (let-exp
    (let-exp22 symbol?)
    (let-exp23 expression?)
    (let-exp24 expression?)))

```

Listing 5.7: let-define-datatype.scm

השוו את קוד 5.7 לקוד המובא באיור 3.6 בעמוד 69 בספר, שמציג את אותו הקוד אחרי "ניקוי" קל.

ערכים שאפשר לבטא. מבנה הנתונים עבור ערכים שניתנים לביטוי בשפת "ויהי" ממומש על-ידי ההגדרות בקוד 5.8:

```

;;; an expressed value is either a number, a boolean or a procval.
(define-datatype expval expval?
  (num-val
    (value number?))
  (bool-val
    (boolean boolean?)))

;;; extractors:
(define expval->num
  (lambda (v)
    (cases expval v
      (num-val (num) num)
      (else (expval-extractor-error 'num v)))))

(define expval->bool
  (lambda (v)
    (cases expval v
      (bool-val (bool) bool)
      (else (expval-extractor-error 'bool v)))))

(define expval-extractor-error
  (lambda (variant value)
    (eopl:error 'expval-extractors "Looking for a ~s, found ~s"
      variant value)))

```

Listing 5.8: let-expressed-values.scm

סביבה. היות שסביבה היא טיפוס נתונים מופשט, מימוש המפרש בלתי־תלוי בייצוג של סביבה. נוכל על כן להשתמש בכל מימוש של סביבה, אך נצטרך להגדיר לשם כך סביבה התחלתית (קוד 5.9):

```

;; init-env : -> Env

;; (init-env) builds an environment in which i is bound to the
;; expressed value 1, v is bound to the expressed value 5, and x is
;; bound to the expressed value 10.
(define init-env
  (lambda ()
    (extend-env
      'i (num-val 1)
      (extend-env
        'v (num-val 5)
        (extend-env
          'x (num-val 10)
          (empty-env)))))))

```

Listing 5.9: let-init-env.scm

שגרות עזר. נגדיר עוד שגרת עזר שתקל עלינו להריץ תכניות בשפת "ויהי"
(קוד 5.10):

```

;; run : String -> ExpVal
(define run
  (lambda (string)
    (value-of-program (scan>parse string))))

```

Listing 5.10: let-run.scm

המפרש. לבסוף, נממש את המפרש (קוד 5.11 וקוד 5.12):

```
;; value-of-program : Program -> ExpVal
(define value-of-program
  (lambda (pgm)
    (cases program pgm
      (a-program (body)
        (value-of body (init-env))))))
```

Listing 5.11: let-value-of-program.scm

Revision : 2.16.05

```

;; value-of : Exp * Env -> ExpVal
(define value-of
  (lambda (exp env)
    (cases expression exp

      (const-exp (num) (num-val num))

      (var-exp (id) (apply-env env id))

      (diff-exp (exp1 exp2)
        (let ((val1
              (expval->num
               (value-of exp1 env)))
              (val2
               (expval->num
                (value-of exp2 env))))
          (num-val
           (- val1 val2))))

      (zero?-exp (exp1)
        (let ((val1 (expval->num (value-of exp1 env))))
          (if (zero? val1)
              (bool-val #t)
              (bool-val #f))))

      (if-exp (exp0 exp1 exp2)
        (if (expval->bool (value-of exp0 env))
            (value-of exp1 env)
            (value-of exp2 env)))

      (let-exp (id rhs body)
        (let ((val (value-of rhs env)))
          (value-of body
                     (extend-env id val env))))

    )))

```

Listing 5.12: let-value-of.scm

★תרגיל 5.7 ממשו והריצו תכנית Hello World בשפת "ויהי".

5.3 בדיקת המימוש

נבדוק את המימוש בעזרת התכנית הבאה.

```
(define test-list
  '(
    ;; simple arithmetic
    (positive-const "11" 11)
    (negative-const "-33" -33)
    (simple-arith-1 "-(44,33)" 11)

    ;; nested arithmetic
    (nested-arith-left "-(-(44,33),22)" -11)
    (nested-arith-right "-(55, -(22,11))" 44)

    ;; simple variables
    (test-var-1 "x" 10)
    (test-var-2 "-(x,1)" 9)
    (test-var-3 "-(1,x)" -9)
```

Listing 5.13: let-lang/tests.scm

Revision : 2.16.05

```

;; simple unbound variables
(test-unbound-var-1 "foo" error)
(test-unbound-var-2 "-(x,foo)" error)

;; simple conditionals
(if-true "if zero?(0) then 3 else 4" 3)
(if-false "if zero?(1) then 3 else 4" 4)

;; test dynamic typechecking
(no-bool-to-diff-1 "-(zero?(0),1)" error)
(no-bool-to-diff-2 "-(1,zero?(0))" error)
(no-int-to-if "if 1 then 2 else 3" error)

;; make sure that the test and both arms get evaluated
;; properly.
(if-eval-test-true "if zero?(-(11,11)) then 3 else 4" 3)
(if-eval-test-false "if zero?(-(11, 12)) then 3 else 4" 4)

;; and make sure the other arm doesn't get evaluated.
(if-eval-test-true-2 "if zero?(-(11, 11)) then 3 else foo" 3)
(if-eval-test-false-2 "if zero?(-(11,12)) then foo else 4" 4)

;; simple let
(simple-let-1 "let x = 3 in x" 3)

;; make sure the body and rhs get evaluated
(eval-let-body "let x = 3 in -(x,1)" 2)
(eval-let-rhs "let x = -(4,1) in -(x,1)" 2)

;; check nested let and shadowing
(simple-nested-let "let x = 3 in let y = 4 in -(x,y)" -1)
(check-shadowing-in-body "let x = 3 in let x = 4 in x" 4)
(check-shadowing-in-rhs "let x = 3 in let x = -(x,1) in x" 2)

))

```

Listing 5.14: let-lang/tests.scm

5.4 "וישגור": שפה עם שגרות (PROC)

קראו את סעיף §3.3 בספר הלימוד.

אם נוסיף שגרות לשפת "ויהי", נקבל שפה מעט יותר שימושית והרבה יותר שגרתית. נקרא לשפה זו שפת "וישגור", שפה שגרתית בהחלט.

5.4.1 תחביר שפת "וישגור"

נרחיב את הדקדוק של שפת "ויהי" באמצעות הוספה של שני חוקים המאפשרים הגדרה וקריאה לשגרות.

תחביר מופשט	תחביר מוחשי
a-program (exp1)	Program ::= Expression
const-exp (num)	Expression ::= Number
diff-exp (exp1 exp2)	Expression ::= -(Expression , Expression)
zero?-exp (exp1)	Expression ::= zero? (Expression)
if-exp (exp1 exp2 exp3)	Expression ::= if Expression then Expression else Expression
var-exp (var)	Expression ::= Identifier
let-exp (var exp1 body)	Expression ::= let Identifier = Expression in Expression
proc-exp (var body)	Expression ::= proc (Identifier) Expression
cal-exp (exp1 exp2)	Expression ::= (Expression Expression)

טבלה 5.2: תחביר שפת "וישגור"

התחביר של שפת "וישגור" מוגדר באמצעות דקדוק כך (קוד 5.15):

Revision : 2.16.05

```

(define the-grammar
  '( (program (expression) a-program)

      (expression (number) const-exp)
      (expression
        ("-" "(" expression "," expression ")")
        diff-exp)

      (expression
        ("zero?" "(" expression ")")
        zero?-exp)

      (expression
        ("if" expression "then" expression "else" expression)
        if-exp)

      (expression (identifier) var-exp)

      (expression
        ("let" identifier "=" expression "in" expression)
        let-exp)

      (expression
        ("proc" "(" identifier ")" expression)
        proc-exp)

      (expression
        "(" expression expression ")")
        call-exp)

  ))

```

Listing 5.15: proc-the-grammar.scm

🔗 בהגדרת שגרה (`proc-exp var body`), נאמר כי המשתנה `var` הוא המשתנה הקשור או הפרמטר הפורמלי. בקריאה לשגרה (`call-exp exp1 exp2`), נאמר כי הביטוי `exp1` הוא האופרטור והביטוי `exp2` הוא האופרנד או הפרמטר האקטואלי. הפרמטר האקטואלי הוא ביטוי, ששערוכו נותן ערך. הערך של הפרמטר האקטואלי נקרא הארגומנט. 🔗



✦ עי'נו לא שכאן שגרה היא ביטוי שמחזיר ערך.

5.4.2 דוגמה לתכניות בשפת "וישגור"

הנה שתי תכניות שניתן לכתוב בשפת "וישגור". תכנית אחת:

```
let f = proc (x) -(x,11)
in (f (f 77))
```

תכנית שנייה:

```
(proc (f) (f (f 77))
  proc (x) -(x,11))
```

Revision : 2.16.05

5.4.3 אפיון קבוצת הערכים

בשפת "וישגור", בדומה לשפת "ויהי", קבוצות הערכים שניתנים לביטוי ($ExpVal$) ולציון ($DenVal$) זהות. נסמן זאת כך:

$$ExpVal = Int + Bool + Proc$$

$$DenVal = Int + Bool + Proc$$

עוד בדומה לשפת "ויהי", הערכים שניתנים לביטוי ולציון יהיו מטיפוס נתונים מופשט עם האפיון האלגברי:

$$\text{num-val} : Int \rightarrow ExpVal$$

$$\text{bool-val} : Bool \rightarrow ExpVal$$

$$\text{proc-val} : Proc \rightarrow ExpVal$$

$$\text{expval} \rightarrow \text{num} : ExpVal \rightarrow Int$$

$$\text{expval} \rightarrow \text{bool} : ExpVal \rightarrow Bool$$

$$\text{expval} \rightarrow \text{proc} : ExpVal \rightarrow Proc$$

כאשר:

$$(\text{expval-num } (\text{num-val } n)) = n$$

$$(\text{expval-bool } (\text{bool-val } b)) = b$$

$$(\text{expval-proc } (\text{proc-val } p)) = p$$

5.4.4 אפיון התנהגות של שגרות

נאפיין את מבנה הנתונים של שגרות:

$\text{procedure} : \text{Symbol} \times \text{Exp} \times \text{Env} \rightarrow \text{Proc}$

$\text{apply-procedure} : \text{Proc} \times \text{ExpVal} \rightarrow \text{ExpVal}$

$(\text{apply-procedure} (\text{procedure } \text{var } \text{body } \rho) \text{ val}) =$
 $= (\text{value-of } \text{body } [\text{var} = \text{val}] \rho)$

עתה אפשר לאפיין את הבנאי `procedure` שמייצר שגרות, ואת המשקיף `apply-procedure` שקורא לשגרות, באופן הבא:

$(\text{value-of } (\text{proc-exp } \text{var } \text{body}) \rho)$
 $= (\text{proc-val } (\text{procedre } \text{var } \text{body } \rho))$

$(\text{value-of } (\text{call-exp } \text{rator } \text{rand}) \rho)$
 $= (\text{apply-procedure } (\text{expval} \rightarrow \text{proc } (\text{value-of } \text{rator } \rho)) (\text{value-of } \text{rand } \rho))$

★תרגיל 5.8 מדוע נדרשת סביבה ρ כארגומנט?

נממש את האפיון על-ידי שנוסיף למפרש טיפול בשני המקרים הנוספים:

Revision : 2.16.05

```

(define value-of
  (lambda (exp env)
    (cases expression exp

      ...

      (proc-exp (bvar body)
        (proc-val
         (procedure bvar body env)))

      (call-exp (rator rand)
        (let ((proc (expval->proc (value-of rator env)))
              (arg (value-of rand env)))
          (apply-procedure proc arg)
        )))
  ))

```

◆ *ע"מ לא לשימוש נוסף כאן ב- let.* ◆

טרם סיימנו את המימוש של שפת "וישגור", כי יש לממש גם את מבנה הנתונים *Proc*.
אולם, כבר אפינו את טיפוס הנתונים *Proc*, ונוכל להיעזר בו בדוגמאות.

5.4.5 דוגמה לשערוך שגרה

הבה נבחן מה קורה כאשר ביטוי מסוג *proc* מופיע במיקום כלשהו בתכנית. בדוגמה להלן, נסמן את עץ התחביר המופשט עבור טקסט x על-ידי $\langle x \rangle$.

◆ *שתי השורות שבזלמה נוצרות מביטויים שהם אלק מתהלות שונה*
 זו מכל. מסקנה: המשמעות השלמה תלויה בסביבה. זו הסיבה לכך
 שצרכ הארומט השלישי בהצגת המשמעות של *proc-exp*.
 ◆

```
(value-of <<let x = 200
  in let f = proc (z) -(z,x)
  in let x = 100
    in let g = proc (z) -(z,x)
      in -((f 1), (g 1))>>
  ρ) =
```

```
= (value-of <<let f = proc (z) -(z,x)
  in let x = 100
    in let g = proc (z) -(z,x)
      in -((f 1), (g 1))>>
  [x=(num-val 200)]ρ) =
```

```
= (value-of <<let x = 100
  in let g = proc (z) -(z,x)
    in -((f 1), (g 1))>>
  [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)]ρ))]
  [x=(num-val 200)]ρ) =
```

```
= (value-of <<let g = proc (z) -(z,x)
  in -((f 1), (g 1))>>
  [x=(num-val 100)]
  [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)]ρ))]
  [x=(num-val 200)]ρ) =
```

```
= (value-of <<-((f 1), (g 1))>>
  [g=(proc-val (procedure z
    <<-(z,x)>>
    [x=(num-val 100)]
    [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)]ρ))]
    [x=(num-val 200)]ρ))]
  [x=(num-val 100)]
  [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)]ρ))]
  [x=(num-val 200)]ρ)
```

Revision : 2.16.05

```
= (-
  (value-of <<(f 1)>>
    [g=(proc-val (procedure z
      <<-(z,x)>>
      [x=(num-val 100)]
      [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ))]
      [x=(num-val 200)] ρ))]
    [x=(num-val 100)]
    [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ))]
    [x=(num-val 200)] ρ)
  (value-of <<(g 1)>>
    [g=(proc-val (procedure z
      <<-(z,x)>>
      [x=(num-val 100)]
      [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ))]
      [x=(num-val 200)] ρ))]
    [x=(num-val 100)]
    [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ))]
    [x=(num-val 200)] ρ) =
```

```
= (-
  (apply-procedure
    (expval->proc (proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ)))
    (num-val 1))
  (apply-procedure
    (expval->proc (proc-val (procedure z <<-(z,x)>>
      [x=(num-val 100)]
      [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ))]
      [x=(num-val 200)] ρ)))
    (num-val 1))) =
```

```
= (-
  (value-of <<-(z,x)>>
    [z=(num-val 1)]
    [x=(num-val 200)] ρ)
  (value-of <<-(z,x)>>
    [z=(num-val 1)]
    [x=(num-val 100)]
    [f=(proc-val (procedure z <<-(z,x)>> [x=(num-val 200)] ρ))]
    [x=(num-val 200)] ρ) =
```

```
= (-
  (- 1 200)
  (- 1 100)) =
```

```
= -100
```


★תרגיל 5.9 יהא $\rho = [x = 10]$. חשבו את הביטוי:

(value-of <<let f = proc(z) -(z,x) in (f 21)>> ρ)

★תרגיל 5.10 יהא $\rho = [x = 10]$. חשבו את הביטוי (שימו לב לדמיון לתרגיל 5.9):

(value-of <<let x = proc(z) -(z,x) in (x 21)>> ρ)

★תרגיל 5.11 יהא $\rho = [x = 10]$. חשבו את הביטוי (שימו לב לדמיון לתרגיל 5.9):

(value-of <<let g = proc(z) -(z,x) in (g 21)>> ρ)

★תרגיל 5.12 יהא $\rho = [x = 10]$. חשבו את הביטוי:

(value-of <<let f=proc(x) proc(y) -(x,y) in let g=(f 55) in (g 33)>> ρ)

5.4.6 ייצוג שגרות

עדיין צריך לממש את טיפוס הנתונים *Proc*. בסעיף 3.3.2 בעמוד 79 בספר מתוארים שני מימושים אפשריים. אין זה משנה מאוד באיזה מימוש נבחר, כי *Proc* הוא טיפוס נתונים מופשט. מה שחשוב הוא שהמימוש יקיים:

Revision : 2.16.05

$$\begin{aligned}
 &(\text{apply-procedure } (\text{procedure } var \text{ body } \rho) \text{ val}) &= \\
 &= (\text{value-of } body \text{ (extend-env } var \text{ val } \rho)) &= \\
 &= (\text{value-of } body [var = val] \rho)
 \end{aligned}$$

אפשר לייצג שגרות על-ידי שגרות בשפת Scheme (קוד 5.16):

```

;; procedure : Symbol * Exp * Env -> Proc

(define procedure
  (lambda (bvar body env)
    (lambda (arg)
      (value-of body (extend-env bvar arg env)))))

;; apply-procedure : Proc * ExpVal -> ExpVal

(define apply-procedure
  (lambda (proc arg)
    (proc arg)))

```

Listing 5.16: proc-procedure-impl.scm

אפשר לייצג שגרות גם על-ידי closure (קוד 5.17):

```

(define-datatype proc proc?
  (procedure
    (bvar symbol?)
    (body expression?)
    (env environment?)))

(define apply-procedure
  (lambda (proc1 arg)
    (cases proc proc1
      (procedure (bvar body saved-env)
        (value-of body (extend-env bvar arg saved-env))))))

```

Listing 5.17: proc-closure-impl.scm

בשני הייצוגים נותר להרחיב את טיפוס הנתונים :expval

;;; an expressed value is either a number, a boolean or a procval.

```

(define-datatype expval expval?
  (num-val
    (value number?))
  (bool-val
    (boolean boolean?))
  (proc-val
    (proc proc?)))

```

Listing 5.18: proc-expval.scm

ובהתאמה להרחיב גם את :value-of

Revision : 2.16.05

```
;; value-of : Exp * Env -> ExpVal
(define value-of
  (lambda (exp env)
    (cases expression exp

      ...

      (proc-exp (var body)
        (proc-val (procedure var body env)))

      (call-exp (rator rand)
        ; (let ((proc (expval->proc (value-of rator env)))
        ; (arg (value-of rand env)))
        (let ((val (value-of rator env))
              (arg (value-of rand env)))
          (let ((proc (expval->proc val)))
            (apply-procedure proc arg))))

    )))
```

Listing 5.19: proc-value-of.scm

שני הייצוגים האלה מזכירים את שני הייצוגים לסביבות. אכן יש שפות תכנות "אמתיות" שבהן המימוש של סביבות ושל שגרות הוא דומה או זהה.

★**תרגיל 5.13** תרגיל 3.19 בעמי 80 בספר: הוסיפו לשפה ביטוי `letproc` במקום הביטוי `proc`.

★**תרגיל 5.14** תרגיל 3.20 בעמוד 80 בספר: ממשו שגרה שמחשבת סכום של שני מספרים.

★**תרגיל 5.15** תרגיל 3.21 בעמוד 80 בספר: הוסיפו לשפה תמיכה בשגרות עם מספר ארגומנטים כלשהו.

★**תרגיל 5.16** תרגיל 3.23 בעמוד 81 בספר: ממשו שגרה שמחשבת $n!$.

★תרגיל 5.17 תרגיל 3.27 בעמוד 81 בספר: הוסיפו לשפה ביטוי `traceproc`.

★תרגיל 5.18 האם אפשר לכתוב תכנית עם לולאה אינסופית בשפת "ויהי"?

★תרגיל 5.19 האם אפשר לכתוב תכנית עם לולאה אינסופית בשפת "וישגור"?

★תרגיל 5.20 איזה ערך מחושב על-ידי התכנית הזו:

```
let f = proc (x) (x x)
in (f f)
```

5.5 "וייסוג": שפה עם שגרות רקורסיביות

👉 קראו את סעיף §3.4 בספר הלימוד. 👈

נרחיב את שפת "וישגור" ונוסיף לה את האפשרות להגדיר שגרה רקורסיבית. נקבל שפה חדשה שנקראת "וייסוג". נסתפק בשגרות רקורסיביות עם פרמטר יחיד.

5.5.1 תחביר שפת "וייסוג"

לדוגמה, ניתן לרשום:

```
letrec double(x) = if zero?(x)
then 0
```

Revision : 2.16.05

תחביר מופשט	תחביר מוחשי
a-program (exp1)	Program ::= Expression
const-exp (num)	Expression ::= Number
diff-exp (exp1 exp2)	Expression ::= -(Expression , Expression)
zero?-exp (exp1)	Expression ::= zero? (Expression)
if-exp (exp1 exp2 exp3)	Expression ::= if Expression then Expression else Expression
var-exp (var)	Expression ::= Identifier
let-exp (var exp1 body)	Expression ::= let Identifier = Expression in Expression
proc-exp (var body)	Expression ::= proc (Identifier) Expression
cal-exp (exp1 exp2)	Expression ::= (Expression Expression)
letrec-exp(proc-name bvar proc-body letrec-body)	Expression ::= letrec Identifier (Identifier)= Expression in Expression

טבלה 5.3: תחביר שפת "וייסוג"

```

else -((double -(x,1)), -2)
in (double 6)

```

ההגדרה תראה כך:

```

(define the-grammar
  '((program (expression) a-program)

    ...

    (expression
      ("letrec"
        identifier "(" identifier ")" "=" expression
        "in" expression)
      letrec-exp)
  ))

```

5.5.2 אפיון של ביטויי letrec

ביטוי letrec מורחב בשפת "וייסוג" על-ידי הביטוי:

$$\begin{aligned} & (\text{value-of } (\text{letrec-exp } \text{proc-name } \text{bound-var } \text{proc-body } \text{letrec-body}) \rho) \\ = & (\text{value-of } \text{letrec-body } (\text{extend-env-rec } \text{proc-name } \text{bound-var } \text{proc-body } \rho)) \\ & \text{כאשר טיפוס הנתונים המופשט עבור ייצוג סביבה מורחב אף הוא באופן הבא:} \end{aligned}$$

$$\text{empty-env} : \rightarrow Env$$

$$\text{extend-env} : Var \times Val \times Env \rightarrow Env$$

$$\text{extend-env} : Var \times Var \times Exp \times Env \rightarrow Env$$

$$\text{apply-env} : Env \times Var \rightarrow Val$$

$$(\text{apply-env } (\text{extend-env } \text{var } \text{val } \text{env}) \text{ var}) = \text{val}$$

$$(\text{apply-env } (\text{extend-env } \text{var}_1 \text{ val } \text{env}) \text{ var}_2) =$$

$$(\text{apply-env } \text{env } \text{var}_2) \quad (\text{var}_1 \neq \text{var}_2)$$

$$(\text{apply-env } (\text{extend-env-rec } \text{var } \text{bvar } \text{body } \text{env}) \text{ var}) =$$

$$(\text{proc-val}$$

$$(\text{procedure } \text{bvar } \text{body } (\text{extend-env-rec } \text{var } \text{bvar } \text{body } \text{env})))$$

$$(\text{apply-env } (\text{extend-env-rec } \text{var}_1 \text{ bvar } \text{body } \text{env}) \text{ var}_2) =$$

$$(\text{apply-env } \text{env } \text{var}_2) \quad (\text{var}_1 \neq \text{var}_2)$$

Revision : 2.16.05

מימוש ההרחבה הוא תרגיל תכנות פשוט. לאחר מימוש ההרחבה אפשר לממש את המפרש עבור "וייסוג":

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      ...

      (letrec-exp (proc-name bvar proc-body letrec-body)
        (value-of letrec-body
          (extend-env-rec proc-name bvar proc-body env))))
    )))
```

5.5.3 דוגמה לשערוך שגרה רקורסיבית

נסתכל שוב על הדוגמה:

```
letrec double(x) = if zero?(x)
  then 0
  else -((double -(x,1)), -2)
in (double 6)
```

ונשערוך את הביטוי שלב אחר שלב:

```
(value-of <<letrec double(x) = if zero?(x)
  then 0
  else -((double -(x,1)), -2)
in (double 6)>> ρ) =
```



```
= (value-of <<(double 6)>>
  (extend-env-rec double x <<if zero?(x) ... >> ρ)) =
```

```
= (apply-procedure
  (value-of <<double>>
    (extend-env-rec double x <<if zero?(x) ... >> ρ))
  (value-of <<6>>
    (extend-env-rec double x <<if zero?(x) ... >> ρ))) =
```

```
= (apply-procedure
  (procedure x <<if zero?(x) ... >>
    (extend-env-rec double x <<if zero?(x) ... >> ρ))
  6)
```

```
= (value-of
  <<if zero?(x) ...>>
  [x=6](extend-env-rec double x <<if zero?(x) ...>> ρ)) =
```

```
= (-
  (value-of
    <<(double -(x,1))>>
    [x=6](extend-env-rec double x <<if zero?(x) ...>> ρ))
  -2) =
```

Revision : 2.16.05

```

= (-
  (apply-procedure
    (value-of
      <<double>>
      [x=6](extend-env-rec double x <<if zero?(x) ...>> ρ))
    (value-of
      <<-(x,1)>>
      [x=6](extend-env-rec double x <<if zero?(x) ...>> ρ)))
  -2) =

= (-
  (apply-procedure
    (procedure x <<if zero?(x) ...>>
      (extend-env-rec double x <<if zero?(x) ...>> ρ))
    5)
  -2) =

= ...

```

```

= (-
  (apply-procedure
    (procedure x <<if zero?(x) ...>>
      (extend-env-rec double x <<if zero?(x) ...>> ρ))
    5)
  -2) =

= ...

```

✦ שימו לב שבשלב האחרון נמצאה השלמה הנכונה. ✦

5.6 תחימה, כריכה, והיקף

👉 קראו את סעיף §3.5 בספר הלימוד. 👈

נבדיל בין הכרזה על משתנה לבין הפניה אל משתנה. משתנה יכול להיות מוכרז (declared) ויכול להיות מופנה (referenced). הכרזה על משתנה כורכת (bind) את המשתנה עם הערך המצוין, ונאמר כי הגדרת המשתנה נכרכה (bound) עם הערך המצוין. הפניה אל משתנה היא שימוש במשתנה. למשל, בביטוי

$$(f \ x \ y)$$

כל המשתנים המופעים הם הפניות. אולם בביטוי

$$(\text{lambda } (x) \ \dots)$$

או בביטוי

$$(\text{let } ((x \ \dots)) \ \dots)$$

המופע של x הוא הכרזה על משתנה. כלומר, הוא מצהיר על המשתנה כשם של ערך מסוים.

ברוב שפות התכנות, הפניה אל משתנה מתייחסת לאחת ההכרזות על המשתנה. ייתכן שיש כמה הכרזות כאלה, שלכולן אותו שם. במקרה כזה, חוקי התחימה קובעים איזו הכרזה רלוונטית. כלומר, לאיזו הכרזה הכוונה. כאשר אפשר לקבוע לכל הפניה באיזו הכרזה על משתנה מדובר מבלי להריץ את התכנית, נאמר שחוקי התחימה הם סטטיים.

תחימה לקסיקלית היא הנפוצה ביותר. בתחימה לקסיקלית, לכל הכרזה מוגדר תחום ההכרזה (scope) שבו ניתן להפנות אל המשתנה המוגדר, אלא אם כן הכרזה נוספת של משתנה עם אותו שם יצרה חור בתחום. יש המגדירים את מתאר תחום

Revision : 2.16.05

ההכרזה כך שיכלול את החורים, ויש שמגדירים אותו ללא החורים. מתאָר התחום נקרא בלועזית קונטור (contour).
לדוגמה:

```
(let ((x 3) ← first x
      (y 4))
  (+ (let ((x ← second x
            (+ y 5)))
      (* x y)) ← second x
      x)) ← first x
```

היקף הכריכה (extent) הוא טווח הזמן שבו המשתנה כרוך לערכו. בשפת "וישגור" ובשפת "וייסוג", לכריכות יש היקף לא־מיועד (indefinite extend). כלומר, לכאורה ההיקף בלתי־מוגבל בזמן. למעשה, חלון הזמן נסגר כאשר המשתנה לא חיוני עוד, או ליתר דיוק - בעת "איסוף האשפה".

כאשר ההיקף מוגבל לחישוב של ביטוי מסוים, הוא נקרא היקף דינמי (dynamic extent).

5.6.1 חילוץ שמות של משתנים

קראו את סעיף §3.6 בספר הלימוד. 📖

העומק המילולי (הלקסיקלי) או העומק הסטטי של הפניה למשתנה הוא המרחק שבין ההפניה להכרזה המתאימה, אשר נמדד במספר גבולות התחום שיש לחצות בכדי להגיע להכרזה. אם ידוע העומק המילולי של הפניה למשתנה בשפת "וישגור" או בשפת "וייסוג", אז אפשר לאתר את ההכרזה של המשתנה גם ללא ידיעת שמו. למשל, בביטוי $(* x y)$ שבדוגמה, העומק המילולי של x הוא 0, ואילו העומק המילולי של y הוא 1.

על-ידי שינוי הפעולה `apply-env` כך שתיקח כתובת מילולית (לקסיקלית) במקום שם משתנה, נוכל לחפש ערך של משתנה ללא צורך בשמו. מהדירים (קומפיילרים) עושים בכך שימוש ליצירת קוד מטרה יעיל יותר.

5.6.2 כיתוב מילולי (לקסיקלי)

👉 קראו את סעיף §3.7 בספר הלימוד. 👈

יש אפשרות לכתוב מתרגם אשר יחליף את כל שמות המשתנים מתכנית נתונה, וימיר אותם בהפניה למשתנים באמצעות כתובת לקסיקלית. לחילופין, נוכל לשנות את השפה. לדוגמה, במקום לרשום

```
(lambda (x)
  ((lambda (a)
    (x a))
   x))
```

נוכל לרשום:

```
(nameless-lambda
  ((nameless-lambda
    (#1 #0))
   #0))
```

✦ המספרים נקראים כתובות מילוליות (לקסיקליות) או אינזקסי דה-ברוין, על שם המתמטיקאי ההולנדי ניקולס גוברט דה-ברוין. ✦

5.7 תרגילים נוספים

✦ שאלות שהופיעו בהחיות פתרון לתרגילים אלה אפשר למצוא באתר הקורס. ✦

★תרגיל 5.21 הרחיבו את שפת "ויהי" (שפת LET) עם ביטוי חדש `let*` הפועל בצורה דומה לביטוי `let*` של שפת Scheme. לדוגמה, בשפה המורחבת תוצאת הביטוי הבא תהיה 2:

```
let
  x=30
in
  let*
    x= -(x,1)
    y= -(x,2)
  in
    -(x,y)
```

★תרגיל 5.22 (שאלה מבחינה) ברצוננו להרחיב את שפת ויהי (שפת LET) עם ביטוי חדש בשם `cond`. להלן הדקדוק המגדיר את התחביר המוחשי של `cond`:

$$Expression ::= \text{cond} \{ Expression ==> Expression \}^* \text{end}$$

כאשר התחביר המופשט הוא: `cond-exp (exps1 exps2)`

בביטוי `cond` הביטויים הרשומים משמאל לסימן `==>` משוערכים לפי הסדר עד אשר ערכו של אחד מהם הוא `true`. ערכו של כל ביטוי ה-`cond` שווה לערכו של הביטוי הרשום מימין לסימן `==>` השייך לביטוי משמאל שערכו היה `true`. במקרה שערכם של כל הביטויים הרשומים משמאל

לסימן $\Rightarrow$ הוא *false* תודפס הודעת שגיאה. להלן דוגמה להמחשת השימוש בביטוי החדש:

```
let
  x=5
in
  cond
    zero? (x) ==> 7
    zero? (-(x,5)) ==> 12
    zero? (-(x,2)) ==>70
  end
```

ערכו של כל ביטוי ה־let בדוגמה זו הוא 12.

★**תרגיל 5.23** (שאלה מבחינה) שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות. ברצוננו להרחיב את השפה עם ביטוי חדש בשם fold. להלן הדקדוק המגדיר את התחביר המוחשי של fold:

$$Expression ::= \text{fold } Expression \ Expression \ [\{ Expression \}^{*(.)}]$$

כאשר התחביר המופשט הוא: fold-exp (proc1 acc vals)

ביטוי fold, מכיל את המרכיבים הבאים: ביטוי proc1 שאמור להיות פרוצדורה עם פרמטר יחיד. ביטוי נוסף acc המכיל ערך התחלתי, ורשימת ביטויים vals המופרדים בפסיק ורשומים בתוך סוגריים מרובעים. ביטוי fold מפעיל את הפרוצדורה proc1 על כל אחד מאיברי הרשימה

Revision : 2.16.05

`vals`, וצובר את תוצאות ההפעלות (פעולת חיבור) ל-`acc`. לבסוף מוחזר ערכו של הצובר `acc` כערכו של כל ביטוי `fold`. להלן דוגמה להמחשת השימוש בביטוי החדש:

```
let
    x = fold proc (y) {(y , -1) 0 [12,{(4,7),9]}
in
    -(x,5)
```

ביטוי ה-`let` מגדיר משתנה בשם x ושם בו את תוצאת ביטוי ה-`fold` שהיא 21, זאת מכיוון שהפעלת הפרוצדורה (המחזירה את הפרמטר שקיבלה בתוספת 1) על כל אחד מאיברי הרשימה הנתונה, תוך שימוש בצובר התחלתי שערכו 0 יוצרת את הסכום 21. ולכן ערכו של כל ביטוי ה-`let` בדוגמה זו הוא 16.

★**תרגיל 5.24** (שאלה מבחינה) הרחיבו את שפת LET עם ביטוי בוליאני שתחבירו המוחשי נתון על-ידי הדקדוק הבא:

$$Expression ::= \text{check } Expression \{ \text{and } Expression \}^* \text{end}$$

ותחבירו המופשט הוא: `check-exp (bexp bexps)`

ביטוי `check` מאפשר שרשור של כמה ביטויים בוליאניים עם פעולת `and` ביניהם. המשמעות של ביטוי `check` היא בוליאנית, וערכו הוא `true` אם ורק אם ערכו של כל אחד מהביטויים הבוליאניים ששורשרו על-ידי `and` הוא `true`. אחרת תוצאתו היא `false`. בחישוב של ביטוי `check` הביטויים הבוליאניים מחושבים משמאל לימין, והחישוב נעשה בצורת `short-circuit`, דהיינו, אם ערכו של אחד מהביטויים הבוליאניים הוא `false`,

החישוב נפסק וערכו של כל הביטוי `check` הוא `false`. על מנת לסייע למנתח התחבירי (`parser`) לזהות ביטוי בוליאני, הביטוי מתחיל במילה `check` ומסתיים במילה `end`.

בתחביר מופשט, הביטוי נקרא `check-exp` והוא מורכב מ-`bexp` המציין את הביטוי הבוליאני הראשון ומ-`bexps` שהיא רשימה של יתר הביטויים הבוליאניים.

להלן דוגמאות מספר שימחישו את השימוש בביטוי החדש:

```
check zero? (x) end
check zero? (x) and zero? (y) end
check zero? (x) and zero? (-(y,1)) end
if check zero? (x) and zero? (y) end then 3 else 4
```

★תרגיל 5.25 (שאלה מבחינה) שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות. הרחיבו את השפה עם ביטוי חדש בשם `count`. להלן הדקדוק המגדיר את התחביר המוחשי של `count`:

$$Expression ::= count Expression [\{ Expression \}^{(*)}]$$

כאשר התחביר המופשט הוא: `count-exp (proc1 vals)`

ביטוי `count`, מכיל את המרכיבים הבאים: ביטוי `proc1` שאמור להיות פרוצדורה עם פרמטר יחיד. ורשימת ביטויים המופרדים בפסיק ורשומים בתוך סוגריים מרובעות. הפרוצדורה המוזכרת בביטוי `count` אמורה לפעול על פרמטר מסוג מספר ולהחזיר עבורו תשובה בוליאנית. ביטוי `count` מפעיל את הפרוצדורה `proc1` על כל אחד מאיברי הרשימה

`vals`, ומחזיר כתשובה כמה ערכי `true` הוחזרו מהפעלת `proc1` על כל אחד מאיברי הרשימה `vals`. להלן דוגמה להמחשת השימוש בביטוי החדש:

```
count proc (y) zero? (-(y,7)) ]7,-(4,7),9,-(9,2)]
```

בדוגמה זו, הפרוצדורה הרשומה במסגרת ביטוי ה-`count` מקבלת פרמטר y ובודקת האם ערכו הוא 7. ברשימת הערכים המוזכרת בדוגמה ניתן לראות שרק האיבר הראשון, והאיבר האחרון ברשימה ערכם הוא 7, ולכן הערך שיוחזר עבור דוגמה זו הוא 2.

★**תרגיל 5.26** (שאלה מבחינה) שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

הרחיבו את השפה עם ביטוי חדש בשם `compose`. להלן הדקדוק המגדיר את התחביר המוחשי של `compose`:

$$Expression ::= \text{compose} [\{ Expression \}^{+(i)}] Expression$$

כאשר התחביר המופשט הוא: `compose-exp (procs val)`

ביטוי `compose` מכיל את המרכיבים הבאים: `procs` היא רשימת ביטויים עם לפחות ביטוי אחד המופרדים בתו ";", וכן ביטוי `val`. ברשימת הביטויים `procs` מופיעים ביטויים המייצגים פרוצדורות בלבד. ביטוי `compose` מפעיל את הפרוצדורה הראשונה ברשימה על הביטוי המיוצג על-ידי `val`, לאחר מכן מופעלת הפרוצדורה השנייה ברשימה על הערך שהוחזר מההפעלה של הפרוצדורה הראשונה. וכך כל פרוצדורה ברשימה (החל מהשנייה) מופעלת על פרמטר שהוא תוצאת ההפעלה של הפרוצדורה שקדמה לה. ערכו של כל ביטוי ה-`compose` הוא התשובה המוחזרת מההפעלה של הפרוצדורה האחרונה ברשימה. להלן דוגמה להמחשת השימוש בביטוי

החדש:

```

let
  p1 = proc (x) -(x, -(0,3))
in
  let
    p2 = proc (x) -(x, 7)
  in
    compose [p1; p2; proc (x) {(10,x)}] 5

```

בדוגמה זו, הערך הסופי המוחזר הוא תוצאת ביטוי ה־`compose`.
 בביטוי זה מופעלת תחילה הפרוצדורה `p1` על הפרמטר 5 ומחזירה 8,
 לאחר מכן מופעלת הפרוצדורה `p2` עם הפרמטר 8 ומחזירה 1, לאחר מכן
 מופעלת הפרוצדורה האחרונה ברשימה על הפרמטר 1 ומחזירה 9. ולכן
 ערכה של תכנית זו הוא 9.

פרק 6

המצב

עד כה דנו בתוצאה של פעולת חישוב: הערך (value) המתקבל מפעולת החישוב. אולם לפעולת חישוב יכולה להיות גם תוצאה (effect): השפעה על מצב המערכת. פרק זה דן במצב.

פרק זה מלווה את פרק 4 בספר הלימוד.

6.1 תוצא חישובי

קראו את סעיף §4.1 בספר הלימוד.

ביטויים בדרך כלל משערכים לערך כלשהו. אך בנוסף יש להם לעתים גם תוצא, כגון שינוי מצב מתקן הקלט-פלט, או מצב מערכת ההפעלה, או מצב הרשת, או מצב זיכרון המחשב (ערכים השמורים במיקום כלשהו בזיכרון של המחשב). לתוצאים, שלא כמו לערכים, יש השפעה גלובלית. השפעתם ניכרת על החישוב כולו. למשל, בעוד שלכריכה של משתנה יש השפעה מקומית בלבד, השמה למשתנה משותף יכולה להשפיע על ההתנהגות של מודולים אחרים שאינם כוללים השמה לאותו משתנה.

Revision : 2.16.06

אנו נתמקד בהשמה לכתובות בזיכרון. נמדל את זיכרון המחשב על־ידי החסן. **החסן** (store) הוא התאמה סופית בין מיקומים לערכים בני־איחסון. במרבית השפות, כל ערך ב־ר־איחסון (storable) הוא לרוב גם ערך ב־ר־הבעה (expressed), וההפך. אבל זה לא בהכרח כך.

משתנה או מבנה נתונים שמייצג מיקום נקרא **הפניה** או **ר־פָּרֹר** (reference). **מיקום** הוא אזור בזיכרון שניתן לאחסן בו ערכים. **הפניה** היא מבנה נתונים שמפנה לאותו מקום.

הפניה נקראת לפעמים **ערך בהסבת שמאל** (L-value) כי היא מופיעה בצד שמאל של פעולת השמה של ערך להפניה. ערך ב־ר־איחסון נקרא לעתים **ערך בהסבת ימין** (R-value) כי הוא מופיע בצד ימין של פעולת השמה (ומשוערך לערך ב־ר־איחסון).

6.2 "ויפנה": שפה עם הפניות מפורשות

קראו את סעיף §4.2 בספר הלימוד. 📖

בשפה עם הפניות מפורשות, הפניות הן סוג של ערך ב־ר־הבעה.

$$ExpVal = Int + Bool + Proc + Ref(ExpVal)$$

$$DenVal = ExpVal$$

מבנה הנתונים של הפניות הוא מבנה נתונים הניתון לשינוי (mutable), ולכן לא נאפיין אותו אלגברית.

נותיר את חוקי הכריכה בעינם. אולם, נוסיף פעולות ליצירה ולשימוש בהפניות:

• **newref** לוקח כארגומנט ערך ב־ר־איחסון, מקצה מיקום חדש, שומר את הארגומנט במיקום החדש, ומחזיר הפניה למיקום.

• `deref` לוקח כארגומנט הפניה ומחזיר את תוכנה.

• `setref` לוקח כארגומנטים הפניה וערך בר־איחסון, ומאחסן את הערך במיקום שאליו מפנים.

הנה דוגמה לשתי שגרות שמתקשרות ביניהן באמצעות משתנה משותף. הן אינן נזקקות לפרמטרים. כאן מועבר מספר סתמי 888 כאגומנט, הואיל ואין יכולת להגדיר בשפה זו שגרות ללא פרמטרים. התכנית בודקת האם 13 הוא מספר אי־זוגי.

```
let x = newref(0)
in letrec even(dummy)
  = if zero?(deref(x))
    then zero?(0)
    else begin
      setref(x, -(deref(x),1));
      (odd 888)
    end
  odd(dummy)
  = if zero?(deref(x))
    then zero?(1)
    else begin
      setref(x, -(deref(x),1));
      (even 888)
    end
  in begin setref(x,13); (odd 888) end
```

★תרגיל 6.1 השוו את הקוד כאן לקוד שמופיע בעמוד 105 בספר. מה הם ההבדלים ומה משמעותם?

★תרגיל 6.2 תרגיל 3.31 בעמוד 84 בספר.

★תרגיל 6.3 תרגיל 3.32 בעמוד 84 בספר.

★תרגיל 6.4 תרגיל 4.4 בעמוד 108 בספר.

Revision : 2.16.06

הנה דוגמה לשגרה עם מצב נסתר הניתן לשינוי. כאן השגרה g שומרת מונה פרטי שסופר כמה פעמים קראו ל- g . התכנית מחזירה 3.

```
let g = let counter = newref(0)
      in proc (dummy)
        begin
          setref(counter, -(deref(counter), -1));
          deref(counter)
        end
      in let a = (g 11)
        in let b = (g 11)
          in let c = (g 11)
            in c
```

בשפת "ויפנה", ההפניות הן ערכים מדרגה-ראשונה. כלומר, הפניה היא ערך בר-הבעה, וכל ערך בר-הבעה הוא גם בר-אחסון, ולכן ניתן לאחסן הפניות. בדוגמה כאן, x מכיל הפניה למיקום שבו שמורה הכתובת שבו מאוחסן 0. התכנית מחזירה 321.

```
let x = newref(newref(0))
  in begin
    setref(deref(x), 321);
    deref(deref(x))
  end
```

6.3 אפיון מבוסס העברת-החסן

קראו את סעיף §4.2.1 בספר הלימוד.  

נעשה שימוש בקיצורים האלה:

• σ – מציין החסן כלשהו.

• $\square$ - מציין החסן ריק.

• $[l = v]\sigma$ - מציין החסן הדומה ל- σ , אלא שבמיקום l מאוחסן הערך v .

באפיון מבוסס העברת-החסן, מעבירים את ההחסן כארגומנט מפורש לפעולה value-of, אשר מחזירה זוג תוצאות: ערך ברהבעה והחסן (מעודכן).

6.3.1 אפיון אלגברי של שלושה סוגי ביטויים

6.3.1.1 פעולה עם תוצא

$$(\text{value-of } \exp_1 \rho \sigma) = (val_1, \sigma_1)$$

כאשר ההחסן במצב σ , הביטוי $\exp_1$ משוערך בסביבה ρ ל- val_1 , ומשנה את מצב ההחסן ל- σ_1 .

6.3.1.2 פעולה ללא-תוצא

$$(\text{value-of } (\text{const-exp } n) \rho \sigma) = ((\text{num-val } n), \sigma)$$

ניתן לראות כי הביטוי לא משנה את ההחסן.

Revision : 2.16.06

6.3.1.3 פעולות עם תוצא סדרתי

$$\begin{array}{l}
 (\text{value-of } exp_1 \rho \sigma) = (val_1, \sigma_1) \\
 (\text{expval} \rightarrow \text{num } val_1) = n_1 \\
 (\text{value-of } exp_2 \rho \sigma_1) = (val_2, \sigma_2) \\
 (\text{expval} \rightarrow \text{num } val_2) = n_2 \\
 n_1 - n_2 = n \\
 \hline
 (\text{value-of } (\text{diff-exp } exp_1 exp_2) \rho \sigma) = ((\text{num-val } n), \sigma_2)
 \end{array}$$

כאן exp_1 משוערך כאשר ההחסן במצב σ , ומותיר אותו במצב σ_1 . exp_2 משוערך כאשר ההחסן במצב σ_1 ומותיר אותו במצב σ_2 . לבסוף מחושב ומוחזר ההפרש שבין הערך של שני הביטויים ללא שינוי נוסף במצב ההחסן.



◆ רצף זיכרון של if-exp בעמוד 108 בספר.

6.3.2 אפיון פעולות על הפניות מפורשות

נרחיב את השפה ונוסיף לה אפשרות הפניה מפורשת. תחביר של שפת "ויפנה" מתואר בטבלה 6.1.

6.3.3 הגדרת פעולות על הפניות המפורשות

$$\frac{(\text{value-of } exp_1 \rho \sigma) = (val_1, \sigma_1) \quad l \notin \text{dom}(\sigma_1)}{(\text{value-of } (\text{newref-exp } exp_1) \rho \sigma) = ((\text{ref-val } l), [l = val_1]\sigma_1)}$$

תחביר מופשט	תחביר מוחשי
a-program (exp1)	Program ::= Expression
const-exp (num)	Expression ::= Number
diff-exp (exp1 exp2)	Expression ::= -(Expression , Expression)
zero?-exp (exp1)	Expression ::= zero? (Expression)
if-exp (exp1 exp2 exp3)	Expression ::= if Expression then Expression else Expression
var-exp (var)	Expression ::= Identifier
let-exp (var exp1 body)	Expression ::= let Identifier = Expression in Expression
proc-exp (var body)	Expression ::= proc (Identifier) Expression
cal-exp (exp1 exp2)	Expression ::= (Expression Expression)
letrec-exp(proc-name bvar proc-body letrec-body)	Expression ::= letrec Identifier (Identifier)= Expression in Expression
newref-exp (exp1)	Expression ::= (newref Expression)
deref-exp (exp1)	Expression ::= (deref Expression)
setref-exp (exp1 exp2)	Expression ::= (setref Expression , Expression)

טבלה 6.1: תחביר שפת "ויפנה"

החוק דלעיל אומר ש- newref-exp משערך את האופרנד exp_1 לקבלת ערך val_1 . הוא מקצה מיקום חדש l בהחסן, ושומר במיקום זה את val_1 . הערך המוחזר הוא הפניה למיקום החדש l .

$$\frac{(\text{value-of } exp_1 \ \rho \ \sigma) = (l, \sigma_1)}{(\text{value-of } (\text{deref-exp } exp_1) \ \rho \ \sigma) = (\sigma_1(l), \sigma_1)}$$

חוק זה אומר ש- deref-exp משערך את האופרנד לערך, שאמור להיות הפניה למיקום l . הערך המוחזר הוא תוכן ההחסן במיקום l , מבלי לשנות את מצב ההחסן.

Revision : 2.16.06

$$\frac{\begin{array}{l} (\text{value-of } \text{exp}_1 \rho \sigma) = (l, \sigma_1) \\ (\text{value-of } \text{exp}_2 \rho \sigma_1) = (val, \sigma_2) \end{array}}{(\text{value-of } (\text{setref-exp } \text{exp}_1 \text{exp}_2) \rho \sigma) = ([23], [l = val] \sigma_2)}$$

חוק זה אומר ש־ `setref-exp` משערך את האופרנדים משמאל לימין. האופרנד הראשון חייב להיות משוערך למיקום. לאחר מכן, הסביבה מורחבת, והערך של האופרנד השני נשמר במיקום החדש. הערך המוחזר שרירותי וחסר משמעות.

★תרגיל 6.5 תרגיל 4.6 בעמוד 109 בספר.

★תרגיל 6.6 תרגיל 4.7 בעמוד 109 בספר.

6.3.4 מימוש

בדרך כלל יש החסן אחד בלבד. זאת בניגוד לסביבות, שמהן יש הרבה. ננצל עובדה זאת, ונממש את ההחסן כמשתנה גלובלי, כך שלא נצטרך להעביר אותו כארגומנט לפעולה `value-of`.

```
;; value-of-program : Program -> ExpVal
```

```
(define value-of-program
  (lambda (pgm)
    (initialize-store!) ; new for explicit refs.
    (cases program pgm
      (a-program (exp1)
        (value-of exp1 (init-env))))))
```

Listing 6.1: value-of-program.scm

נממש את הפעולות על ההחסן כך:

```

;;; World's dumbest implementation of stores:
;;; a store is a list and
;;; a reference is an integer index into the list.

;;; the-store : Store
;;; the-store is the current store

(define the-store (empty-store))

;;; reference? : SchemeVal -> Boolean
;;; (reference? x) is true iff x represents a location

(define reference? integer?)

;;; initialize-store! : -> Unspecified
;;; (initialize-store!) sets the store to the empty store

(define initialize-store!
  (lambda ()
    (set! the-store (empty-store))))

;;; empty-store : -> Store
;;; (empty-store) returns an empty store

(define empty-store
  (lambda () '()))

```

Listing 6.2: store-impl.scm

Revision : 2.16.06

```

;;; newref : ExpVal -> Ref
;;; (newref v) returns a newly allocated reference
;;; initialized to v

(define newref
  (lambda (val)
    (let ((next-ref (length the-store)))
      (set! the-store
        (append the-store (list val)))
      (if (instrument-newref)
        (eopl:printf
          "newref: allocating location ~s~%"
          next-ref))
      next-ref)))

```

Listing 6.3: store-impl.scm/newref

```

;;; deref : Ref -> ExpVal
;;; (deref ref) returns the value currently contained in ref

(define deref
  (lambda (ref) (list-ref the-store ref)))

```

Listing 6.4: store-impl.scm/deref

```

;;; setref! : Ref * ExpVal -> Unspecified
;;; (setref! ref val) changes the current store
;;; by storing val into ref

(define setref!
  (lambda (ref0 val)
    (set! the-store

      ;; setref-inner : Store * Ref -> Store
      ;; (setref-inner sigma l) returns a store
      ;; that is like sigma except that,
      ;; in the store returned by setref-inner,
      ;; the location l contains val.

      (letrec ((setref-inner (lambda (store ref)
                              (cond
                                ((null? store)
                                 (eopl:error 'setref
                                              "illegal reference"))
                                ((zero? ref)
                                 (cons val (cdr store)))
                                (else
                                 (cons
                                  (car store)
                                  (setref-inner (cdr store)
                                                (- ref 1)))))))
                (setref-inner the-store ref0))))))

```

Listing 6.5: store-impl.scm/set-ref!

לבסוף נרחיב את המפרש:

Revision : 2.16.06

```

(newref-exp (exp1)
  (let ((v1 (value-of exp1 env)))
    (ref-val (newref v1))))

(deref-exp (exp1)
  (let ((v1 (value-of exp1 env)))
    (let ((ref1 (expval->ref v1)))
      (deref ref1))))

(setref-exp (exp1 exp2)
  (let ((ref (expval->ref (value-of exp1 env))))
    (let ((v2 (value-of exp2 env)))
      (begin
        (setref! ref v2)
        (num-val 23)))))

```

Listing 6.6: store-interp.scm

★תרגיל 6.7 תרגיל 4.8 בעמוד 112 בספר.

★תרגיל 6.8 תרגיל 4.9 בעמוד 112 בספר.

★תרגיל 6.9 תרגיל 4.10 בעמוד 112 בספר.

★תרגיל 6.10 תרגיל 4.11 בעמוד 112 בספר.

6.4 "וירמוז": שפה עם הפניות מרומזות (IMPLICIT-)

(REFS)

👉 קראו את סעיף §4.3 בספר הלימוד. 👈

בשפה עם הפניות מרומזות, כל משתנה מציין הפניה. הערכים הניתנים לציון הם הפניות למיקומים שמכילים ערכים שניתנים לביטוי (הבעה).

ערך בריהבעה (ביטוי) הוא מספר, או ערך בוליאני, או שגרה, או הפניה:

```
;;; an expressed value is either a number, a boolean, a procval, or a
;;; reference.
```

```
(define-datatype expval expval?
  (num-val
    (value number?))
  (bool-val
    (boolean boolean?))
  (proc-val
    (proc proc?))
  (ref-val
    (ref reference?))
)
```

Listing 6.7: implicit-refs/data-structures.scm/define-datatype

נממש את הפעולה `:expval->ref`

```
(define expval->ref
  (lambda (v)
    (cases expval v
      (ref-val (ref) ref)
      (else (expval-extractor-error 'reference v)))))
```

Listing 6.8: implicit-refs/data-structures.scm/expal->ref

נרחיב את טיפוס הנתונים המופשט המייצג סביבה:

Revision : 2.16.06

```
(define-datatype environment environment?
  (empty-env)
  (extend-env
    (bvar symbol?)
    (bval reference?) ; new for implicit-refs
    (saved-env environment?))
  (extend-env-rec*
    (proc-names (list-of symbol?))
    (b-vars (list-of symbol?))
    (proc-bodies (list-of expression?))
    (saved-env environment?)))
```

Listing 6.9: implicit-refs/data-structures.scm/define-datatype

נממש את הפעולה `:env->list`

```
;; env->list : Env -> List
;; used for pretty-printing and debugging
(define env->list
  (lambda (env)
    (cases environment env
      (empty-env () '())
      (extend-env (sym val saved-env)
        (cons
          (list sym val) ; val is a denoted value-- a
                        ; reference.
          (env->list saved-env))))
      (extend-env-rec* (p-names b-vars p-bodies saved-env)
        (cons
          (list 'letrec p-names '...)
          (env->list saved-env)))))))
```

Listing 6.10: implicit-refs/data-structures.scm

ונממש את הפעולה `expval->printable`:

```
;; expval->printable : ExpVal -> List
;; returns a value like its argument, except procedures get cleaned
;; up with env->list
(define expval->printable
  (lambda (val)
    (cases expval val
      (proc-val (p)
        (cases proc p
          (procedure (var body saved-env)
            (list 'procedure var '... (env->list saved-env))))))
      (else val))))
```

Listing 6.11: implicit-refs/data-structures.scm

נגדיר משתנה גלובלי עבור הסביבה ונאתחל אותו להיות סביבה ריקה:

Revision : 2.16.06

```

;; the-store: a Scheme variable containing the current state of the
;; store. Initially set to a dummy variable.
(define the-store 'uninitialized)

;; empty-store : () -> Sto
;; Page: 111
(define empty-store
  (lambda () '()))

;; initialize-store! : () -> Sto
;; usage: (initialize-store!) sets the-store to the empty-store
;; Page 111
(define initialize-store!
  (lambda ()
    (set! the-store (empty-store))))

```

Listing 6.12: implicit-refs/store.scm

נממש את הפעולה `:gets-store`

```

;; get-store : () -> Sto
;; Page: 111
;; This is obsolete. Replaced by get-store-as-list below
(define get-store
  (lambda () the-store))

```

Listing 6.13: implicit-refs/store.scm/get-store

נממש את הפעולה `:reference?`

```
;; reference? : SchemeVal -> Bool
;; Page: 111
(define reference?
  (lambda (v)
    (integer? v)))
```

Listing 6.14: implicit-refs/store.scm/reference?

נממש את הפעולה `:newref`

```
;; newref : ExpVal -> Ref
;; Page: 111
(define newref
  (lambda (val)
    (let ((next-ref (length the-store)))
      (set! the-store
        (append the-store (list val)))
      (if (instrument-newref)
        (eopl:printf
          "newref: allocating location ~s with initial contents ~s~%"
          next-ref val))
      next-ref)))
```

Listing 6.15: implicit-refs/store.scm/newref

נממש את הפעולה `:deref`

Revision : 2.16.06

```
;; deref : Ref -> ExpVal
;; Page 111
(define deref
  (lambda (ref)
    (list-ref the-store ref)))
```

Listing 6.16: implicit-refs/store.scm/deref

נממש את הפעולה :setref!

```
;; setref! : Ref * ExpVal -> Unspecified
;; Page: 112
(define setref!
  (lambda (ref val)
    (set! the-store
      (letrec
        ((setref-inner
          ;; returns a list like store1, except that position ref1
          ;; contains val.
          (lambda (store1 ref1)
            (cond
              ((null? store1)
               (report-invalid-reference ref the-store))
              ((zero? ref1)
               (cons val (cdr store1)))
              (else
               (cons
                 (car store1)
                 (setref-inner
                  (cdr store1) (- ref1 1)))))))
          (setref-inner the-store ref))))))
```

Listing 6.17: implicit-refs/store.scm/setref

ונממש את הפעולה `get-store-as-list` אשר מוצאת את תוכן הסביבה כרשימה:

```
;; get-store-as-list : () -> Listof(List(Ref,Expval))
;; Exports the current state of the store as a scheme list.
;; (get-store-as-list '(foo bar baz)) = ((0 foo)(1 bar) (2 baz))
;; where foo, bar, and baz are expvals.
;; If the store were represented in a different way, this would be
;; replaced by something cleverer.
;; Replaces get-store (p. 111)
(define get-store-as-list
  (lambda ()
    (letrec
      ((inner-loop
        ;; convert sto to list as if its car was location n
        (lambda (sto n)
          (if (null? sto)
              '()
              (cons
               (list n (car sto))
               (inner-loop (cdr sto) (+ n 1)))))))
      (inner-loop the-store 0))))
)
```

Listing 6.18: `implicit-refs/store.scm/get-store-as-list`

★תרגיל 6.11 תרגיל 4.15 בעמוד 121 בספר.

★תרגיל 6.12 תרגיל 4.16 בעמוד 121 בספר.

★תרגיל 6.13 תרגיל 4.17 בעמוד 121 בספר.

★תרגיל 6.14 תרגיל 4.18 בעמוד 121 בספר.

★תרגיל 6.15 תרגיל 4.19 בעמוד 121 בספר.

★תרגיל 6.16 תרגיל 4.20 בעמוד 121 בספר.

Revision : 2.16.06

★תרגיל 6.17 תרגיל 4.22 בעמוד 122 בספר.

★תרגיל 6.18 תרגיל 4.23 בעמוד 123 בספר.

★תרגיל 6.19 תרגיל 4.24 בעמוד 123 בספר.

★תרגיל 6.20 תרגיל 4.25 בעמוד 123 בספר.

★תרגיל 6.21 תרגיל 4.27 בעמוד 124 בספר.

6.5 "ויבגוד": שפה עם חילופי זוגות (MUTABLE-PAIRS)

קראו את סעיף §4.4 בספר הלימוד.  

בשפת Scheme ניתן להחליף את הראש או הזנב של "מאָזוג" על ידי הפעולות `set-car!` ו-`set-cdr!`. שפת "ויבגוד" מוסיפה יכולת דומה לשפת "וירמוז".

6.5.1 ערכים

זוג בשפת "ויבגוד" מכיל שני מיקומים. נסמן את קבוצות הערכים:

$$ExpVal = Int + Bool + Proc + MutPair$$

$$DenVal = Ref(ExpVal)$$

$$MutPair = Ref(ExpVal) \times Ref(ExpVal)$$

כאשר $MutPair$ הוא קבוצת ערכים שניתנים לביטוי המייצגים זוגות. נתייחס לערכים המייצגים זוגות באמצעות טיפוס נתונים מופשט:

$$\text{newpair} : \text{Expval} \times \text{Expval} \rightarrow \text{MutPair}$$

$$\text{left} : \text{MutPair} \rightarrow \text{ExpVal}$$

$$\text{right} : \text{MutPair} \rightarrow \text{ExpVal}$$

$$\text{setleft} : \text{MutPair} \times \text{ExpVal} \rightarrow \text{Unspecified}$$

$$\text{setright} : \text{MutPair} \times \text{ExpVal} \rightarrow \text{Unspecified}$$

יתר המימוש מופיע בסעיף 4.4.1 בספר הלימוד.

6.6 העברת פרמטרים

👉 קראו את סעיף §4.5 בספר הלימוד. 👈

פרק זה הוא קריאה עצמית.

6.7 תרגילים נוספים



אלקטרה שופיעו בבחינות

★**תרגיל 6.22** (שאלה מבחינה) בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS). ברצוננו להרחיב שפה זו עם ביטוי חדש בשם `setdynamic`. להלן הדקדוק המגדיר ביטוי זה:

$Expression ::= \text{setdynamic } Identifier = Expression \text{ during } Expression$

`setdynamic-exp (var exp1 body)`

בביטוי `setdynamic` מתבצעת השמה **זמנית** של `exp1` לתוך משתנה `var`, לאחר מכן, משוערך ערכו של ה-`body`, ולבסוף ערכו של המשתנה `var` חוזר לערכו המקורי. הערך המוחזר הוא ערכו של ה-`body`. המשתנה `var` חייב להיות כרוך טרם ביצוע ביטוי `setdynamic`.
להלן דוגמה להמחשת השימוש בביטוי `setdynamic`:

```
let
  x = 11
in
  let
    p = proc (y) -(y,x)
  in
    -(setdynamic x =17 during (p 22) , (p 13))
```

ערכו של המשתנה החופשי `x` בפרוצדורה `p` במסגרת הקריאה (p 22) הנו 17, לעומת זאת ערכו של המשתנה החופשי `x` בפרוצדורה `p` במסגרת הקריאה (p 13) הנו 11. לכן ערכו של כל הביטוי בדוגמה הוא $3 = 2 - 5$.

***תרגיל 6.23** (שאלה מבחינה) בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS). ברצוננו לשנות בשפה זו את אופן ההגדרה של פרוצדורות כך שבתחילת ה- `body` שלהן תהיה אפשרות להגדרת משתנים מקומיים סטטיים (`static variables`). משתנה מקומי סטטי מאותחל בקריאה הראשונה לפרוצדורה, ובכל קריאה אחרת לפרוצדורה (השנייה והלאה) ערכו ההתחלתי שווה לערך האחרון שלו מהקריאה הקודמת. במילים אחרות, זהו משתנה משותף לכל הקריאות לפרוצדורה. נשנה את אופן ההגדרה של פרוצדורות על פי הדקדוק הבא המאפשר הגדרת משתנים מקומיים סטטיים:

Expression ::=

`proc(Identifier) {static Identifier = Expression}* Expression`

`proc-exp (var statvars statvals body)`

נסביר דקדוק זה. לפרוצדורה יש פרמטר יחיד (`var`) לאחר מכן, יכולים להופיע 0 או יותר משתנים מקומיים (לוקליים) סטטיים (`statvars`) ולכל משתנה ערך התחלתי הניתן על-ידי (`statvals`) ולבסוף מופיע ביטוי שהוא גוף הפרוצדורה (`body`).

להלן דוגמה להמחשת השימוש באופן החדש להגדרת פרוצדורות:

```
let
  p = proc (x)
    static y = 5
    static z = 10
    begin
      set y = -(z,x);
      set z = -(x,4);
      -(y,z)
```

Revision : 2.16.06

```

    end
in
  -((p 12), (p 2))

```

ערכו של ביטוי זה הוא: 18- , בקריאה הראשונה (p 12) מוחזר ערך של 10-, ואילו בקריאה השנייה (p 2) מוחזר ערך של 8.

★ תרגיל 6.24 (שאלה מבחינה)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS). להזכירכם, בשפה זו, הפרמטר הפורמלי של פרוצדורה מועבר בצורת call-by-value, כלומר, נוצר מקום חדש בהחסן המכיל העתק של ערכו של הפרמטר האקטואלי (מה שנשלח לפרוצדורה), והפרמטר הפורמלי של הפרוצדורה (זה שמקבל את מה שנשלח אליו) הוא reference לאותו מקום שנוצר בהחסן. כך שאם בגוף הפרוצדורה ישונה ערכו של הפרמטר הפורמלי, זה אינו אמור להשפיע כלל על הפרמטר האקטואלי. ברצוננו להרחיב שפה זו עם ביטוי חדש שיאפשר להפעיל פרוצדורה שמקבלת כפרמטר פורמלי reference אל הפרמטר האקטואלי שנשלח. כך שאם בגוף הפרוצדורה נשנה את ערכו של הפרמטר הפורמלי זה מיד ישפיע על ערכו של הפרמטר האקטואלי (במילים אחרות יש כאן העברת פרמטר בצורת call-by-reference) להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

$$Expression ::= \text{callref } Expression \text{ (ref Identifier)}$$

callref-exp (proc var)

ביטוי callref-exp מאפשר להפעיל פרוצדורה ולהעביר לה כפרמטר reference של משתנה כלשהי. ה- $Expression$ המוזכר בביטוי הוא הפרוצדורה

שברצוננו להפעיל, וה- *Identifier* המוזכר בביטוי הוא המשתנה שאת ה-
reference שלו אנו מעוניינים להעביר.

להלן דוגמה: נמחיש תחילה את המצב הקיים (call-by-value):

```
let
  x = 11
in
  let
    p = proc (y) set y = 60
  in
    begin (p x); x end
```

בדוגמה זו, ערכו של x מועבר כפרמטר לפרוצדורה p , המשתנה y מכיל
עותק של ערכו של x , ולכן כאשר משנים את y ל-60 זה אינו משפיע
על x , ולכן ערכו של כל הביטוי בדוגמה הוא כערכו של הביטוי האחרון
בבלוק ה- `begin`, כלומר הדפסת ערכו של x , ולכן נצפה לקבל את הערך
11.

להלן דוגמה להמחשת המצב החדש:

```
let
  x = 11
in
  let p = proc (y) set y = 60
  in
    begin callref p(ref x); x end
```

בדוגמה זו, הקריאה לפרוצדורה p נעשית על פי הביטוי החדש, ולאחר
החזרה מהקריאה, מודפס ערכו של x , ולכן נצפה לקבל את הערך 60
ולא 11.

★תרגיל 6.25 (שאלה מבחינה) בספר הלימוד בעמודים 113-120 מתוארת

שפת "וירמוז" (IMPLICIT-REFS). ברצוננו להרחיב שפה זו עם ביטוי חדש שיאפשר לבצע לולאת while. ערכו של ביטוי זה הוא מספר המחזורים שגוף הלולאה בוצע.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

$$Expression ::= \text{while } Expression \text{ do } Expression$$

while-exp (bool body)

בביטוי while-exp ה־ $Expression$ המוזכר אחר המילה while משמש כתנאי בוליאני עבור הלולאה. ואילו ה־ $Expression$ המוזכר אחר המילה do משמש כגוף הלולאה. ערכו של כל הביטוי הוא מספר הפעמים שגוף הלולאה התבצע. להלן דוגמה:

```
let
  x = 5
in
  let
    p = while zero? (-(x,5)) do set x = {(x,1)}
  in
    p
```

בדוגמה זו, הלולאה מתבצעת רק פעם אחת. ולכן ערכו של p יהיה 1. וזה גם הערך של כל התכנית עצמה.

פרק 7

טיפוסים



זהו פרק טיפוסים לשפות תכנות, תרתי משמע.

פרק זה מלווה את פרק 7 בספר הלימוד.

הפרקים הקודמים דנו במשמעות הדינמית (dynamic semantics) של ביטויים, כפי שהיא נתונה על-ידי האפיון והמימוש של value-of. ואולם, לביטויים תיתכן גם משמעות סטטית (static semantics), שאותה ניתן להסיק מבלי לשערך את הביטוי עצמו.

בטיחות מעוגנת טיפוסים (type safety) היא תכונה חשובה של שפות תכנות (טיפוסים של ביטויים, במקרה שלנו). ניתן לאכוף אותה דינמית (dynamically typed) או סטטית (statically typed). אם ניתן לחשב את הטיפוס המדויק באופן סטטי והדוק, אז נאמר שהשפה ניתנת להטפסה הדוקה (strong typing).

✦ בהטפסה הדוקה (strong typing) הקשר בין הטיפוס לממשלה הטיפוס הדוק. הטפסה הדוקה נקראת אפסחים גם הטפסה חזקה. לעומת זאת, בהטפסה רופפת (weak typing) הקשר בין הטיפוס לממשלה הטיפוס

Revision : 2.16.07

✦ *חופף. הטפסה חופפת נקראת לפעמים גם הטפסה חלשה.*

אם הטיפוס של הביטוי מחושב מבלי לשערך אותו, אז נאמר שהשפה היא מוטפסת באופן סטטי (statically typed) אך לא בהכרח מוטפסת באופן הדוק (strongly typed). למשל, השפה C++ היא כזו, כיוון שמערכת הטיפוסים בה לא נאותה. אף על פי שלביטויים בשפת C++ מחושב טיפוס באופן סטטי, הטיפוס אינו בהכרח חיוני אמין של ערכי הביטוי בזמן ריצה.

בשפת C אין בטיחות מעוגנת טיפוסים. להלן דוגמה לתכנית בשפת C שאינה בטוחה מבחינת הטיפוסים:

```
#include <stdio.h>
#include <stdlib.h>
double f (double * p) {
    *p = 3.14159;
    return *p;
}
int main (int argc, char* argv[]) {
    int n = 12345;
    double * p = (void *) &n;
    double x = f(p);
    double y = f(p+1);
    double z = f(p+2);
    printf ("n = %d\n", n);
    printf ("x = %lf\n", x);
}
```

בהרצת התכנית על מחשבים שונים, נחזה בהתנהגויות שונות ומשונות. נסו!

7.1 הגדרת בטיחות טיפוסים

עבור השפות "ויהי", "וישגור", ו-"וייסוג", בטיחות מעוגנת טיפוסים משמעה:

- בכל שערך של משתנה v , המשתנה v כרוך.
- בכל שערך של ביטוי מהצורה $(\text{diff-exp } exp_1 exp_2)$, הערכים של exp_1 ושל exp_2 הם שניהם מספרים (num-val) .
- בכל שערך של ביטוי מהצורה $(\text{zero?-exp } exp_1)$, הערך של exp_1 הוא מספר (num-val) .
- בכל שערך של ביטוי מהצורה $(\text{if-exp } exp_1 exp_2 exp_3)$, הערך של exp_1 הוא בוליאני (bool-val) .
- בכל שערך של ביטוי מהצורה $(\text{call-exp } exp_1 exp_2)$, הערך של exp_1 הוא שגרה (proc-val) .

נאמר שיש שגיאת טיפוס (type error) אם אחד מהכללים האמורים מופר. נאמר שתכנית בשפת "ויהי", "וישגור", או "וייסוג" היא בטוחת-טיפוס (type-safe) אם ורק אם שערך של ביטויים בשפה לא יכול לגרום לשגיאת טיפוס.

היות שלא כל תכנית בשפת "ויהי", "וישגור", או "וייסוג" היא בטוחת-טיפוס, נשאלת השאלה: האם בטיחות מעוגנת טיפוסים היא תכונה סטטית של השפות האלה?

כלומר, האם השפות הללו ניתנות להטפסה הדוקה (strong typing) ?

מסתבר ששפת "ויהי" אכן ניתנת להטפסה הדוקה (strong typing) . אולם זו אינה תכונה מעניינת במיוחד, כי שפת "ויהי" מוגבלת למדי. למשל, אי אפשר לכתוב בשפת "ויהי" תכנית עם לולאה אינסופית.

האם שפת "וישגור" ניתנת אף היא להטפסה הדוקה (strong typing) ? כלומר, האם יש אלגוריתם שבהינתן תכנית כלשהי בשפת "וישגור" מחשב אם התכנית בטוחה

Revision : 2.16.07

מבחינת טיפוסים? אלגוריתם כזה צריך להיות מסוגל, למשל, לקבוע אם התכנית הבאה בטוחה מבחינת טיפוסים, ולשם כך יהיה עליו להכריע האם $\langle \text{expression} \rangle$ עוצר:

```
if <expression>
then (0,0)
else (0,0)
```

כמובן, היות שבעיית העצירה אינה כריעה, אין אלגוריתם כזה. משמע, שפת "וישגור" אינה ניתנת להטפסה הדוקה. היות ששפת "וייסוג" היא הרחבה של שפת "וישגור", גם היא לא ניתנת להטפסה הדוקה. יתרה מזו, כל שפת תכנות שהיא טיורינג-שלמה אינה ניתנת להטפסה הדוקה. לא לתשובה זו פיללנו...

אולם אפשר לתאם ציפיות על-ידי כך שנשנה מעט את ההגדרות. והדרך לעשות זאת היא זו:

1. להגדיר מערכת טיפוסים סטטית.

2. להגדיר תכניות מוטפסות-היטב (well-typed).

3. להראות שכל תכנית מוטפסת-היטב היא בטוחת-טיפוס.

4. להראות שניתן לקבוע מוטפסות-היטב באופן סטטי.

נגדיר **הטפסה הדוקה** (strong typing) להיות תכונה שמתקיימת בשפה, אם:

1. מוטפסת-היטב הוא תכונה סטטית.

2. מוטפסת-היטב גורר בטיחות-טיפוס.

7.2 קשירת טיפוס לביטוי

👉 קראו את סעיף §7.1 בספר הלימוד. 👈

נגדיר מערכת טיפוסים לשפות "וישגור".

הטיפוס של מספר הוא `int`:

$$(\text{type-of } (\text{const-exp } num) \text{ tenv}) = \text{int}$$

לכל משתנה כרוך יש טיפוס בסביבת הטיפוסים:

$$(\text{type-of } (\text{var-exp } var) \text{ tenv}) = \text{tenv}(var)$$

הטיפוס של `diff-exp` הוא `int` בתנאי שהאופרנדים הם מטיפוס `int`:

$$\frac{\begin{array}{l} (\text{type-of } exp_1 \text{ tenv}) = \text{int} \\ (\text{type-of } exp_2 \text{ tenv}) = \text{int} \end{array}}{(\text{type-of } (\text{diff-exp } exp_1 exp_2) \text{ tenv}) = \text{int}}$$

הטיפוס של `zero?-exp` הוא `bool` בתנאי שהאופרנד הוא מטיפוס `int`:

$$\frac{(\text{type-of } exp_1 \text{ tenv}) = \text{int}}{(\text{type-of } (\text{zero?-exp } exp_1) \text{ tenv}) = \text{bool}}$$

Revision : 2.16.07

הטיפוס של if-exp הוא t בתנאי שהאופרנד הראשון הוא מטיפוס bool והטיפוס של האופרנד השני והשלישי הוא t :

$$\frac{\begin{array}{l} (\text{type-of } exp_1 \text{ } tenv) = \text{bool} \\ (\text{type-of } exp_2 \text{ } tenv) = t \\ (\text{type-of } exp_3 \text{ } tenv) = t \end{array}}{(\text{type-of } (\text{if-exp } exp_1 \text{ } exp_2 \text{ } exp_3) \text{ } tenv) = t}$$

הטיפוס של let-exp הוא t בתנאי ש- $body$ הוא מטיפוס t בסביבה הטיפוסים המורחבת עם כריכת var_1 לטיפוס t_1 של exp_1 :

$$\frac{\begin{array}{l} (\text{type-of } exp_1 \text{ } tenv) = t_1 \\ (\text{type-of } body [var_1 : t_1] \text{ } tenv) = t \end{array}}{(\text{type-of } (\text{let-exp } var_1 \text{ } exp_1 \text{ } body) \text{ } tenv) = t}$$

הטיפוס של proc-exp הוא $(t_1 \rightarrow t_2)$, כלומר, התאמה מקבוצת הערכים מהטיפוס של var_1 לקבוצת הערכים מהטיפוס של $body$ בסביבת הטיפוסים המורחבת:

$$\frac{(\text{type-of } body [var_1 : t_1] \text{ } tenv) = t_2}{(\text{type-of } (\text{proc-exp } var_1 \text{ } body) \text{ } tenv) = (t_1 \rightarrow t_2)}$$

הטיפוס של call-exp הוא t_2 בתנאי שהטיפוס של exp_1 הוא $(t_1 \rightarrow t_2)$ והטיפוס של exp_2 הוא t_1 :

$$\frac{\begin{array}{l} (\text{type-of } exp_1 \text{ tenv}) = (t_1 \rightarrow t_2) \\ (\text{type-of } exp_2 \text{ tenv}) = t_1 \end{array}}{(\text{type-of } (\text{call-exp } exp_1 \text{ } exp_2) \text{ tenv}) = t_2}$$

7.3 תכניות מוטפסות-היטב בשפת "וישגור"

השלב הבא הוא להגדיר מה הן תכניות מוטפסות-היטב.

🔗 תכנית בשפת "וישגור"

(a-program *exp*)

היא מוטפסת-היטב אם ורק אם קיים טיפוס t כך שעל סמך חוקי הטיפסה אפשר להוכיח כי

$(\text{type-of } exp \text{ tenv}_0) = t$

כאשר

$tenv_0 = [i : int, v : int, x : int]$

היא סביבה תחילית של טיפוסים, המפרטת את הטיפוס של כל משתנה שכרוך בסביבה התחילית הרגילה. 🔗

★תרגיל 7.1 תנו דוגמה לתכנית בשפת "וישגור" שאינה מוטפסת-היטב.

★תרגיל 7.2 תנו דוגמה של תכנית בטוחת-טיפוס בשפת "וישגור" שאינה מוטפסת-היטב.

Revision : 2.16.07

*תרגיל 7.3 תנו דוגמה של תכנית בטוחת-טיפוס שאינה מסתיימת.

*תרגיל 7.4 תנו דוגמה לתכנית מוטפסת-היטב בשפת וישגור שאינה מסתיימת.

7.4 נאותות טיפוסים

כעת יש להוכיח נאותות.

משפט 1. בשפת "וישגור": אם P היא תוכנית מוטפסת-היטב, אז P היא בטוחת-טיפוס.

עליכם להוכיח (ההוכחה מושארת לקורא).

רמז: באינדוקציה על מספר הקריאות ל- value-of בזמן שערך P .

השלב הבא הוא להוכיח שמוטפסת-היטב היא תכונה שניתנת להסקה סטטית. נדגים

זאת באמצעות אלגוריתם שמכריע האם תכנית בשפת "ויהי" היא מוטפסת-היטב.

7.4.1 אלגוריתם עבור שפת "ויהי"

בהינתן תכנית $(a\text{-program } exp)$ בשפת "ויהי", בדוק האם הביטוי exp הוא מוטפס-

היטב ביחס לסביבה התחילית $tenv$.

• אם הביטוי exp הוא קבוע מספרי מהצורה $(\text{const-exp } n)$,

- אז הביטוי exp מוטפסת-היטב ביחס לסביבת הטיפוסים $tenv$.

• אם הביטוי exp הוא משתנה מהצורה $(\text{var-exp } x)$,

- אז הביטוי exp מוטפסת-היטב ביחס לסביבת הטיפוסים $tenv$ אם ורק אם x

כרוך בסביבת הטיפוסים $tenv$.

• אם הביטוי exp הוא מהצורה $(\text{diff-exp } exp_1 \ exp_2)$,

- אז הביטוי exp מוטפס־היטב ביחס לסביבת הטיפוסים $tenv$ אם ורק אם גם exp_1 וגם exp_2 הם מוטפס־היטב ביחס לסביבת הטיפוסים $tenv$ וטיפוסם הוא int .

• אם הביטוי exp הוא מהצורה $(zero?-exp\ exp_1)$,

- אז הביטוי exp הוא מוטפס־היטב ביחס לסביבת הטיפוסים $tenv$ אם ורק אם exp_1 מוטפס־היטב ביחס לסביבת הטיפוסים $tenv$ וטיפוסו הוא int .

• אם הביטוי exp הוא מהצורה $(if-exp\ exp_1\ exp_2\ exp_3)$,

- אז הביטוי exp הוא מוטפס־היטב ביחס לסביבת הטיפוסים $tenv$ אם ורק אם exp_1, exp_2, exp_3 ו־ exp_3 מוטפס־היטב בסביבת הטיפוסים $tenv$, הטיפוס של exp_1 הוא $bool$, ול־ exp_2 ול־ exp_3 אותו טיפוס.

• אם הביטוי exp הוא מהצורה $(let-exp\ var\ exp_1\ exp_2\ body)$,

- אז הביטוי exp מוטפס־היטב ביחס לסביבת הטיפוסים $tenv$ אם ורק אם exp_1 מוטפס־היטב בסביבת הטיפוסים $tenv$, ו־ $body$ מוטפס־היטב בסביבת הטיפוסים המורחבת $[var_1 : t_1]tenv$, כאשר t_1 הוא הטיפוס של exp_1 .

סוף האלגוריתם.

✦ האלגוריתם הוא בסך הכול הפעלה פשוטה של חוקי הטיפוס על ביטויים בשפה "ויה". ✦

7.4.2 אלגוריתם עבור שפת "וישגור"

אם ננסה להרחיב את האלגוריתם לטיפול בביטויים בשפת "וישגור" ניתקל בבעיה:

$$\frac{(\text{type-of } body [var_1 : t_1] tenv) = t_2}{(\text{type-of } (\text{proc-exp } var_1 body) tenv) = (t_1 \rightarrow t_2)}$$

עד כה, לכל טיפוס מ"על הקו" היה או טיפוס ידוע כגון *int*, או טיפוס של ביטוי מסוים. אבל בביטויים של שפת "וישגור", כיצד נדע מהו הטיפוס של המשתנה הכרוך? יש שתי תשובות סטנדרטיות לכך:

1. או לחייב את המתכנת לציין את הטיפוס (והאלגוריתם יבדוק את המתכנת).
 2. או לחייב את האלגוריתם לבדיקת הטיפוסים להסיק בעצמו את הטיפוס של המשתנה הכרוך (והמתכנת יבדוק את האלגוריתם).
- לכל פתרון יתרונות וחסרונות:

1. אם מחייבים את המתכנת לומר לנו מה הטיפוס של המשתנה הכרוך, אז קל יותר לממש את השפה, קל יותר להבין תכניות, ואפשר לטפל ביתר קלות בטיפוסים מורכבים יותר כי המתכנת עוזר לנו.
 2. אם מחייבים את האלגוריתם לבדיקת טיפוסים להסיק אוטומטית את הטיפוס של המשתנה הקשור, אז קל יותר לכתוב תכניות כי יש פחות לכתוב, התכניות כלליות יותר, ולכן יש יותר הזדמנויות לשימוש חוזר בקוד.
- מבחינה היסטורית, רוב השפות עוצבו על-ידי האנשים שנוקקו להן לשימושם הם, ורובן מעמיסות את ציון הטיפוסים על המשתמש במקום על הממש.

7.5 "ויבדוק": שפה עם טיפוסים מוצהרים

קראו את סעיף §7.3 בספר הלימוד.  

בכדי לאפשר למתכנת לומר לנו מהו הטיפוס, נצטרך לשנות מעט את התחביר של השפה.

7.5.1 תחביר השפה "ויבדוק"

השינויים לתחביר של שפת "ויבדוק" מסוכמים בטבלה 7.1. התחביר המוחשי מופיע בעמודה השמאלית, ובעמודה הימנית מתואר הייצוג המופשט. שני החוקים האחרונים בטבלה שונו.

תחביר מופשט	תחביר מוחשי
a-program (exp1)	Program ::= Expression
const-exp (num)	Expression ::= Number
diff-exp (exp1 exp2)	::= -(Expression , Expression)
zero?-exp (exp1)	::= zero? (Expression)
if-exp (exp1 exp2 exp3)	::= if Expression then Expression else Expression
var-exp (var)	::= Identifier
let-exp (var exp1 body)	::= let Identifier = Expression in Expression
proc-exp (var ty body)	::= proc (Identifier : <i>Type</i>) Expression
cal-exp (exp1 exp2)	::= (Expression Expression)
letrec-exp(p-result-type p-name b-var b-var-type p-body letrec-body)	::= letrec Identifier (Identifier : <i>Type</i>)= Expression in Expression

טבלה 7.1: תחביר שפת "ויבדוק"

לדוגמה, ניתן לרשום בשפת "ויבדוק" תכניות כאלה:

```
proc (x : int) -(x,1)
```

```
letrec
  int double (x : int) = if zero?(x)
```

Revision : 2.16.07

```

      then 0
      else -((double -(x,1)), -2)
in double

```

```

proc (f : (bool->int)) proc (n : int) (f zero?(n))

```

◆ ע'מלל כ' double מחזירה ערך מט'פול int, אהל הט'פול el
 double ה'ל (int->int).

קראו את סעיף §7.3.1 בספר הלימוד.

7.6 "ויקיש": שפה שמקיישה טיפוסים

קראו את סעיף §7.4 בספר הלימוד.

נראה עתה כיצד ניתן להקיש על סמך ידיעת הטיפוסים של חלקים בתכנית מה הם הטיפוסים של שאר החלקים בתכניות של השפות "ויהי", "וישגור", ו-"וייסוג".
 כבר תיארנו אלגוריתם שמחליט האם תכנית בשפת "ויהי" היא מוטפסת-היטב. אלגוריתם זה מבוסס על הסקת טיפוסים לכל תת-ביטוי שמופיע בתכנית.
 על מנת להרחיב את האלגוריתם לטיפול בשפות "וישגור" ו-"וייסוג", צריך ראשית למצוא שיטה לניחוש הטיפוס של משתנים כרוכים. יש שתי דרכים מקובלות:

1. לנסות ולבדוק את כל הניחושים האפשריים במקביל.
2. לדחות ניחוש הטיפוס על-ידי הוספת משתנה חדש, תוך צבירת אוסף משוואות ואילוצים אחרים שהמשתנים צריכים לספק. לבסוף, לפתור את מערכת המשוואות והאילוצים למציאת ערך לטיפוס.

הדרך הראשונה מובילה לאלגוריתם אקספוננציאלי שאינו מעשי. לכן, הדרך השנייה מקובלת יותר בהקשת טיפוסים ובניתוחים סטטיים אחרים.

7.6.1 אלגוריתם להקשת טיפוסים עבור תכניות בשפת "וייסוג"

אלגוריתם: בהינתן תכנית בשפת "וייסוג", תרגם את התכנית לתכנית שקולה על-ידי שינוי שמות כל המשתנים הקשורים, כך שלאחר השכתוב, אף משתנה לא יהיה קשור ביותר ממקום אחד. הגדר טיפוס חדש t_x עבור כל משתנה x שקשור בביטוי exp . הגדר טיפוס חדש t_e עבור כל תת-ביטוי e שמופיע בתוך exp . בעזרת האלגוריתם הניתן בסעיף 7.6.2 להלן, ייצר את משוואות האילוצים שהטיפוסים t_x ו- t_e צרכים לספק, ופתור את מערכת המשוואות. אם למערכת המשוואות יש פתרון, אז התכנית מוטפסת-היטב. אם מערכת המשוואות אינה פתירה, אז התכנית אינה מוטפסת-היטב.

7.6.2 אלגוריתם לחילול אילוצי טיפוס עבור שפת "וייסוג"

אלגוריתם: בהינתן ביטוי exp בשפת "וייסוג", החזר את אוסף האילוצים על הטיפוסים.

• אם הביטוי exp הוא קבוע מספרי מהצורה $(const-exp\ n)$, אז האילוצים הם:

$$t_{exp} = int -$$

• אם הביטוי exp הוא משתנה מהצורה $(var-exp\ x)$, אז האילוצים הם:

$$t_{exp} = t_x -$$

• אם הביטוי exp הוא מהצורה $(diff-exp\ e_1 e_2)$, אז האילוצים הם:

$$t_{e_1} = int -$$

$$t_{e_2} = int -$$

$$t_{exp} = int -$$

- אם הביטוי exp הוא מהצורה $(zero?-exp \ exp_1)$, אז האילוצים הם:

$$t_{e_1} = int -$$

$$t_{exp} = bool -$$

- אם הביטוי exp הוא מהצורה $(if-exp \ e_1 e_2 e_3)$, אז האילוצים הם:

$$t_{e_1} = bool -$$

$$t_{e_2} = t_{exp} -$$

$$t_{e_3} = t_{exp} -$$

- אם הביטוי exp הוא מהצורה $(let-exp \ x_1 e_1 body)$, אז האילוצים הם:

$$t_{x_1} = t_{e_1} -$$

$$t_{body} = t_{exp} -$$

- אם הביטוי exp הוא מהצורה $(proc-exp \ x_1 body)$, אז האילוצים הם:

$$t_{exp} = (t_{x_1} \rightarrow t_{body}) -$$

- אם הביטוי exp הוא מהצורה $(call-exp \ e_1 e_2)$, אז האילוצים הם:

$$t_{e_1} = (t_{e_2} \rightarrow t_{exp}) -$$

- אם הביטוי exp הוא מהצורה $(letrec-exp \ x_0 x_1 e_1 body)$, אז האילוצים הם:

$$t_{x_0} = (t_{x_1} \rightarrow t_{e_1}) -$$

$$t_{exp} = t_{body} -$$

סוף האלגוריתם.

7.6.3 דוגמה

נצטרך כמובן אלגוריתם כדי לפתור את מערכת האילוצים. אך קודם לכך, נראה דוגמה להקשת טיפוסים עבור תכנית זו:

```
let a = proc (i) proc (j) -(i, -(0,j))
in letrec m (x) = proc (y) if zero?(x)
                        then 0
                        else ((a ((m -(x,1) y)) y)
in ((m 11) 12)
```

הנה משתני הטיפוס שנשייך לכל משתנה קשור (טבלה 7.2).

t_a	a
t_m	m
t_i	i
t_j	j
t_x	x
t_y	y

טבלה 7.2: משתנה טיפוס לכל משתנה קשור

משתני הטיפוס שנשייך לכל תת־ביטוי בתכנית מובאים בטבלה 7.3. מערכת האילוצים שנגזרת מהביטויים מוצגת בטבלאות 7.4 ו־7.5. פתרון מערכת האילוצים שבטבלאות 7.4 ו־7.5 יוכיח כי התכנית מוטפסת־היטב.

7.6.4 פתרון מערכת האילוצים

נסדר ונמייין את האילוצים (טבלאות 7.4 ו־7.5) לרשימה כמו שמופיע בטבלאות 7.6 ו־7.7. נבטל אילוצים מיותרים ונציב טיפוסים ידועים. נקבל את מערכת האילוצים שבטבלאות 7.8 ו־7.9. נחזור על התהליך ונקבל את מערכת האילוצים שבטבלאות 7.10

Revision : 2.16.07

```

t1      11
t2      12
t3      m
t4      (m 11)
t5      ((m 11) 12)
t6      x
t7      zero?(x)
t8      0
t9      x
t10     1
t11     -(x,1)
t12     m
t13     (m -(x,1))
t14     y
t15     ((m -(x,1)) y)
t16     a
t17     (a ((m -(x,1)) y))
t18     y
t19     ((a ((m -(x,1)) y)) y)
t20     if zero?(x) then 0 else ((a ((m -(x,1)) y)) y)
t21     proc (y) if zero?(x) then ... else ...
t22     letrec m (x) = proc (y) ... in ((m 11) 12)
t23     0
t24     j
t25     -(0,j)
t26     i
t27     -(i,-(0,j))
t28     proc (j) -(1, -(0,j))
t29     proc (i) proc (j) -(i, -(0,j))
t30     let a = ... in letrec m (x) = ... in ...

```

טבלה 7.3: משתנה טיפוס לכל תתי-ביטוי

$$\begin{aligned}
t_1 &= int \\
t_2 &= int \\
t_3 &= t_m \\
t_3 &= (t_1 \rightarrow t_4) \\
t_4 &= (t_2 \rightarrow t_5) \\
t_6 &= t_x \\
t_6 &= int \\
t_7 &= bool \\
t_8 &= int \\
t_9 &= t_x \\
t_{10} &= int \\
t_9 &= int \\
t_{10} &= int \\
t_{11} &= int \\
t_{12} &= t_m \\
t_{12} &= (t_{11} \rightarrow t_{13}) \\
t_{14} &= t_y \\
t_{13} &= (t_{14} \rightarrow t_{15}) \\
t_{16} &= t_a \\
t_{16} &= (t_{15} \rightarrow t_{17}) \\
t_{18} &= t_y \\
t_{17} &= (t_{18} \rightarrow t_{19}) \\
t_7 &= bool \\
t_8 &= t_{20} \\
t_{19} &= t_{20}
\end{aligned}$$

טבלה 7.4: מערכת אילוצים

Revision : 2.16.07

$$t_{21} = (t_y \rightarrow t_{20})$$

$$t_m = (t_x \rightarrow t_{21})$$

$$t_{22} = t_5$$

$$t_{23} = int$$

$$t_{24} = t_j$$

$$t_{23} = int$$

$$t_{24} = int$$

$$t_{25} = int$$

$$t_{26} = t_i$$

$$t_{25} = int$$

$$t_{26} = int$$

$$t_{27} = int$$

$$t_{28} = (t_j \rightarrow t_{27})$$

$$t_{29} = (t_i \rightarrow t_{28})$$

$$t_a = t_{29}$$

$$t_{30} = t_{22}$$

טבלה 7.5: מערכת אילוצים (המשך)

$$\begin{aligned}
t_a &= t_{16} \\
t_a &= t_{29} \\
t_i &= t_{26} \\
t_j &= t_{24} \\
t_m &= t_3 \\
t_m &= t_{12} \\
t_m &= (t_x \rightarrow t_{21}) \\
t_x &= t_6 \\
t_x &= t_9 \\
t_y &= t_{14} \\
t_y &= t_{18} \\
t_1 &= int \\
t_2 &= int \\
t_3 &= tm \\
t_3 &= (t_1 \rightarrow t_4) \\
t_4 &= (t_2 \rightarrow t_5) \\
t_5 &= t_{22} \\
t_6 &= t_x \\
t_6 &= int \\
t_7 &= bool \\
t_7 &= bool \\
t_8 &= int \\
t_8 &= t_{20} \\
t_9 &= tx \\
t_9 &= int \\
t_{10} &= int \\
t_{10} &= int \\
t_{11} &= int \\
t_{12} &= t_m \\
t_{12} &= (t_{11} \rightarrow t_{13}) \\
t_{13} &= (t_{14} \rightarrow t_{15}) \\
t_{14} &= t_y
\end{aligned}$$

טבלה 7.6: מערכת אילוצים ממוינת

$$\begin{aligned}
t_{16} &= t_a \\
t_{16} &= (t_{15} \rightarrow t_{17}) \\
t_{17} &= (t_{18} \rightarrow t_{19}) \\
t_{18} &= t_y \\
t_{19} &= t_{20} \\
t_{20} &= t_{19} \\
t_{20} = t_8 t_{21} &= (t_y \rightarrow t_{20}) \\
t_{22} &= t_5 \\
t_{22} &= t_{30} \\
t_{23} &= int \\
t_{23} &= int \\
t_{24} &= t_j \\
t_{24} &= int \\
t_{25} &= int \\
t_{25} &= int \\
t_{26} &= t_i \\
t_{26} &= int \\
t_{27} &= int \\
t_{28} &= (t_j \rightarrow t_{27}) \\
t_{29} &= (t_i \rightarrow t_{28}) \\
t_{30} &= t_{22}
\end{aligned}$$

טבלה 7.7: מערכת אילוצים ממויינת (המשך)

$$\begin{aligned}
t_a &= t_{16} \\
t_a &= t_{29} \\
t_i &= int \\
t_j &= int \\
t_m &= t_3 \\
t_m &= t_{12} \\
t_m &= (t_x \rightarrow t_{21}) \\
t_x &= int \\
t_y &= t_{14} \\
t_y &= t_{18} \\
t_1 &= int \\
t_2 &= int \\
t_3 &= t_m \\
t_3 &= (int \rightarrow t_4) \\
t_4 &= (int \rightarrow t_5) \\
t_5 &= t_{22} \\
t_6 &= t_x \\
t_6 &= int \\
t_7 &= bool \\
t_8 &= int \\
t_9 &= t_x \\
t_9 &= int \\
t_{10} &= int \\
t_{11} &= int \\
t_{12} &= t_m \\
t_{12} &= (int \rightarrow t_{13}) \\
t_{13} &= (t_{14} \rightarrow t_{15}) \\
t_{14} &= t_y
\end{aligned}$$

טבלה 7.8: מערכת אילוצים מצומצמת

Revision : 2.16.07

$$\begin{aligned}
t_{16} &= t_a \\
t_{16} &= (t_{15} \rightarrow t_{17}) \\
t_{17} &= (t_{18} \rightarrow t_{19}) \\
t_{18} &= t_y \\
t_{19} &= int \\
t_{20} &= int \\
t_{21} &= (t_y \rightarrow int) \\
t_{22} &= t_5 \\
t_{22} &= t_{30} \\
t_{23} &= int \\
t_{24} &= t_j \\
t_{24} &= int \\
t_{25} &= int \\
t_{26} &= t_i \\
t_{26} &= int \\
t_{27} &= int \\
t_{28} &= (t_j \rightarrow int) \\
t_{29} &= (t_i \rightarrow t_{28}) \\
t_{30} &= t_{22}
\end{aligned}$$

טבלה 7.9: מערכת אילוצים מצומצמת (המשך)

7.11-7.13. נחזור שוב על התהליך ונקבל את מערכת האילוצים שבטבלאות 7.12 ו-7.13.

עד כאן השלב הקל. עכשיו מתחיל להיות מעניין יותר. נתמקד באילוצים שבטבלה 7.14. האחדה שלהם תיתן לנו את האילוצים שבטבלה 7.16. בפישוט נוסף נקבל את טבלה 7.16. וכן הלאה. לבסוף נגיע למערכת האילוצים שמוצגות בטבלאות 7.17 ו-7.18.

סוף סוף, התכנית מוטפסת־היטב!

Revision : 2.16.07

$$\begin{aligned}
 t_a &= t_{16} \\
 t_a &= t_{29} \\
 t_i &= int \\
 t_j &= int \\
 t_m &= t_3 \\
 t_m &= t_{12} \\
 t_m &= (t_x \rightarrow t_{21}) \\
 t_x &= int \\
 t_y &= t_{14} \\
 t_y &= t_{18} \\
 t_1 &= int \\
 t_2 &= int \\
 t_3 &= t_m \\
 t_3 &= (int \rightarrow t_4) \\
 t_4 &= (int \rightarrow t_5) \\
 t_5 &= t_{22} \\
 t_6 &= int \\
 t_7 &= bool \\
 t_8 &= int \\
 t_9 &= int \\
 t_{10} &= int \\
 t_{11} &= int \\
 t_{12} &= t_m \\
 t_{12} &= (int \rightarrow t_{13}) \\
 t_{13} &= (t_{14} \rightarrow t_{15}) \\
 t_{14} &= t_y
 \end{aligned}$$

טבלה 7.10: צמצום נוסף של מערכת האילוצים

$$\begin{aligned}
t_{16} &= t_a \\
t_{16} &= (t_{15} \rightarrow t_{17}) \\
t_{17} &= (t_{18} \rightarrow int) \\
t_{18} &= t_y \\
t_{19} &= int \\
t_{20} &= int \\
t_{21} &= (t_y \rightarrow int) \\
t_{22} &= t_5 \\
t_{22} &= t_{30} \\
t_{23} &= int \\
t_{24} &= int \\
t_{25} &= int \\
t_{26} &= int \\
t_{27} &= int \\
t_{28} &= (int \rightarrow int) \\
t_{29} &= (t_i \rightarrow t_{28}) \\
t_{30} &= t_{22}
\end{aligned}$$

טבלה 7.11: צמצום נוסף של מערכת האילוצים (המשך)

Revision : 2.16.07

$$\begin{aligned}
t_a &= t_{16} \\
*t_a &= (int \rightarrow (int \rightarrow int)) \\
t_i &= int \\
t_j &= int \\
t_m &= t_3 \\
t_m &= t_{12} \\
*t_m &= (int \rightarrow t_{21}) \\
t_x &= int \\
t_y &= t_{14} \\
t_y &= t_{18} \\
t_1 &= int \\
t_2 &= int \\
t_3 &= t_m \\
t_3 &= (int \rightarrow t_4) \\
t_4 &= (int \rightarrow t_5) \\
t_5 &= t_{22} \\
t_6 &= int \\
t_7 &= bool \\
t_8 &= int \\
t_9 &= int \\
t_{10} &= int \\
t_{11} &= int \\
t_{12} &= t_m \\
t_{12} &= (int \rightarrow t_{13}) \\
t_{13} &= (t_{14} \rightarrow t_{15}) \\
t_{14} &= t_y
\end{aligned}$$

טבלה 7.12: מערכת אילוצים מעודכנת

$$*t_{16} = (int \rightarrow (int \rightarrow int))$$

$$t_{16} = (t_{15} \rightarrow t_{17})$$

$$t_{17} = (t_{18} \rightarrow int)$$

$$t_{18} = t_y$$

$$t_{19} = int$$

$$t_{20} = int$$

$$t_{21} = (t_y \rightarrow int)$$

$$t_{22} = t_5$$

$$t_{22} = t_{30}$$

$$t_{23} = int$$

$$t_{24} = int$$

$$t_{25} = int$$

$$t_{26} = int$$

$$t_{27} = int$$

$$t_{28} = (int \rightarrow int)$$

$$*t_{29} = (int \rightarrow (int \rightarrow int))$$

$$t_{30} = t_{22}$$

טבלה 7.13: מערכת אילוצים מעודכנת (המשך)

Revision : 2.16.07

$$\begin{aligned}
 t_y &= t_{14} \\
 t_y &= t_{18} \\
 t_{15} &= int \\
 t_{16} &= (int \rightarrow (int \rightarrow int)) \\
 t_{16} &= (t_{15} \rightarrow t_{17}) \\
 t_{17} &= (t_{18} \rightarrow int) \\
 t_{18} &= int \\
 t_{18} &= t_y
 \end{aligned}$$

טבלה 7.14: מערכת אילוצים לפני האחדה

$$\begin{aligned}
 t_y &= t_{14} \\
 t_y &= t_{18} \\
 t_{16} &= (int \rightarrow (int \rightarrow int)) \\
 t_{16} &= (t_{15} \rightarrow t_{17}) \\
 t_{17} &= (t_{18} \rightarrow int) \\
 t_{18} &= t_y
 \end{aligned}$$

טבלה 7.15: מערכת האילוצים לאחר האחדה

$$\begin{aligned}
 t_y &= int \\
 t_{13} &= (int \rightarrow t_{15}) \\
 t_{14} &= int \\
 t_{16} &= (t_{15} \rightarrow (int \rightarrow int)) \\
 t_{17} &= (int \rightarrow int) \\
 t_{18} &= int
 \end{aligned}$$

טבלה 7.16: מערכת האילוצים לאחר פישוט

$$\begin{aligned}
 t_a &= (int \rightarrow (int \rightarrow int)) \\
 t_i &= int \\
 t_j &= int \\
 t_m &= (int \rightarrow (int \rightarrow int)) \\
 t_x &= int \\
 t_y &= int
 \end{aligned}$$

טבלה 7.17: מערכת האילוצים הסופית

Revision : 2.16.07

$$\begin{aligned}
t_1 &= int \\
t_2 &= int \\
t_3 &= (int \rightarrow (int \rightarrow int)) \\
t_4 &= (int \rightarrow int) \\
t_5 &= int \\
t_6 &= int \\
t_7 &= bool \\
t_8 &= int \\
t_9 &= int \\
t_{10} &= int \\
t_{11} &= int \\
t_{12} &= (int \rightarrow (int \rightarrow int)) \\
t_{13} &= (int \rightarrow int) \\
t_{14} &= int \\
t_{15} &= int \\
t_{16} &= (int \rightarrow (int \rightarrow int)) \\
t_{17} &= (int \rightarrow int) \\
t_{18} &= int \\
t_{19} &= int \\
t_{20} &= int \\
t_{21} &= (int \rightarrow int) \\
t_{22} &= int \\
t_{22} &= int \\
t_{23} &= int \\
t_{24} &= int \\
t_{25} &= int \\
t_{26} &= int \\
t_{27} &= int \\
t_{28} &= (int \rightarrow int) \\
t_{29} &= (int \rightarrow (int \rightarrow int)) \\
t_{30} &= int
\end{aligned}$$

טבלה 7.18: מערכת האילוצים הסופית (המשך)

7.7 תרגילים נוספים



אלמות שהפיסו בחינות

★תרגיל 7.5 (שאלה מבחינה) נתונה תכנית בשפת "ויקיש" (INFERRED):

```
proc (f:?)
  proc (g:?)
    proc (p:?)
      proc (x:?)
        if (p (f x)) then (g 1) else -((f x),1)
```

ברצוננו לקבוע מהו טיפוס התכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

1. הקצו לביטוי הנתון בשאלה ולתתי-הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי-הביטויים שלו.

2. פתרו את המשוואות שהרכבתם בסעיף 1 תוך שימוש ב־substitution ו־unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התכנית הנתונה.

★תרגיל 7.6 (שאלה מבחינה) נתונה תכנית בשפת "ויקיש" (INFERRED):

```
proc (w)
let
  x=10
```

Revision : 2.16.07

```
in
  if zero? (-(x,3)) then (w x) else (w 8)
```

ברצוננו לקבוע מהו טיפוס התכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

1. הקצו לביטוי הנתון בשאלה ולתתי-הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי-הביטויים שלו.

2. פתרו את המשוואות שהרכבתם בסעיף 1 תוך שימוש ב־ substitution ו־ unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התכנית הנתונה.

★תרגיל 7.7 (שאלה מבחינה) נתונה תכנית בשפת "ויקיש" (INFERRED):

```
proc (f:?)
proc (g:?)
if (g (f g)) then (f g) else (g 3)
```

ברצוננו לקבוע מהו טיפוס התכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

1. הקצו לביטוי הנתון בשאלה ולתתי-הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי-הביטויים שלו.

2. פתרו את המשוואות שהרכבתם בסעיף 1 תוך שימוש ב־ substitution ו־ unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון

המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התכנית הנתונה.

★**תרגיל 7.8** (שאלה מבחינה) נתונה תכנית בשפת "ויקיש" (INFERRED):

```
letrec
  ? f(x:?) = (f -(x,-1))
in (f 1)
```

ברצוננו לקבוע מהו טיפוס התכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

1. הקצו לביטוי הנתון בשאלה ולתתי-הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי-הביטויים שלו.

2. פתרו את המשוואות שהרכבתם בסעיף 1 תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התכנית הנתונה.

★**תרגיל 7.9** (שאלה מבחינה) נתונה תכנית בשפת "ויקיש" (INFERRED):

```
(proc (x:?)
  let
    x= zero? ((x 9))
  in
    if x then 5 else 7
  proc (y:?) -(y,7))
```

ברצוננו לקבוע מהו טיפוס התכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

Revision : 2.16.07

1. הקצו לביטוי הנתון בשאלה ולתת-הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתת-הביטויים שלו.

2. פתרו את המשוואות שהרכבתם בסעיף 1 תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התכנית הנתונה.

פרק 8

פרדיגמות תכנות

שפות שונות מציגות למתכנת צורות חשיבה שונות על בעיות תכנות. ניתן למפות את השפות לקבוצות בעלות מאפיינים משותפים, הדומות בסגנון ובצורת החשיבה המתאימה להן. צורות חשיבה כאלה נקראות **פרדיגמות**. בפרק זה נסקור בקצרה כמה פרדיגמות. אין הכוונה להציג תיאור מלא של כל פרדיגמה, אלא לתת סקירה כללית של סגנונות התכנות המרכזיים וסקירה מעמיקה יותר של הסגנון הפונקציונלי.

8.1 תכנות אימפרטיבי

תכניות בשפות אימפרטיביות מורכבות מסדרה של פעולות או מסדרה של פקודות. שפות אימפרטיביות בולטות הן פסקל, C, פורטרן וכד'.



שפת C נלמדת בקורס 20465 – מעבדה בתכנות מערכות.

Revision : 2.16.21

8.2 תכנות פונקציונלי

תכנות פונקציונלי מבוסס על הפעלה של פונקציות. הכוונה לפונקציות הדומות לפונקציות מתמטיות: תוצאת הפעלתן תלויה אך ורק בפרמטרים שלהן, ולא בשום גורם אחר. שתי הפעלות של פונקציה עם אותם פרמטרים יחזירו תמיד את אותו הערך. תכונה זו נקראת referential transparency, ומושגת בשפות פונקציונליות טהורות (כלומר כאלה שאינן מאפשרות תכנות אימפרטיבי) הודות לכך שאינן מאפשרות השמה למשתנים. שינוי ערך של משתנה, או שינוי אחר במצב המערכת, נקרא side effect.

תכונת ה־referential transparency חשובה, שכן היא מקלה את הבנת מערכות תוכנה. על־מנת להבין את המשמעות של הפעלת פונקציה, אין צורך לדעת מתי בדיוק היא מופעלת ובאילו תנאים. מסיבה זו גם קל יותר לנתח התנהגות של תכניות באופן מתמטי, ואף להוכיח את נכונותן. היות שאין בהן פעולות השמה, מבנה הבקרה המרכזי בשימוש בשפות פונקציונליות הוא רקורסיה.

הוויכוח על יתרונות התכנות הפונקציונלי לעומת התכנות האימפרטיבי נמשך זה זמן רב. במהלך הקורס נראה דוגמאות רבות לתכנות פונקציונלי, שכן צורת תכנות זו אופיינית לשפת Scheme, וכן היות שסגנון זה מתאים מאוד לאופי התכניות שנכתוב. עם זאת, לא נימנע ממימוש שפות בסגנון אימפרטיבי.

שפות התכנות הפונקציונליות הנפוצות כיום תומכות, בנוסף לתכונות לעיל, גם בתכונות משמעותיות נוספות, ובהן מערכות טיפוסים עשירות וניהול זיכרון אוטומטי (garbage collection). אמצעים אלה, עשויים לתרום להגדלת התפוקה של מתכנתים מיומנים. שפות פונקציונליות בולטות הן ML, Haskell, Mirnada.



שפת ML נלמדת בקורס 20504 – סדנה בעקרונות שפות תכנות.

8.3 תכנות לוגי

תכנות לוגי קשור קשר הדוק לשפת התכנות פרולוג, שפותחה למטרות עיבוד שפה טבעית בשנת 1970. המאפיינים הבולטים של סגנון התכנות הלוגי הוא הסתמכות על אפיון לוגי של הדרישות מהתוכנה, והסתמכות על מנגנוני הסקה לוגית ככלי חישובי. מקובל לכנות סגנון זה כסגנון תכנות הצהרתי (declarative), שכן הוא מבוסס על השאיפה שהמתכנת יתאר את התכונות של המערכת, ולא את הדרך לביצוע החישוב. דוגמה למערכת כזו המטפלת בקשרי משפחה:

```
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
female(pam).
male(tom).
female(liz).
male(bob).
female(ann).
offspring(Y, X) :- parent(X, Y).
mother(X, Y) :- parent(X, Y), female(X).
```

בדוגמה מדובר על קבוצת עובדות המייצגות קשרי משפחה. קבוצת המשפטים הראשונה מייצגת עובדות שהן נכונות במערכת.

שני המשפטים האחרונים מייצגים כללי היסק. הראשון קובע ש Y הוא צאצא של X, אם X הוא הורה של Y. הכלל השני קובע ש X היא אם של Y אם X היא הורה ובנוסף לכך X נקבה. לאחר מתן כללים אלה ניתן להציג שאלות למפרש של פרולוג:

```
?- female(pam).
Yes
```

Revision : 2.16.21

```
?- female(X).
X = pam ; X= liz ; X= ann;
?- mother(pam, bob).
Yes
```

כאשר מציגים למפרש ביטוי המכיל משתנים (משתנים בפרולוג מתחילים באות גדולה), מחפש המפרש את כל הערכים שיהפכו את הטענה לנכונה. לכן הביטוי `female(X)` מחזיר יותר מתשובה אחת.

★תרגיל 8.1 הסבירו את התכנית הזו:

```
jewish(Y) :- mother(X,Y), jewish(X).
female(sara).
jewish(sara).
```

שפת פרולוג נלמדת בקורס 20596 - שפת פרולוג והיבטים לבינה מלאכותית.



8.4 תכנות מונחה עצמים

תכנות מונחה עצמים מאפשר סימולציה של המערכת בעזרת עצמים. מנגנון ההפשטה העיקרי הוא המושג מחלקה (class) המתאר מאפיינים משותפים של עצמים. מנגנוני הפשטה נוספים מאפשרים לתאר מאפיינים משותפים של מחלקות, כגון ירושה ודפוס. ירושה מאפשרת למשל לעדן מחלקה לתת-מחלקות המקבלות בירושה את תכונות המחלקה.

מקורו של התכנות מונחה העצמים בשפת Simula שפותחה בשנות ה-60 של המאה העשרים לצורך תכנות סימולציות. שפה חשובה נוספת היא Smalltalk. שפה זו תוכננה כסביבת תכנות ידידותית, ואכן השפה מספקת סביבת פיתוח גרפית למשתמש, המאפשרת לבחון את המחלקות והעצמים, ואת הקשרים שביניהם. סגנון התכנות מונחה העצמים זוכה לפופולריות, ורבות השפות התומכות בו. הבולטות הן C#, C++, Java, Eiffel, ועדה.

 שפת סמולטוק נלמדת בקורס 20906 - תכנות מונחה עצמים.

שפת C# נלמדת בקורס 20586 - סדנה בתכנות מונחה עצמים.

שפת עדה נלמדה בקורס 24402 - הנדסת תוכנה: הפשטה ועיצוב מונחה



עצמים. קורס זה נגזר זה מכבר.

8.5 סיכום

הגבול בין פרדיגמות אחת למשניה אינו חד-משמעי, ושפות רבות תומכות ביותר מסגנון תכנות אחד. מקובל לומר על שפות כאלה שהן מולטי-פרדיגמטיות. ישנן גם שפות תכנות ומערכות המכילות מרכיבים שפתיים שאינם מתאימים לסיווגים שהוצגו. דוגמאות בולטות לכך הן שפות ויזואליות, שבהן המתכנת מתאר את המערכת באופן גרפי; שפות "קטנות" (little languages) המותאמות לפתרון סוג מצומצם של בעיות (למשל שפות מקרו של חבילות תוכנה); ושפות המיועדות לשימושם של משתמשי קצה ולא של מתכנתים, המדמות ככל האפשר שפה טבעית.

התפתחות שפות תכנות במקרים רבים היא אבולוציונית. שפות חדשות מאמצות תכונות של שפות שקדמו להן תוך כדי הוספת תכונות חדשות והכנסת שינויים ושיפורים. דוגמא בולטת לכך היא שפת C++ שהוסיפה תמיכה בתכנות מונחה עצמים

Revision : 2.16.21

לשפת C. שפות מעטות אומצו על-ידי מכרז (הליך של פנייה לקבלת הצעות להגדרת השפה). דוגמה בולטת היא שפת עדה.

סביב שפות התכנות השונות וסגנונות התכנות השונים צומחות קהילות של מתכנתים, מתפרסמים ספרי לימוד וספרי עזר, ויוצאים לאור עיתונים מקצועיים. פיתוח והטמעה של שפה חדשה הם תהליכים קשים לא רק עקב הקושי הטכני בעיצוב ובמימוש השפה, אלא גם בגלל הקושי בהוראת שפה חדשה ובצורך לתמוך במשתמשים. המאפיינים הטכניים אינם מנבאים איזו שפת תכנות תצליח לכבוש נתח שוק משמעותי ואיזו שפה תיעלם. מתוך הסתכלות על ההיסטוריה של שפות תכנות ניתן לראות שבמקרים רבים הסיבות להצלחה או לכישלון היו קשורות למקום ולזמן שבו פותחה שפה, ומי היו משתמשיה הראשונים. תכנות מונחה עצמים החל עם Simula (1967) שנוצרה בנורווגיה. אולם דווקא שפת Smalltalk (1972-1980) חשפה סגנון זה לקבוצת רחבה יותר של מתכנתים. אולם, כיוון ששפה זו מבוססת מפרש (interpreter) ולא מהדר, ומשום שסביבת הפיתוח הגרפית דרשה משאבי חישוב שלא היו זמינים לכולם, השפה לא זכתה להצלחה רבה כשפת פיתוח כאשר היא פותחה לראשונה. השפה C++ ירשה מ-Simula את הרעיונות של תכנות מונחה עצמים, אולם ירשה תכונות רבות גם משפת C, שאולי החשובות שבהן היא התמיכה בהידור יעיל, והתחביר שהיה מוכר למתכנתים רבים. C++ (1983) הפכה לשפה פופולרית, וחשפה מתכנתים רבים לעקרונות התכנות מונחה העצמים.

★תרגיל 8.2 כתבו בשפת תכנות כלשהי שאתם מכירים פונקציה שאינה referentially transparent, כלומר פונקציה שניתן להפעילה פעמיים עם אותו פרמטר ולקבל תוצאות שונות.

★תרגיל 8.3 אופטימיזציה נפוצה במהדרים היא להחליף פעולות יקרות, כגון כפל, בפעולות שקולות זולות יותר, כגון חיבור. האם מהדר של שפה אימפרטיבית יכול תמיד להחליף את הביטוי $2 * f(x)$ בביטוי $f(x) + f(x)$?

האם מהדר של שפה אימפרטיבית יכול, בהינתן הביטוי $f(x) + f(x)$, ליצור קוד יעד שמחשב את $f(x)$ פעם אחת בלבד?

★**תרגיל 8.4** גם כאשר מתכנתים בשפות אימפרטיביות מקובל לממש פונקציות ללא תוצאי לוואי (side effects) כסגנון תכנות מועדף. מדוע?

★**תרגיל 8.5** תנו שתי דוגמאות לשפות קטנות, המעוצבות באופן יעודי לצורך פתרון סוג מסוים של בעיות.

★**תרגיל 8.6** מנגנון הדופס (templates) של C++ הושפע ממנגנון היחידות הג'נריות בשפת עדה. אם מוכרות לכם שתי שפות אלו, השוו בין עיצוב מנגנון זה בשתי השפות.

★**תרגיל 8.7** מנגנון החריגות של C++ הושפע ממנגנון החריגות בשפת עדה. אם מוכרות לכם שתי שפות אלה, השוו בין עיצוב מנגנון זה בשתי השפות.

★**תרגיל 8.8** נסו לקבוע היכן יש למפות את שפת תיאור HTML המשמשת לבניית אתרים ברשת, בהתאם לאפשרויות שתוארו. האם הייתם מגדירים את HTML כשפת תכנות?

סוף דבר

מדריך למידה זה ליווה אתכם במהלך הלימוד של הקורס "שפות תכנות". בקורס מימשנו מפרשיים לכמה שפות תכנות קטנות, ובהן:

- שפת "ויהי"
- שפת "וישגור"
- שפת "וייסוג"
- שפת "ויפנה"
- שפת "וירמוז"
- שפת "ויבגוד"
- שפת "ויבדוק"
- שפת "ויקיש"

נותר עוד לעבור בהצלחה את שפת "ויבחן" - וחסל.

Revision : 2.16.10

יחידה II

מדריך לשפת Scheme

פרק 9

מדוע שפת Scheme?

בין אנשי חינוך לאנשי מדעי המחשב קיימים באופן תמידי חילוקי דעות לגיטימיים לגבי איזו שפת תכנות מתאימה יותר להוראה של נושא מסויים. אולם, תחום מדעי המחשב נבחן באורח רוח לוגי, המתווה כללים שבעזרתם ניתן להסיק מסקנות ולבסס טענות המושתתות על תורת ההיגיון. ראשית, נסקור מספר מאפיינים של שפות תכנות, שגם בעניינם יש כמובן חילוקי דעות. אחרי כן, ננמק את הבחירה לא מחויבת המציאות של שפת התכנות לקורס זה.

9.1 דרישות משפת תכנות

פשטות. שפה צריכה להיות מורכבת ממספר מינימלי של מושגים ומרכיבים.

אחידות. המושגים היסודיים צריכים להיות מיושמים באופן עקבי ואוניברסלי. צריך שיהיה אפשר להשתמש במבנים בשפה בהקשרים שונים, מבלי צורך לשנות את צורתם. חוסר אחידות יכול להיות מטריד. בפסקל, למשל, לא ניתן לאתחל את ערכו של קבוע בעזרת ביטוי אלא צריך לכתוב ערך, זאת בעוד שבכל מקום אחר בשפה שבו צריך לתת ערך ניתן להשתמש בביטויים. חוסר אחידות יכולה

Revision : 2.16.22

להביא לטעויות.

אורתוגונליות (orthogonality). תכונות לא תלויות צריכות להיות נשלטות על-ידי מנגנונים לא תלויים. במתמטיקה וקטורים לא תלויים נקראים אורתוגונליים. למשל, ניתן לטעון שהעובדה שבשפת C++ מבנה המחלקה משמש הן לחלוקת המערכת לרכיבים והן למנגנון הירושה, מהווה פגיעה באורתוגונליות של השפה.

הפשטה. צריכה להיות דרך להפריד בין העיקר והטפל.

בהירות. רכיבי השפה צריכים להיות מוגדרים היטב, והתוצאה של קטע קוד צריכה להיות צפויה. מתכנתים צריכים להיות מסוגלים לקרוא קוד בשפה ולהבינו בקלות. יש המבקרים את שפת C, למשל, על כך שקל להתבלבל פה בין השמה ובדיקת שיוויון.

הסתרת מידע. יחידות תוכנה צריכות להיות מסוגלות להתייחס רק למידע שהן זקוקות לו, למשל תוך הסתרת פרטים בקופסה שחורה, שניתן להתייחס אליה רק דרך ממשק מוגדר היטב.

מודולריות. ממשקים בין יחידות תוכנה צריכים להיות מאופיינים בצורה מפורשת.

בטיחות (safety). טעויות סמנטיות צריכות להיות ניתנות לזיהוי, רצוי בזמן הידור. כלומר, לפני זמן הרצה. דוגמה לתכונה זו היא השימוש בטיפוסים בשפות, כדי למנוע פעולות בין נתונים לא מתאימים – ואילו המתכנת לבצע הסבה מפורשת בין טיפוסים כאשר הוא מעוניין בכך.

יכולת הבעה. מגוון רחב של תכניות צריכות להיות ניתנות להבעה בשפה. מרבית השפות הן Turing-complete, וניתן לממש בהן כל אלגוריתם. הכוונה כאן היא לקלות שבה ניתן לתאר את האלגוריתמים השונים.

יעילות. צריכה להיות אפשרות להפיק מהשפה קוד יעיל. שפות פונקציונליות המשתמשות ברקורסיה יכולות לסבול עקב מימוש לא יעיל של רקורסיה.

בחירת שפת תכנות תלויה בחלקה בסוג התכנות הדרוש ובחלקה בגורמים חיצוניים כגון זמינות כלי פיתוח, תמיכה והכשרה.

★**תרגיל 9.1** בחרו שפת תכנות כלשהי, כגון C ופסקל, ובדקו כיצד היא עומדת ביעדים שפורטו לעיל.

9.2 נימוקים כבדי משקל

להלן שיקולים מספר שהניעו אותנו לבחור ב־ Scheme לצורך קורס זה.

- ספר הלימוד שנבחר, שהינו ספר מוביל בתחום, משתמש בשפת Scheme. זהו, כמובן, טיעון לוגי מעגלי במקצת, שכן כותבי הספר בחרו ב־ Scheme בין היתר בגלל הסיבות המפורטות כאן.
- שפת Scheme מבוססת על תחשיב למדא. תחשיב למדא הוא פורמאליזם מקובל בחקר שפות תכנות, והיכרות עמו היא אחת ממטרות הקורס.
- שפת Scheme תומכת בסגנון תכנות פונקציונלי המתאים לסוג התכניות הנכתבות במהלך הקורס (מפרשים של שפות).
- שפת Scheme מספקת ניהול אוטומטי של זיכרון (garbage collection), מה שחוסך מאמץ בפיתוח תכניות. בזכות זאת ניתן להתמקד בנושאי לבח של שפות תכנות ופחות במימוש.
- ככלל, הקוד הנדרש לצורך יישום המפרשים שנכתוב בשפת Scheme הוא קצר ותמציתי. לכן קל יותר להבנה, וחוסך זמן תכנות שיגזול מזמן הלימוד.

Revision : 2.16.22

• ישנן סביבות פיתוח ידידותיות וזמינות עבור Scheme. זאת בהשוואה לחלק מהשפות האחרות בעלות אותן התכונות הטכניות.

• רבים מאוד הקורסים על שפות תכנות בעולם המשתמשים בשפת Scheme. שימוש ב־ Scheme מאפשר לנו לנצל חומרי לימוד שלהם, ומאפשר לסטודנטים למצוא חומר עזר רב.

רשימת שיקולים אלו היא חלקית אך ממחישה את מורכבות השיקולים שעולים בתהליך בחירת שפת תכנות.

★**תרגיל 9.2** בחנו ארוע כלשהי מניסיונכם שבו היה צורך לבחור שפת תכנות, ונסו למפות את השיקולים שהשפיעו על ההחלטה.

★**תרגיל 9.3** האם השיקולים שתפעילו בבחירת שפה עבור תכנית קטנה זהים לשיקולים שתפעילו בבחירת שפה לפרויקט תוכנה גדול? אם ישנם הבדלים, מהם?

9.3 הכרת תודה

מקצת החומר בחלק זה של המדריך הֵדן בשפת Scheme שוכתב והורחב מרשימות של אהוד לם ואורי ברוסט.

פרק 10

פגישה עם סביבת הפיתוח Racket

פרק זה מתאר את סביבת הפיתוח.



ראשית, התקינו את התוכנה לפי ההנחיות באתר הקורס.

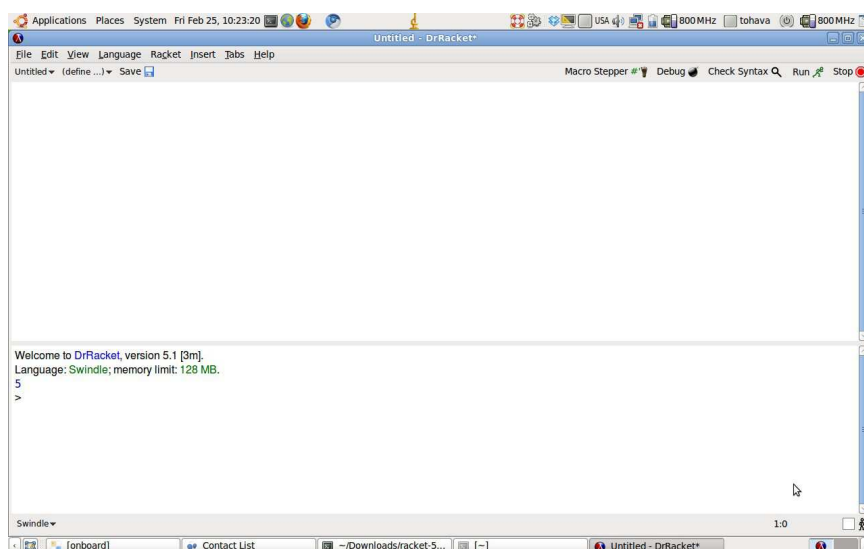
10.1 תפעול בסיסי של Racket

לאחר שהתקנתם את Racket על המחשב שלכם והפעלתם את התוכנה דרך תפריט התחל, יתקבל החלון שמוצג באיור 10.1.

נתאר את החלקים בחלון של Racket אשר נשתמש בהם במהלך עבודתנו:

1. **תיבת האינטראקציה (Interactions)** - בתיבה זו ניתן לשלוח הוראות אשר מבוצעות על ידי Racket באופן מיידי (כלומר, ברגע הלחיצה על מקש Return). נוסף על כך, תיבת האינטראקציה היא המקום שאליו התכנית המורצת מדפיסה הודעות ומקבלת קלט מהמשתמש. תיבה זו לא נועדה לכתיבת תכניות שלמות.

Revision : 2.16.24



איור 10.1: סביבת הפיתוח RacketRacket

2. **תיבת ההגדרות (Definitions)** - בתיבה זו ניתן לכתוב רצף של הוראות אשר יורץ אך ורק כאשר ילחץ כפתור ההרצה (Run). תיבה זו נועדה לכתובת תכניות שלמות ואת תוכנה ניתן לשמור בעזרת בחירה ב־ File->Save.
3. **כפתור ההרצה (Run)** - לחיצה על כפתור זה תמחק מהזיכרון את כל המשתנים בזיכרון של Racket, לאחר מכן תריץ את התכנית בתיבת ההגדרות (Definitions).

10.2 הרצת תכנית פשוטה

עתה, נכתוב תכנית פשוטה. לחצו בעזרת העכבר של תיבת ההגדרות, וכתבו בה:

```
(define x (* 5 (+ 1 6)))
```

לחצו על כפתור ההרצה ולאחר מכן כיתבו בתיבת האינטראקציה

(display x)

בתיבת האינטראקציה יופיע המספר

35

אם מילאתם אחר ההוראות לעיל, כתבתם והרצתם את תכנית הראשונה שלכם.

◆ שימו לב: לאחר הרצת התכנית ניתן לעשות למשתנה x זריק חלון האינטראקציות ראוי לצייין שהוראת (display x) אינה ספציפית לתיבת האינטראקציה וניתן היה לכתוב אותה גם בשורה שנייה בתיבת ההעזרות

פרק 11

פגישה עם שפת Scheme



פרק זה מתאר את העבודה עם Scheme בסביבת הפיתוח.

11.1 צורות מיוחדות וצורת הפעלה

אלמנט בסיסי וחשוב בתכניות הוא הביטוי (expression). נסתכל על הביטויים בשורה הראשונה בתכניות הקצרה שכתבנו:

```
(define x (* 5 (+ 1 6)))
```

השורה לעיל הינה ביטוי. ביטויים אשר מתחילים ב־define (אך לא רק הם) נקראים צורות מיוחדות (special forms). ביטוי מהצורה המיוחדת הזו מקבל תמיד שני פרמטרים. הפרמטרים נכתבים לאחר מילת המפתח define. במקרה שלנו, הפרמטרים הם: x ו־ $(* 5 (+ 1 6))$.

שימו לב שטקסט סגור בסוגריים נחשב כולו לפרמטר אחד. הפרמטר הראשון שלנו

Revision : 2.16.26

x איננו ביטוי כיוון ש־define הוא אינו ביטוי (נסביר זאת בהמשך כאשר נדבר על מה define עושה). הפרמטר השני שלנו $((6 + 1) * 5)$ הוא כן ביטוי, אשר מתחיל בסימן $*$.

הסימן $*$ אינו מזהה צורה מיוחדת. כאשר ביטוי אינו צורה מיוחדת, הוא נקרא **צורת הפעלה** (application form). על ההבדל בין צורת הפעלה לצורה מיוחדת נדבר בהמשך. מאחר ולצורת ההפעלה אין סימן מזהה קבוע (למשל, גם $+$ וגם $*$ מתחילים צורות הפעלה במקרה שלנו) אנחנו חושבים גם על $*$ בתור פרמטר של צורת ההפעלה. לצורת ההפעלה שלנו אם כך, יש שלושה פרמטרים: אופרטור $*$, ושני אופרנדים: 5 ו־ $((6 + 1))$.

המשמעות של צורת ההפעלה היא: "הפעל את השגרה שמתאר הפרמטר הראשון (האופרטור) על שאר פרמטרים". במקרה שלנו, בצע כפל בין 5 לבין $((6 + 1))$. שימו לב שכל הפרמטרים בצורת ההפעלה הם גם ביטויים. הסימן $*$ הוא ביטוי המתייחס לפעולת הכפל. 5 הוא ביטוי המתייחס למספר חמש. $((6 + 1))$ הוא צורת הפעלה נוספת.

◆ שימו לב: ביטויים אשר אינם מתחילים בסוגריים הם לרוב ביטויים המכילים אלף ערך של משתנה מסוים (לדוגמה $+$ ו־ $*$ הם משתנים) או מייצגים ערך מסוים (5 מייצג ערך, "Hello" מייצג מחרוזת בה כתוב שלום באנגלית), ואינם נזקקים לפרמטרים. ◆

11.2 תחביר של ביטויים

עבור ביטויים המקבלים פרמטרים, כלומר, מתחילים בסוגריים, יש תחביר (Syntax) אשר מגדיר מה על הפרמטרים להיות. נתאר את התחביר של הצורה

```
(define <SYMBOL> <EXPRESSION>)
```

הפרמטר הראשון של ה-`define` חייב להיות סמל (Symbol) או רשימת סמלים, אשר אותם עלינו לכתוב בתוך סוגריים, אחרת כל אחד מהם יחשב לפרמטר נפרד. סמל הוא כל רצף של תווים המקיים תכונות מסוימות (למשל: אין ביניהם רווח ולא כולם ספרות). פרמטר השני של ה-`define` חייב להיות ביטוי (Expression). שים לב שתי הדוגמאות הבאות:

```
(define x y)
```

```
(define y x)
```

בשורה הראשונה, `y` הוא ביטוי ו-`x` אינו ביטוי. בשורה השנייה ההפך הוא הנכון. משמע, הבחירה בהינתן פרמטר מסוים `A` של ביטוי `B`, אם להתייחס אל `B` כביטוי, היא בחירה התלויה ב-`A`.

נסתכל על התחביר של ביטוי נוסף:

$$\langle \text{EXPRESSION}_0 \rangle \langle \text{EXPRESSION}_1 \rangle \dots \langle \text{EXPRESSION}_N \rangle$$

בניגוד לתחביר הקודם, בתחביר זה אנו יכולים לראות שאין שום חלק בביטוי שחייב להיות קבוע מסוים (בניגוד לביטוי הקודם שחייב להתחיל ב-`define`) משום כך, ניתן לזהות כי מדובר בצורת ההפעלה ולא באחת מהצורות המיוחדות. בנוסף, ניתן לראות כי בניגוד לביטוי הקודם, בביטוי זה מספר הפרמטרים אינו קבוע. מספר דוגמאות לביטויים של צורות הפעלה הם:

```
(x y z (+ 5 3))
```

```
(z (y (x a b c) d) e)
```

על מנת לבדוק את הבנתכם מומלץ לבדוק האם ביכולתכם למנות את מספר צורות ההפעלה ומספר הפרמטרים של כל אחת מהן בביטויים לעיל.

✦ שימו לב: יתכן שהסקרנים מביינים ניסו לכתוב ביטויים כגון

```
(define x (define y 5))
```

על-פי כללי הדקדוק שכתבנו ביטוי זה אמור להתקבל כחלק מהשפה מכיוון ש- x הוא סמל והביטוי `(define y 5)` תקין. אך ניסיון להריץ ביטוי זה יוביל לשגיאה. הסיבה לכך היא שהתחביר שהצלנו כאן איננו התחביר המדויק אלא גרסה פשוטה של התחביר למטרות הכירות עם השפה. ✦

11.3 סמנטיקה

הגדרות התחביר של ביטוי לא אומרות דבר על "מה הביטוי עושה?". לתשובה לשאלה זו אנו קוראים משמעות או "סמנטיקה". הדרך הכי פשוטה להמחיש את ההבדל בין תחביר למשמעות היא בעזרת הדגמת ההבדל בין שגיאה סמנטית לשגיאה תחבירית. אך לפני שנעבור על הדוגמה, עלינו להכיר חלק נוסף במנשק של סביבת הפיתוח: כפתור בדיקת התחביר (Check Syntax).

כפתור בדיקת התחביר גורם לסביבת הפיתוח לבדוק האם תחביר התכנית הכתובה בתיבת ההגדרות תקין. יש לשים לב שפעולה זו אינה דורשת את הרצת התכנית, כלומר, לבדיקת התחביר אין צורך לדעת מה משמעות הביטויים. אם תחביר התכנית תקין, לא תופיע שום הודעה וצבעי הטקסט ישתנו מעט. אם תחביר התכנית שגוי, תופיע הודעת שגיאה. בנוסף על כך, לחיצה על כפתור בדיקת התחביר תעלים את תיבת האינטראקציות. הסיבה לכך היא שתיבת האינטראקציות נועדה להשתמש במשתנים ובתוצאות של תכנית אשר הורצה, ובדיקת תחביר אינה מריצה את התכנית. לחיצה על כפתור ההרצה תחזיר את תיבת האינטראקציות למקומה.

11.4 בדיקת תחביר

עתה, נסתכל על התכנית הבאה:

```
(define x "foobar")

(define y 2785)

(+ x y)
```

התכנית לעיל נכונה מבחינה תחבירית, כפי שתראה לנו לחיצה על כפתור בדיקת התחביר. אולם, ניסיון להריץ אותה ייתן לנו את השגיאה

```
+: expects type <number> as 1st argument
```

הסיבה לשגיאה היא שהפונקציה `+` היא פונקציה אשר נועדה לחבר בין שני מספרים. כאשר פונקציה זו מקבלת מחרוזת `foobar` כפרמטר, היא לא מסוגלת לפעול בצורה תקינה ולכן מדווחת על שגיאה סמנטית. מתוך מבט על הביטוי `(+ x y)`, הוא נראה תקין לחלוטין, רק כאשר התכנית מורצת `x` ו-`y` מקבלים את ערכיהם ואז ניתן לראות את הבעייתיות שבביטוי.

דוגמה זו מראה לנו את ההבדל בין תחביר לסמנטיקה:

- תחביר אומר לנו איך הביטויים בשפה נראים, שגיאת תחביר מופיעה בביטוי שלא כתוב בצורה נכונה. בעברית דוגמה לשגיאה כזו היא "אני כתיבה ספר", זוהי שגיאה תחבירית שעשויה לנבוע מהניסיון לכתוב "אני כותב ספר".
- סמנטיקה אומרת לנו מה המשמעות של ביטוי, שגיאת סמנטיקה מופיעה בביטוי שמשמעותו אינה הגיונית. בעברית דוגמה לשגיאה כזו היא מילות השיר: "הילד

Revision : 2.16.26

בן 30, יש לו חום גבוה", זאת מכיוון שמשמעות המילה "הילד" אינה מתייחסת בדרך כלל לאדם בן 30 (ברוב המקרים לפחות).

◆ שימו לב: אחת מהשאלות שניתן לזהות היא האם אפשר בעזרת תחביר נכון למנוע אחרוּטין שליליות סמנטיות? למשל, בשפת C לכל משתנה יש טיפוס וכך ניתן לקבוע למשל כחלק מהתחביר שמשתנה מסוים הוא מספר. התשובה היא שבמקרה הכללי לא ניתן לעשות זאת זאת כיוון שהזריזות הסמנטיות יכולות להיות במקרים מסוימים בלתי ניתנות לבדיקה בעזרת מחשב. דוגמה לזריזה כזו היא שהפונקציה לא "תיתקע". תבונה זו היא זריזה שהוכח מתמטית שלא ניתן לבדוק. היא קשורה למושג "בעיית העצירה", אשר נלמדת בקורס חישוביות. ◆

פרק 12

המפרש של Scheme

בפרק זה נציג את הבסיס לתכנות ב־Scheme. פרק זה אינו תיאור פורמלי של השפה, שאותו ניתן למצוא במסמך RS6R, אלא מדריך להכרת השפה. עיקר הלימוד הוא מעשי, ומבוסס על התרגילים לאורך הפרק. במקביל ללימוד הפרק, יש לעיין בהגדרות הפורמליות ב־RS6R של כל אחד מהביטויים שיוצגו. כמו־כן במהלך פתרון התרגילים, עליכם להיעזר ב־RS6R על־מנת למצוא פונקציות עזר וכד'.

12.1 המפרש

בשונה משפות אחרות שסביר שנתקלתם בהן, שפת Scheme אינה מאורגנת סביב פקודות (statements) אלא סביב ביטויים בעלי ערך (expressions). צורת העבודה הבסיסית עם Scheme היא בסביבה של מפרש (interpreter), המקבל בלולאה ביטויים, שיכולים גם לכלול הגדרת משתנים ושגרות, ומחשב ומדפיס את ערכם. תהליך זה נקרא ה־read-eval-print loop. המילה eval היא קיצור של evaluation, כלומר חישוב ערך של ביטוי. צורת עבודה אינטראקטיבית זו אינה הכרחית: ניתן לקרוא תכניות מקבצים, ואף להדר תכניות בעזרת שימוש במהדר Scheme. להלן דוגמה פשוטה של "שיחה"

Revision : 2.16.23

עם מפרש Scheme.

לאורך כל הדוגמאות שורות הקלט למפרש יתחילו בסימן $>$. תשובות המפרש מופיעות ללא ה־prompt בתחילת השורה:

```
> 3
3
> (+ 2 3)
5
> +
#<Procedure>
> (define x 3)

> x
3
> (+ (* x x) (* 9 9)) 90
180
```

נבחן את הדוגמה. בשורה הראשונה המפרש קיבל את המספר 3. זהו ביטוי חשבוני פשוט: ערך קבוע. המפרש "חישב" את ערכו, והדפיס את התוצאה.

הביטוי השני מורכב יותר. ביטוי זה הוא הפעלה של שגרה. ב־Scheme מופיעה השגרה המופעלת, תמיד, בתחילת הביטוי ("כתיב תחילי"). במקרה זה השגרה היא פעולת החיבור, המקבלת את הפרמטרים 2 ו־3. לאחר חישוב הביטוי, מדפיס המפרש את התוצאה 5. הביטוי הבא שהוצג למפרש היה $+$. $+$ היא שגרה, והמפרש לא יודע

להדפיס ערכים מטיפוס שגרה, ולכן מוצגת רק התשובה `<Procedure>` שמציינת שערך הביטוי הוא שגרה. מערכות Scheme שונות יכולות להציג תשובה שונה כדי לייצג מידע זה.

השורה הבאה שונה מכל השורות הקודמות. בשורות הקודמות סיפקנו למפרש ביטוי שאותו הוא חישב. מטרת השורה `(define x 3)` לתת ערך למשתנה `x`. אנו לא מעוניינים בערך של הביטוי, אלא בתוצאה של ביצוע הפעולה: שינוי ערכו של `x`. ביצוע השורה משנה את מצב המערכת, שכן מעתה התייחסות ל-`x` תחזיר את ערכו. ה-`define` אינו שגרה רגילה, אלא מבנה מיוחד בשפה המשמש להגדרות. מבנים כאלו נקראים `special forms`. ביצוע ה-`define` משנה או קובע את ערכו של המשתנה, וכפי שרואים אינו מחזיר ערך בשונה מביצוע של שגרה רגילה.

בשורה הבאה אנו מבקשים לראות את ערכו של `x`, והמפרש מראה שערכו הנוכחי הוא 3. הביטוי האחרון שסיפקנו הוא ביטוי חשבוני מורכב יותר. ודאו שאתם מבינים מדוע הוא כתוב כפי שהוא כתוב.

◆ לתרגיל הפעלו את המפרש, והעזרו בו לצורך פתרון השאלות
הבאות

★**תרגיל 12.1** מה התשובה שמחזיר המפרש שבו אתם משתמשים כאשר תיתנו לו כקלט את הביטויים הבאים: `define +`?

★**תרגיל 12.2** מה התשובה שמחזיר המפרש כאשר הוא מקבל שם של משתנה שטרם קיבל ערך על-ידי `define`?

★**תרגיל 12.3** מה יהיה ערכם של `x`, `y` אם לאחר ביצוע השורות לעיל, יקבל המפרש את השורה הבאה: `(define y x)`

★**תרגיל 12.4** מה יהיה ערכם של `x`, `y` אם לאחר ביצוע השורות לעיל, יקבל המפרש את השורה הבאה: `(define y 'x)`

12.2 רשימות ורקורסיה

מבנה המידע המרכזי ב Scheme הוא הרשימה. רשימה היא אוסף של פריטים, מוקפים בסוגריים. למשל:

```
() ;the empty list
(1 2 3)
(1 a 2 b)
(scheme is beautiful)
(1 (1 2) a b c)
```

ראינו כבר שביטויים בשפת Scheme הם עצמם רשימות, ואכן הזהות הזו בין מידע לקוד ביצועי, מאפשרת טכניקות תכנותיות וגמישות המאפיינים את שפת Scheme. כאשר מפרש Scheme נתקל ברשימה, הוא מצפה שערכו של האיבר הראשון בה יהיה שגרה, ושאר האיברים יהיו הארגומנטים להפעלת השגרה, כמו בדוגמה שכבר ראינו (+ 2 3). על־מנת למנוע ניסיון חישוב (evaluation) של הרשימה, מקדימים לה את התו ' (סימן זה נקרא quote). סימן זה גורם למפרש להתייחס לרשימה כאל קבוע או ליטרל, ולא כאל ביטוי. למשל:

```
> (+ 3 2)
5
> '(+ 3 2)
(+ 3 2)
```

עבודה עם רשימות מתבססת בעיקרה על שתי פעולות: פעולת `car` המחזירה את האיבר הראשון ברשימה, דהיינו את ראש הרשימה, ופעולת `cdr` המחזירה רשימה המכילה את כל האיברים פרט לראש הרשימה, דהיינו את זנב הרשימה:

```
> (car '(a b c))
a
> (cdr '(a b c))
(b c)
```

הדרך ליצור רשימה חדשה מראש רשימה וזנב, היא בעזרת הפעולה `:cons`:

```
> (cons 'a '(b c))
(a b c)
> (cons 'a '())
(a)
> ((car (cons + -)) 3 2) ; can you figure out what this expression means?
5
```

את הקשר בין שלוש הפעולות הללו ניתן לסכם כך:

Revision : 2.16.23

תמיד

$$(\text{car } (\text{cons } x \ y)) = x$$

$$(\text{cdr } (\text{cons } x \ y)) = y$$

אבל לא תמיד

$$(\text{cons } (\text{car } x) \ (\text{cdr } x)) = x$$

✦ שימו לב: הפרמטר השני ל `cons` לא חייב להיות רשימה. בפרק
הבא נסביר כיצד `cons` מתנהג כאשר הפרמטר השני אינו רשימה. ✦

★ **תרגיל 12.5** כתבו ביטוי המחזיר את האיבר השני של הרשימה `x`.

★ **תרגיל 12.6** בדקו מה קורה כאשר מפעילים את `car` ואת `cdr` על ביטוי שאינו רשימה.

★ **תרגיל 12.7** עבור הרשימה `((1 2) (3 4) 5)` שנשמנה `x`, קבעו מה תהיה תוצאת כל אחד מהביטויים הבאים, ובדקו את תשובתכם בעזרת המפרש:

$$(\text{car } (\text{car } x)) \text{ .a}$$

$$(\text{car } (\text{cdr } x)) \text{ .b}$$

$$(\text{cdr } (\text{car } x)) \text{ .c}$$

$$(\text{caddr } x) \text{ .d}$$

ביטוי זה יכול להירשם גם כך: `(caddr x)`

החלוקה הטבעית של רשימות לראש וזנב, וההגדרה הרקורסיבית של רשימות מתאימה מאוד לטיפול רקורסיבי. נוכל להגדיר, למשל, אורך של רשימה, באופן הרקורסיבי הבא:

$$(\text{list-length nil}) = 0$$

$$(\text{list-length } (\text{cons } x \ y)) = 1 + (\text{list-length } y)$$

כדי לתרגם הגדרה זו ל Scheme, אנחנו צריכים לדעת כיצד להגדיר שגרה, כיצד לבצע התניות, וכיצד לבדוק שרשימה היא ריקה:

```
(define list-length
  (lambda (lst)
    (if (null? lst)
        0
        (+ 1 (list-length (cdr lst))))))
```

ננתח את הגדרת השגרה. כזכור, `define` משמש לקביעת ערך של משתנה. במקרה זה, הוא מצמיד לשם `list-length` את השגרה שאנחנו מגדירים. למעשה, הביטוי `(lambda (lst) ...)` הוא הגדרת השגרה עצמה, ומכנה ה-`define` רק נותן לה שם. לכן נאמר לעתים ש `lambda` יוצרת שגרה אנונימית (ללא שם). הרשימה אחרי המילה `lambda` מכילה את שמות הפרמטרים של השגרה. למשל, ניתן להציג למפרש Scheme את הביטוי הבא:

```
> ((lambda (x) 0) 'a)
0
```

הפעלנו את השגרה האנונימית שיצרנו, על הארגומנט `'a`. השגרה מתעלמת מהפרמטר שלה, `x`, ומחזירה תמיד אפס, ולכן זו תוצאת הביטוי.

בהמשך נראה שניתן להשתמש ב `lambda` על-מנת להגדיר פונקציות עזר, על-מנת להחזיר שגרה כערך חזרה משגרה ועוד.

גוף השגרה `list-length` מורכב מתנאי אחד, הממומש על-ידי המבנה `if`, שניתן לתארו באופן כללי כך:

```
(if <test> <consequent> <alternate>)
```

אם חישוב ערכו של `test` הוא אמת (בהמשך נראה בדיוק כיצד מוגדרים ב `Scheme` ערכים בוליאניים), מוחזר ערך חישוב ה-`consequent`, אחרת מוחזר ערך ה-`alternate`. חלק ה-`alternate` הוא אופציונלי.

השגרה הפנימית `null?` בודקת האם רשימה היא ריקה. הפעלת השגרה `list-length` תיעשה באותה דרך בה מופעלת כל שגרה – תוך שימוש בכתוב תחילי:

```
> (list-length '(a b c))
```

```
3
```

★**תרגיל 12.8** תהגדירו שגרה בשם `mult` המקבלת רשימה של מספרים ומחזירה את מכפלתם. השגרה תחזיר את הערך 1 עבור רשימה ריקה.

★**תרגיל 12.9** הגדירו שגרה בשם `fact` המקבלת מספר ומחזירה את העצרת שלו. רמז: כדי לבדוק שמספר הוא אפס (תנאי העצירה), השתמשו בביטוי `(= n 0)`.

★**תרגיל 12.10** הגדירו שגרה בשם `fact-list` המקבלת רשימה של מספרים, ומחזירה רשימה המכילה את העצרת של כל אחד מהמספרים בארגומנט.

★**תרגיל 12.11** הגדר שגרה בשם `member?` המקבלת איבר ורשימה, ובודקת האם האיבר נמצא ברשימה. הדרכה: ערכים בוליאנים מסומנים על-ידי `#f`, `#t` (אמת ושקר, בהתאמה); בדיקת שיוויון נעשית על-ידי השגרה `equal?`.

★**תרגיל 12.12** הגדירו צמד פונקציות `even?` ו-`odd?`. `even?` תחזיר `#t` כאשר הארגומנט שלה יהיה מספר זוגי, ו-`#f` אחרת. `odd?` תחזיר `#t` כאשר הארגומנט שלה יהיה אי-זוגי, ו-`#f` אחרת. שתי הפונקציות יעבדו רק על מספרים. רמז: השתמשו בשגרה אחת על-מנת להגדיר את השנייה.

12.2.1 ביטוי Cond

במקרים רבים נראה שיש יותר מתנאי עצירה אחד לרקורסיה, או שיש צורך לקנן תנאים, זה בתוך זה, במבנה של `if-then-else`. על-מנת להקל את כתיבת קוד כזה (וקריאתו), קיים ב Scheme מבנה מיוחד.

```
(cond (<test> <expression>)
      (<test> <expression>)
      ...
      (else <expression>))
```

העיבוד מתבצע סדרתית מלמעלה. אם תנאי כלשהו מתקיים, מתבצע הביטוי

הצמוד לו, ומסתיים העיבוד של ה-`cond` כולו. אם אף אחד מהתנאים לא מתקיים יתבצע חלק ה-`else`. חלק זה הוא אופציונלי.

✦ שימו לב שהתנאים ב-`cond` יכולים להיות תנאים כלשהם, ואינם חייבים להיות קשורים זה לזה כמו במבנה `case` רגיל. ✦

נכתוב מחדש את `member?` בעזרת `cond`.

```
(define member?
  (lambda (a lst)
    (cond ((null? lst) #f)
          ((equal? (car lst) a) #t)
          (else (member? a (cdr lst))))))
```

✧ **תרגיל 12.13** הגדירו שגרה `remove-first` המקבלת איבר ורשימה, ומחזירה רשימה בעלת אותם איברים ובאותו סדר, פרט לכך שהמופע הראשון של האיבר מוסר מן הרשימה.

✧ **תרגיל 12.14** הגדירו שגרה `remove` המתנהגת כמו `remove-first`, פרט לכך שהיא מסירה את כל המופעים של האיבר.

✧ **תרגיל 12.15** הגדירו שגרה `firsts` המקבלת רשימה, שיכולה להיות ריקה או להכיל אוסף של רשימות לא ריקות. השגרה תחזיר רשימה, שתורכב מהאיברים הראשונים בכל אחת מהרשימות הפנימיות. בדקו כיצד מתנהגת השגרה שכתבת כאשר היא מקבלת פרמטר לא תקין.

★**תרגיל 12.16** הגדירו שגרה `insertR` המקבלת שלושה פרמטרים: שני איברים `old` ו `new`, ורשימה. השגרה מחזירה רשימה הזזה לרשימה המקורית, פרט לכך ש `new` מופיע צמוד מימין למופע הראשון של `old` ברשימה.

★**תרגיל 12.17** הגדירו שגרה `insertL` הזזה ל `insertR` פרט לכך שהאיבר `new` מוכנס משמאל למופע הראשון של `old`.

★**תרגיל 12.18** הגדירו שגרה `(subst old new lst)` המחליפה את המופע הראשון של `old` ב-`lst` באיבר `new`.

★**תרגיל 12.19** הגדירו שגרה `pick` המקבלת מספר `n` ורשימה, ומחזירה את האיבר במקום `n` ברשימה. אם האינדקס גדול מדי, תוחזר הרשימה הריקה.

★**תרגיל 12.20** הגדירו שגרה `rempick` המקבלת מספר `n` ורשימה, ומחזירה את הרשימה לאחר הסרת האיבר במקום `n`.

פרק 13

קריאה נוספת

13.1 כותרים של האו"פ

- דוד הרצל, אלגוריתמיקה, יסודות מדעי המחשב, האוניברסיטה הפתוחה, 1994

13.2 ספרות מומלצת ומקורות מקוונים

להלן מספר מקורות מידע מומלצים.

- Bentley Jon, Little Languages, in Bentley Jon, More Programming Pearls, Addison-Wesley, 1988.
- Hughes John, Why Functional Programming Matters, Computer Journal, 32:2, 98-107, 1989. <http://www.cs.chalmers.se/~rjmh/Papers/whyfp.html>
- Raphael Finkel, Advanced Programming Language Design, Addison-Wesley, 1996. <http://cseng.aw.com/book/related/0,3833,0805311912+20,00.html>
- Sethi Ravi, Programming Languages: Concepts & Constructs, 2nd edition, Addison-Wesley, 1996

Revision : 2.15.29

13.3 מקורות נוספים ללימוד Scheme

להלן מספר מקורות מידע מומלצים ללימוד שפת Scheme.

- Abelson H., Sussman G. J., Structure and Interpretation of Computer Programs, 2nd edition, McGraw Hill, 1996
- Dybvig R. K, The Scheme Programming Language, 2nd edition, Prentice Hall, 1996. <http://www.scheme.com/tspl2d>
- Friedman D. P., Felleisen M., The Little Schemer, 4th edition, MIT Press, 1996.
- Sitaram Dorai, Teach Yourself Scheme in Fixnum Days.
<http://www.cs.rice.edu/~dorai/t-y-scheme/t-y-scheme.html>

נספחים



פתרונות לתרגילים נבחרים

פתרון תרגיל מפרק המבוא

פתרון תרגיל 0.1

פתרון: (ג). 

□

פתרון תרגילים מפרק 4

פתרון תרגיל 4.3

פתרון: נגדיר (קוד 13.1): 

```
(define times  
  (lambda (x y)  
    (if (is-zero? x) (zero)  
        (plus y (times (predecessor x) y)))))
```

Listing 13.1: exe-times.scm

□

פתרון תרגיל 4.44

פתרון התרגיל הובא בסעיף 4.5 במדריך.

פתרון תרגיל 4.45

פתרון: נגדיר: 

```
;; parse-expression: SchemeVal -> LcExp
(define parse-expression
  (lambda (datum)
    (cond
      ((symbol? datum) (var-exp datum))
      ((pair? datum)
       (if (eqv? (car datum) 'lambda)
           (lambda-exp (caadr datum)
                        (parse-expression (caddr datum)))
           (app-exp
            (parse-expression (car datum))
            (parse-expression (cadr datum))))))
      (else (eopl:error 'parse-expression
                        "Invalid concrete syntax ~s" datum)))))
```

Listing 13.2: parse-expression.scm

ונגדיר:

```
;; unparsed-expression: LcExp -> SchemeVal
(define unparsed-expression
  (lambda (exp)
    (cases lc-exp exp
      (var-exp (id) id)
      (lambda-exp (id body)
        (list 'lambda (list id)
              (unparsed-expression body)))
      (app-exp (rator rand)
        (list (unparsed-expression rator)
              (unparsed-expression rand))))))
(eopl:error 'apply-env "No binding for ~s" search-var)))
```

Listing 13.3: unparsed-expression.scm

□

פתרון תרגיל 4.49

פתרון התרגיל מובא באתר הקורס באמצעות קוד תגובה מהירה (QR) הבא:



פתרון: 

□

פתרון תרגיל 4.50



פתרון: 

□

Revision : 2.16.55

פתרון תרגיל 4.51



פתרון: 



פתרון תרגיל 4.52



פתרון: 



פתרון תרגיל 4.53



פתרון: 



פתרון תרגיל 4.54



פתרון: 



פתרון תרגילים מפרק 5

פתרון תרגיל 5.9

פתרון: 

```
(value-of <<let f = proc(z) -(z,x) in (f 21)>> ρ) =
= (value-of <<(f 21)>>
  [f=(value-of <<proc(z) -(z,x)>> ρ)]ρ) =
= (value-of <<(f 21)>>
  [f=(value-of (procedure z <<-(z,x)>> ρ)]ρ) =
= (apply-procedure
  (value-of <<f>> [f=(procedure z <<-(z,x)>> ρ)]ρ)
  (value-of <<21>> [f=(procedure z <<-(z,x)>> ρ)]ρ))=
= (apply-procedure
  (procedure z <<-(z,x)>> ρ)
  21)=
= (value-of <<-(z,x)>> [z=21]ρ)
= (- (value-of <<z>> [z=21]ρ)
  (value-of <<x>> [z=21]ρ))
= (- 21 10)
= 11
```

□

Revision : 2.16.55

פתרון תרגיל 5.10

פתרון: 

```

(value-of <<let x = proc(z) -(z,x) in (f 21)>> ρ) =
= (value-of <<(x 21)>>
  [f=(value-of <<proc(z) -(z,x)>> ρ)]ρ) =
= (value-of <<(x 21)>>
  [x=(value-of (procedure z <<-(z,x)>> ρ)]ρ) =
= (apply-procedure
  (value-of <<x>> [x=(procedure z <<-(z,x)>> ρ)]ρ)
  (value-of <<21>> [x=(procedure z <<-(z,x)>> ρ)]ρ))=
= (apply-procedure
  (procedure z <<-(z,x)>> ρ)
  21)=
= (value-of <<-(z,x)>> [z=21]ρ)
= (- (value-of <<z>> [z=21]ρ)
  (value-of <<x>> [z=21]ρ))
= (- 21 10)
= 11

```

□

פתרון תרגילים נוספים מפרק 5

פתרון תרגיל 5.21

 פתרון: קוד 13.4 מתאר את שורות הקוד שיש להוסיף ל-`parser` בקובץ

`.lang.scm`

```
(expression
  ("let*" (arbno identifier "=" expression) "in" expression)
  let*-exp)
```

Listing 13.4: `lang.scm`

קוד 13.5 מתאר את שורות הקוד שיש להוסיף לפרוצדורה `value-of`

בקובץ `.interp.scm`

```
(let*-exp (vars exps body)
  (if (null? vars)
    (value-of body env)
    (let* ((v1 (value-of (car exps) env))
          (e1 (extend-env (car vars) v1 env)))
      (value-of (let*-exp (cdr vars) (cdr exps) body) e1))))
```

Listing 13.5: `interp.scm`

□

Revision : 2.16.55



את מלוא הקוד ניתן להוריד מכאן:

פתרון תרגיל 5.22

 **פתרון:** קוד 13.6 מתאר את שורות הקוד שיש להוסיף ל- `parser` בקובץ

`.lang.scm`

```
(expression
  ("cond" (arbno expression "==>" expression) "end")
  cond-exp)
```

Listing 13.6: `lang.scm`

קוד 13.7 מתאר את שורות הקוד שיש להוסיף לפרוצדורה `value-of`

בקובץ `.interp.scm`

```
(cond-exp (exps1 exps2)
  (if (null? exps1)
    (display "error: cond error")
    (if (expval->bool (value-of (car exps1) env))
      (value-of (car exps2) env)
      (value-of (cond-exp (cdr exps1) (cdr exps2)) env))))
```

Listing 13.7: `interp.scm`

□



פתרון תרגיל 5.23

פתרון: קוד 13.8 מתאר את שורות הקוד שיש להוסיף ל־ parser בקובץ `.lang.scm` 

```
(expression
  ("fold" expression
    expression
    "[" (separated-list expression ",") "]" )
  fold-exp)
```

Listing 13.8: lang.scm

קוד 13.9 מתאר את שורות הקוד שיש להוסיף לפרוצדורה `value-of` בקובץ `.interp.scm`

Revision : 2.16.55

```

(fold-exp (exp1 exp2 exps )
  (letrec
    ((proc (expval->proc (value-of exp1 env)))
     (acc (expval->num (value-of exp2 env)))
     (lst (map (lambda (exp)
                 (expval->num (apply-procedure proc (value-of exp env))))
               exps))
     (sum-all (lambda (l)
                  (if (null? l) 0 (+ (car l) (sum-all (cdr l))))))
     (num-val (sum-all lst))))
  )))

```

Listing 13.9: interp.scm



פתרון תרגיל 5.24

פתרון: קוד 13.10 מתאר את שורות הקוד שיש להוסיף ל־ parser בקובץ

lang.scm

```

(expression
  ("check" expression (arbno "and" expression) "end" )
  check-exp)

```

Listing 13.10: lang.scm

קוד 13.11 מתאר את שורות הקוד שיש להוסיף לפרוצדורה value-of בקובץ interp.scm.

```
(check-exp (exp1 exps)
  (if (expval->bool (value-of exp1 env))
    (if (null? exps)
      (bool-val #t)
      (value-of (check-exp (car exps) (cdr exps)) env))
    (bool-val #f)))
```

Listing 13.11: interp.scm



פתרון תרגיל 5.25

 פתרון: קוד 13.12 מתאר את שורות הקוד שיש להוסיף ל- parser בקובץ lang.scm.

```
(expression
  ("count" expression
    "[" (separated-list expression ",") "]")
  count-exp)
```

Listing 13.12: lang.scm

קוד 13.13 מתאר את שורות הקוד שיש להוסיף לפרוצדורה value-of

Revision : 2.16.55

בקובץ interp.scm

```
(count-exp (prc exps)
  (letrec ((p (lambda (prc exps cnt)
    (if (null? exps)
      cnt
      (if (expval->bool (value-of (call-exp prc (car exps)) env))
        (p prc (cdr exps) (+ 1 cnt))
        (p prc (cdr exps) cnt))))))
    (num-val (p prc exps 0))))
```

Listing 13.13: interp.scm



פתרון תרגיל 5.26

פתרון: קוד 13.14 מתאר את שורות הקוד שיש להוסיף ל־ parser בקובץ

.lang.scm

```
(expression
  ("compose" "[" expression (arbno ";" expression) "]" expression)
  compose-exp)
```

Listing 13.14: lang.scm

קוד 13.15 מתאר את שורות הקוד שיש להוסיף לפרוצדורה value-of

בקובץ interp.scm

```
(compose-exp (firstexp exps val)
  (let* ((res (value-of (call-exp firstexp val) env))
        (resexp (var-exp 'res)))
    (if (null? exps)
        res
        (value-of (compose-exp (car exps) (cdr exps) resexp)
```

Listing 13.15: interp.scm



פתרון תרגילים מפרק 6

פתרון תרגיל 6.22

 **פתרון:** קוד 13.16 מתאר את שורות הקוד שיש להוסיף ל-`parser` בקובץ

`.lang.scm`

```
(expression
  ("setdynamic" identifier "=" expression "during" expression)
  setdynamic-exp)
```

Listing 13.16: `lang.scm`

קוד 13.17 מתאר את שורות הקוד שיש להוסיף בפרוצדורה `value-of`

בקובץ `.interp.scm`

```
(setdynamic-exp (var exp1 body)
  (let*
    ((varref (apply-env env var))
     (oldval (deref varref))
     (tempval (value-of exp1 env)))
    (begin
      (setref! varref tempval)
      (let ((result (value-of body env)))
        (begin
          (setref! varref oldval)
          result))))))
```

Listing 13.17: `interp.scm`

□



פתרון תרגיל 6.23

 **פתרון:** בשאלה זו נדרש לעדכן את הדקדוק המגדיר את אופן ההגדרה של פרוצדורה. קוד 13.18 מתאר את שורות הקוד החדשות עבור הגדרת ביטוי `proc-exp` שאותם יש לעדכן ב־ `parser` בקובץ `lang.scm`.

```
(expression
  ("proc" "(" identifier ")")
  (arbno "static" identifier "=" expression)
  expression)
proc-exp)
```

Listing 13.18: lang.scm

הפתרון מבוסס על בנייה של הסביבה שאיתה תשוערך הפרוצדורה, כך שתכיל כבר בזמן ההגדרה את המשתנים הסטטיים וערכן. קוד 13.19 מתאר את שורות הקוד שיש להוסיף לפרוצדורה `value-of` בקובץ `interp.scm`.

Revision : 2.16.55

```

(proc-exp (var statvars statvals body)
  (letrec
    ((vals (map (lambda (exp)
                  (value-of exp env))
                 statvals))
     (expand-env (lambda (vars vals env1)
                   (if (null? vars)
                       env1
                       (expand-env
                        (cdr vars)
                        (cdr vals)
                        (extend-env
                         (car vars)
                         (newref (car vals))
                         env1))))))
    (env-with-static (expand-env statvars vals env))))
(proc-val (procedure var body env-with-static))))

```

Listing 13.19: interp.scm

□



פתרון תרגיל 6.24

פתרון: קוד 13.20 מתאר את שורות הקוד שיש להוסיף ל־ parser בקובץ

.lang.scm

```
(expression
  ("callref" expression "(" "ref" identifier ")")
callref-exp)
```

Listing 13.20: lang.scm

קוד 13.21 מתאר את שורות הקוד שיש להוסיף לפרוצדורה value-of בקובץ interp.scm. שורות אלה משתמשות בפרוצדורת עזר בשם apply-procedure2 המתוארת בהמשך.

```
(callref-exp (rator rand)
  (let
    ((proc (expval->proc (value-of rator env)))
     (arg (apply-env env rand)))
    (apply-procedure2 proc arg)))
```

Listing 13.21: interp.scm

קוד 13.22 מתאר את פרוצדורת העזר apply-procedure2 שיש להוסיף לקובץ interp.scm.

```
(define apply-procedure2
  (lambda (proc1 arg)
    (cases proc proc1
      (procedure (var body saved-env)
        (let ((new-env (extend-env var arg saved-env)))
          (value-of body new-env))))))
```

Listing 13.22: apply-procedure2



פתרון תרגיל 6.25

 **פתרון:** קוד 13.23 מתאר את שורות הקוד שיש להוסיף ל־ parser בקובץ

.lang.scm

```
(expression
  ("while" expression "do" expression)
while-exp)
```

Listing 13.23: lang.scm

קוד 13.24 מתאר את שורות הקוד שיש להוסיף לפרוצדורה value-of בקובץ .interp.scm

```
(while-exp (bl body)
  (letrec
    ((do-body (lambda (c)
      (if (expval->bool (value-of bl env))
        (begin
          (value-of body env)
          (do-body (+ c 1)))
        c))))
    (num-val (do-body 0)))))
```

Listing 13.24: interp.scm



פתרון תרגילים מפרק 7

פתרון תרגיל 7.5



פתרון תרגיל 7.6



פתרון תרגיל 7.7



פתרון תרגיל 7.8



פתרון תרגיל 7.9



Revision : 2.16.55



מפתח

אָשׁוּשׁ	דְּקֻדָּק
רִיקְנוֹת, 37	חֲסֵר־הַקֶּשֶׁר, 41
אַקְסִיּוֹמָה, 41	הַגְדָּרָה
אוֹפֶרְנֵד, 73, 115	אֵינְדוּקְטִיבִּית, 33
אוֹפֶרְטוֹר, 73, 115	דוּקְטִיבִּית, 33
עֵץ	כִּלְלִי הַיֶּסֶק, 33
אַפְרוּט, 42	מִן הַמָּסַד מַעֲלָה, 33
גִּזִּירָה, 42, 68	מִהֲטָפְחוֹת מָטָה, 33
הֵיֶסֶק, 42	הֶנְחָה, 41
תַּחבִּיר מוֹפְשֵׁט, 91	הֶזְחָה, 38
עֵרֶךְ	הַכְרָזָה, 131
בַּהֲסַבַּת שְׁמָאֵל, 142	הַפְּקָה, 42
בַּהֲסַבַּת יָמִין, 142	הַפְּנִיָּה, 131
בֶּרֶאֱיַחְסוֹן, 142	הֶחֱסֵן, 142
בֶּרֶהֱבָעָה, 142	הַתְּנַהֲגוֹת, 31
עֲרִכִּים, 30	הַטְּפָסָה
בְּנָאִי, 46	הַדּוּקָה, 167
בִּטּוּחַת־טִיפּוֹס, 169	חִלְשָׁה, 168
דִּיאֶדָה, עֵינַי צֶמֶד	חֻזְקָה, 167
דָּפָס, 204	רוֹפְפָת, 167

היקף, 131	מנבא, 47
חתימה, 38	מרעע, 47
חזוה, 42	משמעות, 31
כריכה, 131	דינמית, 167
היקף, 132	סטטית, 167
דינמי, 132	משקיף, 64, 72
כלל היסק, 42	משתנה
כריכה, 79	גלובלי, 148
מבנה תחבירי מפשט, 31	חופשי, 38, 91
מחלץ, 72, 79	הגדרה, 39
מן המסד מעלה, 18	מימוש ב"ת ייצוג, 74
מנשק, 79	מוכרז, 131
מפרט, 79	מופנה, 131
מפרט אכלול אינדוקטיבי, 42	קשור, 115
מתחם, 42	מהטפחות מטה, 18
מנבא, 71	נתח תחבירי, 31
מארוזוג, 35, ע"ע צמד	פרדיגמה, 17
משקיף, 79	אימפרטיבי, 201
מפרש	לוגי, 203
גישת המפרשים, 17	מונחה עצמים, 204
מפרשים, 17, 209	פונקציונלי, 17, 202
מפרש, 17	פרמטר
תוכניות פרוש, 17	אקטואלי, 115
מאשש, 47	פורמלי, 115
מהדר, 31	פרדיקט,

תוצא-לווא, 30	ע"ע מנבה
תוצא חישובי, 141	קבוצה
תוצאה, 141	המספרים הטבעיים, 34
תג, 65	השלמים האי-שליליים, 34
תחביר, 31	קוד-לקוח, 79
מוחשי, 67, 80, 93, 113, 126, 147,	קידוד צ'רץ, 55
177	רפרור, ע"ע הפניה
מופשט, 67, 80, 93, 113, 126, 147,	רשימה
177	חישוב אורך של רשימה, 37
שפת ויבדוק, 177	סביבה, 79, 91
שפת ויהי, 93	ריקה, 65
שפת וישגור, 113	סמנטיקה, 31
שפת וייסוג, 126	שגיאת טיפוס, 169
שפת ויפנה, 147	שפת תכנות
תחביר אבסטרקטי, ע"ע תחביר מופשט	שפת ויבגוד, 160
תחביר מופשט	שפת וישגור, 113
טיפוס נתונים רקורסיבי, 66	שפת ויפנה, 142
תחביר קונקרטי, ע"ע תחביר מוחשי	שפת וירמוז, 152
תחשיב למדא, 69	שפת ויהי, 92
תחום	שפת וייסוג, 125
מתאר, 132	שפת ויבדוק, 176
קונטור, 132	שפת ויקיש, 178
תחום הכרזה, 132	שפת וייבחן, 209
תחום הכרזה, 30	שפות קטנות, 91
תחימה, 131	תוצא, 30, 141

142 ,L-value	חוקי תחימה, 30
142 ,R-value	סטטיים, 131
142 ,reference	לקסיקלית, 131
33 ,rule of inference	תג, 59, 80
35 ,S-list	טיפוס נתונים מופשט, 79
142 ,storable	צמד, ע"ע מאגרוג
142 ,store	צומדיים יתדיים, 47
59 ,symbol	צורה מיוחדת, 221
33 ,18 ,top-down	צורת BNF, 42
141 ,value	צורת הפעלה, 222
	ירושה, 204
	תרגיל, 19, 29, 30, 35, 36, 38, 39, 41,
	55-48, 57, 60, 64, 66, 69, 74,
	79, 81, 82, 84-87, 95-97, 103,
	104, 111, 117, 121, 124, 125,
	134-138, 143, 148, 152, 159,
	160, 162-165, 173, 174, 197-
	199, 204, 206, 207, 215, 216,
	229, 232, 234-237
	33 ,18 ,bottom-up
	75 ,define-datatype
	141 ,effect
	142 ,expressed
	34 ,grammar

Revision : 2.16.55



Listings

3.1	list-length.scm	38
3.2	occurs-free.scm	40
4.1	plus.scm	49
4.2	plus-unary-rep-1.scm	51
4.3	plus-unary-rep-2.scm	52
4.4	plus-binary-rep.scm	53
4.5	church.scm	56
4.6	church-test-1.scm	58
4.7	church-test-2.scm	58
4.8	env-example.scm	61
4.9	env-def.scm	63
4.10	env-report.scm	64
4.11	env-func.scm	65
4.12	env-alt.scm	66
4.13	lambda-cons.scm	70
4.14	lambda-pred.scm	71
4.15	lambda-extr.scm	73
4.16	lambda-occurs-free.scm	74

4.17	define-datatype.scm	76
4.18	occurs-free-cases.scm	77
4.19	parse-expression.scm	78
4.20	unparse-expression.scm	78
5.1	expval-datatype.scm	95
5.2	expval-num.scm	95
5.3	expval-datatype.scm	96
5.4	expval-error.scm	96
	code/value-of-expression-1.txt	98
	code/value-of-expression-2.txt	98
	code/value-of-expression-3.txt	99
	code/value-of-expression-4.txt	99
	code/value-of-expression-5.txt	99
	code/value-of-expression-6.txt	99
	code/value-of-expression-7.txt	99
	code/value-of-expression-8.txt	100
	code/value-of-expression-9.txt	100
5.5	let-the-lexical-spec.scm	104
5.6	let-the-grammar.scm	105
5.7	let-define-datatype.scm	106
5.8	let-expressed-values.scm	107
5.9	let-init-env.scm	108
5.10	let-run.scm	108
5.11	let-value-of-program.scm	109
5.12	let-value-of.scm	110

5.13 let-lang/tests.scm	111
5.14 let-lang/tests.scm	112
5.15 proc-the-grammar.scm	114
code/proc-example-1.txt	119
code/proc-example-2.txt	119
code/proc-example-3.txt	119
code/proc-example-4.txt	119
code/proc-example-5.txt	119
code/proc-example-6.txt	120
code/proc-example-7.txt	120
code/proc-example-8.txt	120
code/proc-example-9.txt	120
5.16 proc-procedure-impl.scm	122
5.17 proc-closure-impl.scm	123
5.18 proc-expval.scm	123
5.19 proc-value-of.scm	124
code/letrec-example.txt	128
code/letrec-example.txt	129
code/letrec-example.txt	129
code/letrec-example.txt	129
code/letrec-example.txt	129
code/letrec-example.txt	129
code/letrec-example.txt	130
code/letrec-example.txt	130
6.1 value-of-program.scm	148

6.2	store-impl.scm	149
6.3	store-impl.scm/newref	150
6.4	store-impl.scm/deref	150
6.5	store-impl.scm/set-ref!	151
6.6	store-interp.scm	152
6.7	implicit-refs/data-structures.scm/define-datatype	153
6.8	implicit-refs/data-structures.scm/expal-ζref	153
6.9	implicit-refs/data-structures.scm/define-datatype	154
6.10	implicit-refs/data-structures.scm	154
6.11	implicit-refs/data-structures.scm	155
6.12	implicit-refs/store.scm	156
6.13	implicit-refs/store.scm/get-store	156
6.14	implicit-refs/store.scm/reference?	157
6.15	implicit-refs/store.scm/newref	157
6.16	implicit-refs/store.scm/deref	158
6.17	implicit-refs/store.scm/setref	158
6.18	implicit-refs/store.scm/get-store-as-list	159
13.1	exe-times.scm	243
13.2	parse-expression.scm	244
13.3	unparse-expression.scm	245
13.4	lang.scm	249
13.5	interp.scm	249
13.6	lang.scm	250
13.7	interp.scm	250
13.8	lang.scm	251

13.9 interp.scm	252
13.10lang.scm	252
13.11interp.scm	253
13.12lang.scm	253
13.13interp.scm	254
13.14lang.scm	254
13.15interp.scm	255
13.16lang.scm	256
13.17interp.scm	256
13.18lang.scm	257
13.19interp.scm	258
13.20lang.scm	259
13.21interp.scm	259
13.22apply-procedure2	259
13.23lang.scm	260
13.24interp.scm	261

תיקון טעויות בספר הלימוד (Errata)

- Page 29: Exercise 1.33: Line 3: integer ==> number End of first display:
Add))) to close up (mark-leaves-with-red-depth ...
- Page 30: Exercise 1.35: Add))) to close up (number-leaves ...
- Page 124: Section 4.4: Line 5: newpair ==> make-pair
- Page 195: Replace next to last line of Figure 6.1 with (set! val (* saved-n
val))
- Page 196: Replace next to last line of Figure 6.2 with (set! val (* saved-n
val))
- Page 208: a-program ==> cps-a-program const-exp ==> cps-const-exp
var-exp ==> cps-var-exp
- Page: 209: The contract should be apply-procedure/k : Proc * ExpVal
* Cont -> ExpVal. Also, cps- should precede let-exp, letrec-exp, if-exp,
and call-exp.
- Page: 216: Line 10: (line 4 of the second display): The last paren should
be preceded by >>, leading to exp_n>> K) >>) Lines 9 and 14 of the

final display: The two closing parentheses should each be preceded by >> leading to >>) >>)

- Page 217: Line 7 of last display: There is an unmatched set of <<'s and >>'s. Need to move the >> to the next term following an equal sign and have it follow the K.
- Page 219: The type and name say it is for cps-of-diff-exp, but the code is for cps-of-sum-exp. So, we need the type: `cps-of-sum-exp : Listof (InpExp) * SimpleExp -> TfExp`
- Page 223: The type is missing the type of the letrec-body. It is, of course, just InpExp. Here is what the type should say. `;; cps-of-letrec-exp : Listof(Listof(Var)) * Listof(InpExp) * InpExp * SimpleExp -> TfExp`
- Page 230: Line -4: `apply-procedure ==> apply-procedure/k`
- Page 231: Line 2: `apply-procedure ==> apply-procedure/k`
- Page 265: Section 7.4.3: `optype ==> otype`